Generative Specification: A Pragmatic Programming Paradigm for the Stateless Reader

Author: Juan Carlos Ghiringhelli (Independent)
Version: 1.0
Date: March 2026
Status: Preprint
Contact: jcghiri@gmail.com · linkedin.com/in/jghiringhelli · github.com/jghiringhelli

Abstract

The ladder has been moving in one direction since the first compiler freed the engineer from machine code. Each step produced a more capable reader. Each more capable reader demanded a richer specification.

The dominant failure mode of AI-assisted software development is not incorrect code — it is architectural drift: structurally incoherent output produced at generation speed across sessions that share no persistent context. Each AI session starts stateless. Without an explicit, self-contained specification, intent degrades with every context boundary. This paper addresses that failure mode directly.

Generative Specification (GS) is the first programming discipline of the *pragmatic dimension*: the tier at which *derivability* — what a stateless reader can correctly determine from the artifacts alone — becomes a binding constraint. Where Robert C. Martin's paradigm sequence (structured, object-oriented, functional) constrains syntactic form, and the semantic disciplines (SOLID, TDD, DDD) constrain meaning for a contextual reader, GS constrains what can be derived by a reader carrying no accumulated context. That reader now exists at scale: a large language model that approximates, *by structural analogy*, what Chomsky classifies as context-sensitive reading — output whose meaning depends on surrounding context in ways no context-free parser can achieve — stateless by architecture, reading the specification as the only available instrument. *Paradigm* carries Martin's sense throughout: a discipline defined by what it removes from programmer freedom, not Kuhn's sense of a scientific revolution. Whether GS constitutes the latter is a determination for the community; the claim this paper advances is narrower — that GS occupies the pragmatic tier left vacant by prior disciplines — and is answerable by structural inspection. The seven properties that define a generative specification — Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, and Executable — operationalize this

constraint as a measurable artifact standard. A precisely stated use case simultaneously seeds an implementation contract, an acceptance test, and user documentation: three artifacts from one production rule, with test difficulty serving as the diagnostic for underspecification.

Empirical evidence across six production projects demonstrates consistent structural outcomes. SafetyCorePro: a brownfield takeover — 411 source files, zero unit tests — transformed in under 48 hours of active development to 174 changed files, 16,229 lines inserted, 484 tests covering a fully layered architecture. ForgeCraft: a developer tooling system built under the methodology it implements — 40 commits, 307 tests, a complete package rename executed in one commit with zero regressions. The six cases span five challenge categories (takeover, brownfield, greenfield, extension, migration), all executed by one engineer with AI assistance and all verifiable from public or reviewer-accessible repository history.

The paper defines the theoretical principle, presents the artifact grammar through which GS operates in practice, documents the empirical record, and presents one completed replication study (Rx, evidence committed) and one practitioner study scheduled for April 2026 (Dx). The multi-agent adversarial study (AX) — seven conditions (three pre-registered, four post-hoc), results incorporated below — tests derivation quality as a function of specification completeness under controlled conditions. The Replication Experiment (RX) independently derived and executed a scoped subset of the benchmark domain — user management, articles, profiles, and tags; comments and favourites are explicitly out of scope per the RX specification — using a fresh GS document, producing 104 passing tests, zero failures, across seven test suites against a live PostgreSQL instance; the evidence is committed to `https://github.com/jghiringhelli/generative-specification/tree/main/experiments/rx/` and is reproducible by any reader with an Anthropic API key. Reproducing the Rx experiment requires only the committed GS document and an API key — ForgeCraft produced the document, but the document is the reproducible artifact; the tool is not required for this verification step. The human practitioner experiment (DX), scheduled for April 2026 (40 developers, dual rubric), will test whether the methodology transfers to engineers other than its author; its design is stated in §7.7.A. The adversarial study provides the controlled condition the single-author practitioner cases cannot supply; the practitioner study will establish between-practitioner replication when completed. The paradigm characterization is advanced as a theoretical claim for community evaluation; both studies are designed to test it against its most exposed flanks. A structural corollary — the community convergence theorem — establishes that when a practitioner community contributes to a shared GS methodology under quality gates, the specification floor across all governed domains rises monotonically and, by construction, cannot retreat while quality gates hold, compounding across domains without conflict. The structural argument, grounded in five architectural properties of the methodology, is developed in §10; its implication extends beyond software to any domain in which human work can be reduced to organizable intent.

1. Introduction

In 1957, Noam Chomsky published *Syntactic Structures*, introducing a formal hierarchy of grammars that would define the theoretical limits of computation. In the same year, John Backus introduced the first high-level programming language, FORTRAN, freeing engineers from the mechanical expression of machine code. The parallel was not coincidence. It was convergence under the same structural pressure, felt independently in two domains.

The syntactic grammars of programming languages have almost universally been specified as context-free — the parsing problem has traditionally been treated as Type 2.[^1] Compilers are deterministic, pushdown-automaton parsers. The syntactic rigidity of programming languages was never an aesthetic preference. It was the constraint imposed by the parsing capability of the machines that read them. The compiler cannot tolerate ambiguity. A missing semicolon is a parse error. A misplaced bracket is a program failure.

The engineer adapted to the machine. Decades of practice, pedagogy, and tooling were built around writing code that a context-free parser could unambiguously interpret.

In 2017, Vaswani et al. published *Attention Is All You Need*, describing the transformer architecture that would eventually produce the large language models now reshaping software engineering practice. The models were born from the distributional hypothesis: J.R. Firth's 1957 observation that "you shall know a word by the company it keeps." Meaning emerges from context. Tokens are interpreted by their relationship to surrounding tokens across arbitrarily long windows. By structural analogy with Chomsky's hierarchy, this approximates context-sensitive reading: the model's output depends on surrounding context in ways no context-free parser's output can.[^2] Generative Specification compounds this probabilistic reading with deterministic structural gates — pre-commit hooks, CI rules, and TDD phase gates enforced through the Defended property (§4.3) — making incorrect output architecturally unreachable rather than merely statistically unlikely. The probabilistic and the deterministic together produce something closer to genuine context-sensitive enforcement than the model's distributional reading alone.

For the first time in the history of the profession, the machine that executes software development work reads at a higher level of the grammar hierarchy than the specifications it has traditionally been given. The appropriate response is not improved prompting; it is a new programming discipline that imposes structure on implicit context the way structured programming imposed structure on control flow. The migration case study in §7 makes the point at its sharpest: a broken implementation is, structurally, a complete specification with a bad executor. The methodology replaces the executor. That paradigm is **Generative Specification**.

Related work is positioned in §5, after the theoretical framework, because the independent contributions of Gordon (2024) and Thirolf (2025) can only be situated against the GS contribution after that contribution has been stated. Their work validates the problem formulation; evaluating the distance between their conclusions and the paradigm claim requires the claim to come first.

---

2. The Abstraction Ladder

Software engineering's history is a sequence of abstraction jumps, each requiring a more expressive reader than the one before.

Ascending from assembly to declarative configuration: Each step in the ladder moved the engineer further from machine expression and closer to intent. Assembly addressed registers directly. C introduced functions and control flow; the compiler translated. Object orientation introduced type systems, garbage collection, and polymorphism; memory management moved to the runtime. Declarative frameworks (Rails, Django, Spring) allowed configuration over construction; HTTP routing, ORM schemas, and authentication pipelines were *described*, not *built*. In each case: specification up, implementation down.

Large language models extend the ladder — reading context-sensitively, by structural analogy: The engineer can now describe intent in natural language, and the model produces implementation. But the model reads context-sensitively. It extracts meaning from everything surrounding the instruction: the file it is editing, the files that surround it, the names in scope, the history of commits, the architectural rules in the project's documentation. These are not optional enrichments. They are the grammar the model uses to determine what a valid sentence in this system looks like. We are at the next rung.

The direction was never accidental. Every major shift in the ladder is the same operation applied to a higher layer: take something previously prescribed as explicit execution steps, replace it with a declaration of intent, and let a more capable reader derive the execution. The engineer stopped managing registers when the compiler could derive machine code from expressions. The engineer stopped wiring object graphs when the framework could derive them from configuration. The engineer stopped writing route definitions when the framework could derive them from annotations. At each step, the field's intuition was consistent — *specify what, not how* — and each step required a reader whose expressive capability matched the richness of the new specification. Context-free parsers could read structured grammars; they could not read intent. The recurring pattern the field has been executing, intuitively or intentionally but without naming it, is: identify a layer where intent is still being prescribed as execution, find or build a reader capable of deriving execution from a richer specification, and remove the prescription.

Generative Specification names that pattern, applies it to the lifecycle layer — architecture, decisions, conventions, rationale — and derives the discipline that follows from the reader now available.

The ladder does not stop at structured text. §3 identifies prosody and paralanguage as the gap a written specification must currently close by hand. Instrumenting that channel — recording meetings, annotating transcripts with prosody metadata, analyzing individual presence signals from room cameras, distilling the annotated corpus into specification artifacts — is one direction the next rung points. At that rung, the specification act shifts from structured writing to natural articulation monitored by a pipeline.

---

3. The Theoretical Gap: From Context-Free to Context-Sensitive Practice

An AI coding assistant starts from the artifacts present. Within a session it holds a context window — a bounded, self-contained view of what it has been given — and that window does participate in shaping output: prior turns, loaded files, and injected rules all constrain what the model generates next. But the window resets at the session boundary, carries no institutional memory of decisions made in prior sessions (Tulving, 1972, 1985; Squire, 1987), is populated entirely by what the practitioner chose to feed it, and has no mechanism for deterministic judgment: every output is a probability distribution over possible continuations, not a reasoned decision from accumulated knowledge. It is a channel, not a memory or a mind. What the channel carries is itself a specification act — and if the artifacts fed into it are incoherent, the window amplifies the incoherence at generation speed. No persistent memory across sessions, no colleague to ask, no institutional context accumulated over months, no tolerance for a poorly named variable. A system whose coherence exists in the heads of a tenured team is, to the AI, an impoverished grammar. Where a human engineer *interprets* an underspecified requirement — compensating across the gap with memory, inference, and accumulated context — the AI *processes* what is present and generates output from it. The model's output is determined entirely by the coherence of what was externalized. The human compensation layer does not exist.

The missing context is of a specific kind: not an ambiguity in a present signal but an absence of institutional memory. The AI is not mishearing a tone of voice — it was not in the room when the decisions were made, and the transcript it reads is incomplete. The human language analog illuminates what that compensation layer consists of. Spoken natural language tolerates enormous formal ambiguity because it is accompanied by prosody (stress, rhythm, intonation) and paralanguage (gesture, expression, physical presence, shared interpretive context). A

colleague who says “that doesn’t feel right” communicates a precise technical concern through the combination of the words and everything the room provides to interpret them. The AI has none of this. The formal gap that prosody and paralanguage bridge in human dialogue is a gap that the specification must close explicitly — because the AI has only what was written, and the cost of ambiguity is not misunderstanding but drift, produced at generation speed.

This gap changes the cost function of imprecision. Imprecision given to a human produces misunderstanding: local, correctable, visible in the next conversation. The same imprecision given to an AI produces **drift**: implementation that is locally valid and tests-passing but architecturally incoherent, produced at the AI’s output speed and propagated across every subsequent session that inherits the corrupted context. The specification does not merely describe the system. Processing it *causes* the system to be what it becomes. That is the operational consequence that changes what a specification must be: not a specification designed for a human reader who compensates across gaps, but one designed for a stateless reader who cannot. That is the condition the pragmatic tier of programming discipline addresses. It is a condition that did not exist, at consequential scale, before 2017.

One apparent escape from this conclusion is an expanding context window: if the model can eventually read everything, the problem dissolves. The objection conflates capacity with content. An infinite window over an underspecified codebase is not infinite derivability — it is an infinite drift surface. The model reads more of the implicit record; it cannot derive intent that was never externalized. And the question of what to include in that window — which files, which sessions, in what order — is itself a specification act. GS answers it with a grammar. Without GS, curation is deferred to query time rather than resolved at build time, which is a worse trade at larger scale. A final point: the structural enforcement layer — commit hooks, CI gates, phase guards — is independent of context size. A model with unlimited memory still cannot prevent incorrect output from entering the codebase unless the discipline makes that output architecturally unreachable.

---

4. Generative Specification: The Principle

Two terminological notes before the argument. First: *paradigm* throughout this paper carries Robert C. Martin’s precise sense — a discipline defined by what it removes from programmer freedom, as structured programming removed `goto`, OOP removed unconstrained direct access to data, and functional programming removed assignment. This is not Kuhn’s sense of a revolution that reconstitutes a field’s foundational questions. Whether GS constitutes a Kuhnian paradigm shift is a community determination that awaits replication; whether it constitutes a Martin-sense paradigm is structural, answerable by inspection. Second: the semiotic tripartition used here — *syntactics*, *semantics*, *pragmatics* — is **Charles W. Morris’s**

(1938 *Foundations of the Theory of Signs*), not Peirce's. Peirce's semiotics, while influential on Morris, divides signs differently: icon, index, symbol. Morris formalized the three-term functional classification that maps to programming discipline categories. The pragmatic tier in this paper refers to Morris's definition precisely: the relation of signs to their interpreters *in context of use* — not 'pragmatic' in the colloquial sense of practical, and not Peirce's sign taxonomy.

4.1 The Mechanism

The pragmatic tier of semiotic analysis, as Morris articulated it, is the study of the relation of signs to their interpreters in context of use. A stateless reader — one carrying no accumulated institutional memory, no interpretive context built from shared history — cannot access this relation. Only the formal layer (the permitted constructs) and the denotative layer (the explicit meaning of tokens) remain available. A programming discipline of the pragmatic tier therefore consists of making derivable from artifacts what was previously accessible only through interpretive context. That is the obligation a changed reader makes structurally necessary.

4.1.a The Grammar Mechanism

We define a **Generative Specification** as a finite, coherent set of system artifacts sufficient to generate any valid implementation state of the system without requiring external human context.

The central structural property this definition names is *derivability*: a system's lifecycle layer is derivable when a stateless reader, given its artifact set alone, can correctly determine what should be built, where, why, and to what behavioral and architectural contracts — without requiring external human context. Derivability is what the pragmatic tier exists to protect, and what no prior discipline stated the obligation to provide.

The operative word is *valid* — and it carries exactly the meaning Chomsky gives it in generative grammar: a sentence is valid if and only if the grammar generates it. The grammar does not restrict what the machine can produce; it declares what counts as a correct sentence in *this system*. The grammar does not *constitute* what is valid — it formalizes what is independently known to be valid. A wrongly-specified grammar is therefore possible, and is the methodology's primary failure mode: a grammar that generates output conforming to its own rules while failing the system's actual behavioral and architectural obligations. This circularity — validity defined relative to the grammar that captures it — is not a flaw in the formalism; it is an irreducible property of any specification system. The defense against it is not a purer definition but a richer practice: the specification faces the same verification discipline as the implementation it governs. §8.10 develops this directly. This circularity is addressed in §8.10 where the completeness ceiling is grounded. The specification captures correctness; it does not produce it. An LLM generates according to its statistical model — a space orders of magnitude larger than any specific system

requires, producing code, documents, diagrams, CLI pipelines, generative art assets, infrastructure declarations, test suites, legal summaries, business strategies, and everything in between. Without a specification, any output drawn from that space is valid by definition, because nothing has ruled anything out. Generative Specification is the act of ruling things out — carving the correct subset out of the model's vast default distribution — and in doing so, making the outputs that remain derivable with precision. A generative specification is therefore the finite grammar from which the infinite implementation space of a software system can be correctly derived. The Chomsky hierarchy operates in this paper as structural analogy, not as strict formal type classification — but the analogy is load-bearing, not decorative. It supplies the vocabulary (valid sentence, grammar, derivation) without which the discipline cannot be precisely stated. Practicing GS means writing a grammar for a stateless reader: a finite rule set that generates all valid implementation states and rejects invalid ones. Morris then provides the classification: a discipline that constrains derivability for a stateless reader occupies the pragmatic tier, the one prior disciplines left vacant.

This framing carries its central structural argument across two axes in tension. The first is Chomsky's, pointing up: as readers gain expressive power, the specification required to use that power correctly must become richer. The second is Martin's, pointing down: each programming paradigm removes a degree of programmer freedom, increasing discipline and reliability. At their intersection, **constraint and generative capacity move in the same direction**. *Throughout this paper, paradigm carries Martin's sense — a discipline defined by what it removes from programmer freedom — not Kuhn's sense of a scientific revolution that reconstitutes the field's questions. Whether GS constitutes the latter is a determination for the community; that it constitutes the former is the structural claim this section advances, answerable by inspection of what the discipline requires and what it removes.* A specification with no constraints generates nothing correctly: any output is valid, which means every output is arbitrary. As constraints accumulate — naming conventions narrow the token space, architectural boundaries constrain where a class may live, ADRs close which decisions remain open — the set of valid sentences shrinks. But the AI's ability to derive *the correct sentence for a given requirement* grows. Every degree of freedom removed from the specification — every intent previously left implicit — becomes a surface on which the agent can operate without human guidance. *Restriction*, as used throughout this paper, denotes precisely this: an intent made structurally present in the artifact set, ruling out the class of outputs that would have been generated in its absence. The restriction *is* the expansion mechanism.

The practical implication is a single imperative: **assume nothing**. Every assumption is a gap in the grammar — a surface the stateless reader cannot parse and the agent will fill arbitrarily. The engineer who assumes the reader understands the team's naming conventions has not saved a line of specification; they have introduced a degree of freedom the agent will resolve incorrectly, at generation speed, across every session that inherits the result. An underspecified project is not

one where the engineer failed to write documentation. It is one where the engineer trusted compensating context that no longer exists. The discipline begins precisely at the decision not to assume.

4.1.b The Economic Consequence

The economic consequence of this structure is the methodology's most practically significant property: **the cost of iteration approaches zero**. An incomplete specification is not a project risk — it is the normal starting condition. When the output is wrong, the diagnosis is structural: a constraint is missing or imprecise. Adding it costs minutes — though the diagnosis itself, identifying *which* constraint is absent and formulating its correction, is the nonzero residual: bounded by the practitioner's domain fluency and the system's observability, not by coordination overhead. The pipeline reruns. The output changes. In prior development models, an incorrect requirement was paid for across the full sprint cycle — design, implementation, review, revision, redeployment — with coordination overhead multiplying the cost at every boundary. Here the gap between identifying what was wrong and seeing corrected output is measured in the time it takes to write one sentence. This compression collapses the agile sprint from weeks to hours, and often to minutes. Incomplete specifications arising from careless drafting, incomplete requirement gathering, or a creative process still in motion are all recoverable at negligible cost. The methodology does not require a perfect specification to begin; it requires only the discipline to tighten the specification when the output reveals the gap. The process is self-correcting by design: each iteration makes the grammar more precise, and a more precise grammar generates better output, which reveals the next gap. The specification builds its own path forward. For the engineer, architect, or founder who carries a hundred projects and not enough lifetimes to build them, this changes the calculus entirely: the friction that previously made partial ideas unbuildable — the overhead of translating an incomplete vision through layers of human coordination — is no longer the bottleneck. The specification is.

The portfolio-scale implication follows directly. If iteration cost approaches zero per project, the constraint on how many projects a practitioner can carry concurrently shifts from execution capacity to specification bandwidth — the rate at which intent can be correctly externalized. The operative model is not strict parallelism but cycling: at any screen session, six or seven projects are active; the others are in a waiting state — a session has produced output that needs review, a deploy is running, or the next direction has not yet been provided. A project in a waiting state requires no execution from the practitioner. When it is ready, it cycles back into the active set. The portfolio size is bounded not by how much can be executed simultaneously but by how many projects can be kept in a coherent waiting state without losing track of where each one is. That is a status-management problem, not an execution problem — and it is precisely the problem the ambient interface described in §9.2 is designed to solve.

This mechanism has a scope that extends well beyond application code. Every domain where desired state can be specified and actual output can be observed becomes reachable by the same structure: infrastructure provisioning, build environments, generative art pipelines, data systems, automated QA. Each new surface is not an exception to the restriction; it is a surface *the restriction opened*, made reachable precisely because the specification must now be explicit enough for a stateless reader to act on it without human guidance.

The scope does not end at digital systems. The same three-element structure — declared specification, capable executor, observable outcome with a defined correction mechanism — governs any system where an autonomous agent acts on declared intent: industrial robots, surgical assist systems, autonomous vehicles, building management infrastructure, processing plants. The executor's domain is incidental to the principle. GS is software's instance of a convergent discipline that emerges in any domain where the executor becomes capable enough that the specification becomes the bottleneck. The full argument is developed in §10.

The formal implication is worth stating plainly, because it is easy to mistake for a claim about AI capability when it is actually a claim about grammatical completeness. Once a grammar is complete, derivation is mechanical — not because the model is intelligent but because completeness closes the space of valid outputs to those that are correct. A partial grammar produces partial, and therefore arbitrary, derivation. A complete grammar produces correct derivation at every point the grammar covers.

The history of software abstraction is a history of adopted black boxes. Libraries, frameworks, programming languages — each introduced with friction, each carrying failure modes that took years to understand, each eventually dominant because the productivity advantage exceeded the residual cost. The condition for adoption has never been perfection; it has been a favorable margin. The same dynamic governs GS, with one difference in magnitude: the cost being settled is not a layer of implementation but the act of converting intent into implementation itself. The empirical projects in §7 document this concretely at production scale across five challenge categories. The productivity gap they demonstrate is not incremental — a compression of engineering effort per delivered feature not documented at comparable scale in the projects' own prior histories. The cause is grammatical completeness, not model capability. GS names the structural condition under which that gap opens: it does not make engineering easier; it makes the engineering act available to anyone who can specify what is valid.

The Convergent Principle. The grammar mechanism described above is not unique to software or to large language models. It is specific to the relationship between three elements: a declared specification of desired state, an executor capable of producing outputs from it, and an observation mechanism capable of measuring the gap between actual output and specified intent and triggering correction. Any system where all three elements are present operates under the same discipline: *the correctness of the outcome is a function of the completeness of the*

specification. GS is software engineering's first formal instance of this principle applied to the lifecycle layer; its empirical record is in §7, its implications in §8, and its convergent form across other executor domains in §10.

4.1.c A Concrete Illustration: The Art Generation Pipeline

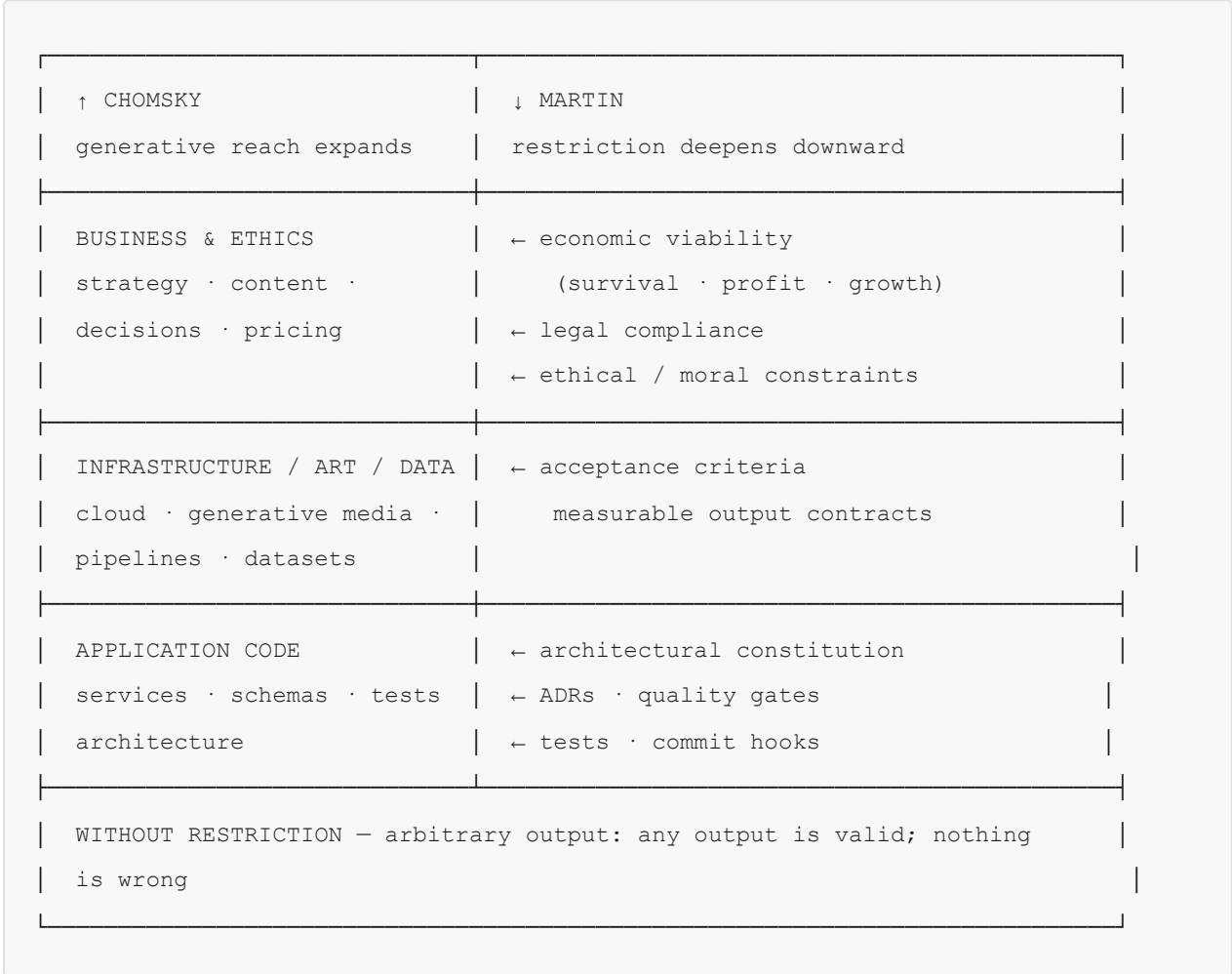
A strategy game concept is given to the system as a narrative idea. The precision of that idea is the ceiling of everything that follows: the AI will derive a specification from whatever is stated, and whatever is not stated will be filled with its own defaults — coherent by its standards, arbitrary by the designer's. A vague idea produces a generic game. A precise one — factions with named ideological conflicts, unit archetypes tied to faction doctrine, a color palette grounded in environmental lore — produces a specification the AI can execute with fidelity. The specification layer activates immediately — not prompting in the conversational sense but specifying in the GS sense: the AI derives from the idea a complete creative document covering factions, units, lore, inter-faction relationships, color palettes, and visual identity rules. That document is itself a specification. The infrastructure tier follows entirely from it: the AI identifies the toolchain required (a GPU-accelerated image generation backend, LoRA models matching the established visual language), installs and configures the tools, downloads the models, and validates the setup against acceptance criteria without human direction at any step. The art generation tier then operates under quality constraints stated as specification: viewing angle, bilateral symmetry, background isolation, LoRA weighting for each faction's color schema. The AI identifies candidate post-processing libraries, evaluates them against those constraints, and selects without human involvement. Pixel art conversion, sprite sheet layout, animation frame generation, faction logo derivation, background environments, explosion and effect sequences, character portraits for dialogue and rendezvous scenes, and UI frame art follow as sequential derivations — each step's output fully determined by the specification produced at the previous step, with the list bounded only by what the specification names.

Before the pipeline reaches steady state, a calibration phase is required. A human reviews sample outputs not to correct them but to determine whether the constraint set is sufficient: if a sprite passes all stated checks and is still wrong, the constraint set is incomplete, and the specification needs tightening. That tightening is a specification exercise, not a correction loop — one that closes permanently once the constraints are adequate. This calibration phase can itself be automated: a vision-capable model given the stated quality rules and a sample output can evaluate conformance, identify which constraints the output violates or which the rules fail to cover, and return a structured gap analysis. The human's final role is to decide that the gap analysis is empty.

Once the constraint set is sufficient, the chain from idea to production-ready game asset involves no human-in-the-loop correction at any intermediate stage, because there is no gap the specification leaves open. The quality constraints *are* the specification; the AI cannot produce an

asymmetric sprite because the specification already closed that surface. This is not a property of generative AI. It is a property of the restriction depth: remove enough freedom from the output space, and every reachable output is a correct one.

Figure 1. The Generative Specification Domain Stack



Left column: domains where the paradigm operates, from most to least established. Right column: the restriction vocabulary for each domain. The restriction is the floor that makes the generative reach above it possible.

One surface this constraint vocabulary cannot cover is aesthetic judgment: questions of visual weight, compositional tension, emotional resonance, and the craft decisions an experienced artist develops through sustained attention to what works. A constraint that closes symmetry and palette conformance produces a *technically correct* asset. Whether that asset is compelling — in the sense a skilled art director means when they say a piece is right — is a judgment the specification cannot make and the pipeline cannot render. The generated outputs at every stage of this pipeline are materials, not final deliverables. The appropriate role for a professional artist is not to correct the pipeline's mistakes but to bring the criteria the specification cannot hold: the difference between what the constraint set permits and what the work should be. The pipeline

does not replace the artist. It collapses the distance between an idea and a revisable, systematically consistent first form — which is precisely what an artist needs to begin.

4.2 The Three-Tier Taxonomy

Syntactic disciplines (Martin's three paradigms, and structural schema such as clean architecture) constrain the *form* of source artifacts: what constructs are permitted, what dependency directions are allowed. Whether every principle widely discussed as a paradigm falls neatly into this tier is a taxonomy debate this paper notes for completeness and takes no part in; the productive claim is directional: these disciplines constrain *what is permitted in the artifact*.

Semantic disciplines (SOLID, test-driven development, domain-driven design, behavior-driven development, conventional commits) constrain the *meaning* that structure communicates to a human reader who brings context to the interpretation. A SOLID-violating codebase compiles; its cost is paid by engineers who recognize the deficit. TDD makes a codebase *certifiable*: it removes the option of shipping unproven code, and the enforcement layer that makes it a discipline rather than a preference is real (CI gates, coverage requirements, deployment blocks). These disciplines assume a reader with state: colleagues, institutional memory, interpretive context built over shared history. TDD occupies the boundary between this tier and the pragmatic: a test suite is the closest prior art to a stateless, machine-readable behavioral contract, and TDD's verification posture is carried directly into GS as its Verifiable property (§4.3). What places TDD in the semantic tier is its incompleteness as a derivation grammar — tests certify behavior but leave architecture, naming, decision history, and rationale implicit. GS subsumes TDD rather than extending it.

Generative Specification is a programming discipline of the pragmatic dimension — the first to state the obligation to make the lifecycle layer derivable for a stateless reader[^3]. It constrains not what is constructed and not what communicates to a reader with context, but what is *derivable by a reader with zero context*: no colleagues to ask, no institutional memory persisting across sessions, no informal channels through which intent can travel. Every intent that would previously have been resolved through shared knowledge must be externalized as formal artifact, because the channel through which shared knowledge travels does not exist for the stateless reader. The pragmatic tier had no prior occupant not because the distinction was unrecognized, and not because stateless readers did not exist — IDLs and formal specification languages are both stateless by design, and they predate LLMs by decades — but because no widely-deployed stateless reader *of the pragmatic kind* existed: one deployed to read lifecycle intent and derive from it what should be built, where, and why, without requiring a human to navigate the gap between the specification and the implementation. IDLs read interface contracts. Formal specification languages verify property invariants. Neither reads the lifecycle layer. The transformer architecture produced the first widely-deployed reader that does

— and with its deployment, leaving the lifecycle layer implicit changed from a recoverable cost paid by skilled humans to a structural failure propagated at generation speed.

The lifecycle layer is the subject of the derivability obligation GS states. It comprises: *architectural identity* — what the system is, how it is structured, and why; *evolutionary intent* — which directions of change are valid and which violate structural invariants; *quality contracts* — the behavioral, performance, security, and compliance obligations the system must satisfy; and *decision history* — what alternatives were considered and why they were rejected. The lifecycle layer excludes the type system, the test suite, and the source code — those belong to the syntactic and semantic tiers respectively. A codebase that satisfies SOLID and has full test coverage is syntactically and semantically specified; it is not lifecycle-specified if a stateless reader cannot determine from its artifacts alone whether a proposed change is architecturally valid.

Interface definition languages (OpenAPI specifications, wire protocol schemas, RPC definitions) are specifications designed for stateless machine readers. But they operate at the interface layer: they define what crosses a boundary. A stateless consumer of an OpenAPI spec can call an endpoint. It cannot determine whether a new endpoint *should* exist, which service it belongs in, or whether adding it violates a boundary that was intentionally held. The gap is not interface consumption (IDLs handle that) but implementation generation: deriving what should be built, where, and why, from a grammar the stateless reader must be able to parse without human guidance. Those determinations require access to the lifecycle layer: what the system is, why it is structured as it is, how it should evolve. No prior discipline stated the obligation to make the lifecycle layer derivable to a stateless reader. GS is the first to occupy that surface. Formal specification languages (TLA+, Alloy, Z notation, B-Method) are stateless and system-level, but they occupy the syntactic/semantic boundary, not the pragmatic tier, for two reasons that compound. The first is what they constrain: they are property verifiers, not derivation grammars. They establish that a design satisfies a stated invariant; they do not generate the naming convention, the module boundary, the decision record, or the commit discipline the AI reads before implementing. The second, and more fundamental, is the reader they were designed for: a deterministic, rule-bound verifier — TLC, the Alloy Analyzer, Z/EVES — that checks whether an explicitly modeled finite state system satisfies an explicitly stated logical property. That reader operates at the syntactic/semantic boundary by definition: it reads formal syntax and checks semantic invariants. It cannot reason about evolutionary intent, architectural rationale, naming signal, or the direction of valid change — not because it is insufficiently powerful, but because those questions are not expressible in the language it reads. A formal methods reviewer who argues that TLA+ was already doing what GS claims is making the case that these two readers are the same reader. They are not. The formal verifier and the context-sensitive natural language reader differ in kind, not degree. GS is designed for the second. No prior discipline was.

This discipline becomes visible when its failure mode becomes undeniable: the accumulated cost of leaving intent implicit in AI-assisted development (architectural drift produced at generation

speed, propagating silently across every session that inherits a corrupted context). Independent academic work has begun observing that cost from adjacent directions (§5), establishing that the structural observation is not unique to this practitioner context.

[^3]: The syntactic/semantic/pragmatic trichotomy originates in Charles Morris's 1938 *Foundations of the Theory of Signs* — settled semiotic theory. Applying that trichotomy to classify the obligation structure of programming discipline is this paper's proposal: a theoretical frame, not a discovery within the semiotic tradition itself. The claim that GS opens the pragmatic tier rests on the structural observation in §4.2 — that no prior discipline stated the obligation to make the lifecycle layer derivable for a stateless reader — not on the taxonomy alone. The taxonomy debate over which existing disciplines fall in which tier is live and unresolved here; the classification provides orienting vocabulary. Jim Gray's 2009 Microsoft Research volume *The Fourth Paradigm: Data-Intensive Scientific Discovery* applies a paradigm count to scientific methodology — a distinct domain, distinct lineage, no overlap in argument.

[^4]: The architectural constitution is agent-agnostic as a concept; agent-specific filenames are enumerated in the artifact grammar table (§6). The case studies in this paper use `CLAUDE.md` because Claude was the primary agent throughout. The paradigm's claim holds regardless of which agent or filename is used; these are interchangeable implementations of the same production rule.

[^1]: The qualifier is necessary. Perl is the clearest case: "only perl can parse Perl" is well-attested in the PL community, referring to genuine semantic-level dependencies — the meaning of a token depends on what has been loaded at runtime — that resist formal context-free characterization. C and C++ are the other canonical example: the "typedef problem" (whether an identifier is a type name or a variable cannot be resolved from the syntactic grammar alone) requires a symbol table lookup during parsing. Importantly, the standard syntactic grammar specification remains formally context-free — the ambiguity is resolved by a separate semantic pass, not by the grammar itself. This is a parsing implementation workaround, not a grammar-level exception. Most languages' static semantics (name resolution, type checking, scope rules) require context-sensitive analysis regardless of how the syntactic grammar is formally specified. The claim holds at the level of formal syntactic grammar definitions (BNF/EBNF); it does not hold for the full language including semantic constraints.

[^2]: The hierarchy is used throughout this paper as a structural analogy for the expressive requirements placed on the primary consumer of each era's artifacts — not as a strict formal classification of programming language types. Real programming languages occupy complex and contested positions in the formal hierarchy. The productive claim is the directional one: LLMs extract meaning from context in ways that context-free parsers cannot, and this changes what a well-designed system must look like. Describing the Chomsky hierarchy as an analogy removes the formal classification burden; it does not diminish the explanatory role. The vocabulary the

analogy imports — valid, grammar, derivation — is the vocabulary that makes the discipline statable.

The pragmatic tier has a specific failure mode that names it. A system that leaves context implicit is not merely poorly documented: it is *underspecifiable*: a stateless agent reading it cannot derive correct output because the grammar is incomplete. The consequence (architectural drift at generation speed) is structural, not stylistic. The distinction between the semantic and pragmatic tiers is therefore not one of intensity but of kind: semantic disciplines produce worse systems when violated; a pragmatic violation produces a grammar the stateless reader cannot parse — the failure is not a quality deficit at higher intensity but a derivability collapse.

A system has achieved generative specification when its artifact set is designed so that any AI coding assistant, given access to those artifacts alone, has what it needs to: correctly identify what should and should not change for any given requirement; produce output that conforms to the system's architectural, quality, and behavioral contracts; and detect when any existing artifact violates those contracts.

Whether a given AI model succeeds in practice is an empirical question about the model. Whether a given artifact set satisfies this design criterion is a structural question about the specification — answerable by inspection against the seven properties below.

Generative specification is a stronger property than "well-documented code." Documentation can be narrative and passive: it can exist in a README that three people have read and that the AI session will never be given. Generative specification is active: the artifacts are themselves executable, verifiable, and self-correcting. The distinction is operational: a system cannot violate a generative specification without a mechanism triggering.

4.3 The Seven Properties

Self-describing. The system explains its own architecture, decisions, and conventions from its own artifacts. No external knowledge is required to understand what the system is, how it is structured, or why it was built that way. Self-describing addresses the *rationale layer*: not just what the structure is, but why it is that way and what rules govern it. This is the artifact-layer analog of SOLID's Single Responsibility: one system, one source of truth.

Bounded. Every unit of work has explicit scope and seams. Functions do one thing. Modules own one concern. The context window required to correctly modify any unit is predictably bounded. Bounded addresses the *structural layer*: where each piece lives and what it is responsible for. The distinction from Self-describing matters: a perfectly structured system with no architectural constitution fails the self-describing test; a well-annotated system with blurry module boundaries fails the Bounded test. Both are required. This directly maps to SOLID's SRP and Interface Segregation, but extends to include the documentation and test artifacts that define

those boundaries. The underlying principle — that a module's correctness properties should be decidable from its specification without knowledge of its implementation — was stated rigorously by Parnas (1972). Conway's Law (Conway, 1968) provides a complementary structural observation: that system boundaries tend to mirror the communication structure of the organization that produces them. In a GS context, both observations invert into a design imperative: draw the boundaries explicitly, in the specification, before any session begins, so that the AI's output reflects the intended structure rather than the incidental structure of how the work is divided.

Verifiable. The correctness of any output can be checked without human judgment. Types, tests, lint rules, coverage gates, and schema contracts form a continuous verification layer. Verification is automatic, fast, and blocking: not aspirational. In a generative specification context, the test suite carries an adversarial role beyond the TDD contract: the agent is explicitly directed to write tests designed to fail on incorrect code: to find the input or condition that exposes a violation, not to document behavior assumed correct. The test is a hunter, not a witness. This adversarial posture requires a structural commitment: tests must be written against interfaces, not implementations. A test that verifies internal state or call sequences is a test of the current implementation: it fails on correct refactors and passes on behavioral violations that happen to preserve internal structure. A test that verifies observable behavior through the public interface is a test of the contract: it survives every refactor that preserves the contract and fails on every violation of it. These two properties — correct specification and effective fault detection — are inseparable. A test suite coupled to implementation is neither. This posture extends to the full hardening surface: load tests, penetration tests, and chaos probes are specifications of adversarial conditions with explicit acceptance thresholds (§8.11). Verifiable establishes that the check infrastructure exists and is structurally enforced; whether the implementation actually satisfies those checks when exercised against a real runtime environment is the Executable property (§4.3), scored separately.

Defended. Destructive operations are structurally prevented rather than merely discouraged. Commit hooks, branch protection rules, format enforcement, and MCP tool boundaries make certain classes of mistake architecturally unreachable. The system rejects malformed input the way a parser rejects a syntax error.

This structural logic extends to the development process itself. Test-driven development requires a strict phase sequence: failing test, confirmed failure, then implementation. In a human workflow, temporal separation enforces this gate. A generative agent in a single context window has no such separation — the agent that will write the implementation is already present when it writes the test. This is not a discipline failure; it is a structural one: *phase-collapse*, the RED phase ceasing to exist as a distinct moment because no temporal barrier separates test authorship from implementation authorship. An agent told to "write a failing test first" can comply in grammar while violating the property substantively — it will write a test shaped to fail against a

not-yet-existing function, then immediately create that function. Instructions cannot close this gap. Only structural gates can. Forbidden patterns in the architectural constitution can prohibit implementation choices before a failing test commit is certified. A TDD workflow skill can require pasted test output as a mandatory stop gate before phase advance is permitted. A pre-commit hook can reject a test-only commit where all tests pass — the signature of post-hoc or vacuous tests added after the fact. The `[RED]` commit naming convention makes TDD phase sequence machine-readable in the git log: a CI rule can detect a `feat:` commit without a preceding `test: [RED]` commit and block the merge automatically. Applied to process rather than artifact, the Defended property means the RED phase cannot be bypassed any more than a malformed commit message can be pushed.

Defended in zero-tolerance execution domains. In software, Defended's obligation is to make incorrect outputs structurally unreachable. In domains where execution is irreversible — surgical systems, autonomous vehicles, industrial control, administered medication, signed legal instruments — Defended acquires a second, distinct obligation: the specification must explicitly classify the *consequence tier* of each executor action, marking which operations are reversible, recoverable, or irreversible, and naming the human confirmation gate required before any irreversible operation may proceed. An executor that knows the grammar of a domain but does not know which of its correct sentences must not be completed without human authorization is not Defended in this sense. It is guarded at the syntax layer while unguarded at the consequence layer. The distinction is precise: the incorrectness that Defended addresses in software — wrong output from an underspecified grammar — is different in kind from the harm that consequence classification prevents — correct output from a complete grammar, applied without the human gate the domain requires. In software, where $C_i \approx 0$ and $R \approx 1$, the classification overhead is unnecessary: iteration absorbs the residual. In domains where the correction loop cannot run after the fact — where the incision has already been made, the concrete poured, the instrument signed — "do no harm" is not a runtime check. It is a specification obligation. The deployment gate framework in §9.4 formalizes the *threshold* at which an executor may be trusted with consequential action; consequence classification within the specification is what makes that threshold machine-readable at the operation level rather than only at the domain level.

Auditable. The current state of the system, and the history of how it arrived there, is fully recoverable from the artifacts alone. Conventional atomic commits form a typed corpus of change. Architecture Decision Records document why the grammar evolved. Status files record the current implementation state. Nothing requires asking someone who was present at the time. Without an auditable trail, the AI will treat intentional architectural tradeoffs as defects to correct, producing drift silently, across every session that inherits the corrupted context. Full recoverability requires that commit discipline and the ADR record are both maintained. A specification without commit discipline provides partial auditability — behavioral contracts

survive, but the reasoning behind session-level decisions does not. The Shattered Stars case (§7.6) demonstrates this boundary precisely: the spec held the system's structural contracts across sessions; what it could not hold was the provenance of the decisions that shaped them.

Composable. Units can be combined and extended without unexpected coupling. Clean architecture's dependency inversion and the pure function model from functional programming ensure that composition is predictable. The AI can work on any unit without unexpected propagation effects because isolation is structural, not assumed. (Complete independent isolation of any unit additionally requires the Bounded property: predictably-scoped context ensures that isolation holds across seams, not merely within them.)

Executable. The generated output runs correctly against its specification — not merely compiles and passes static analysis, but satisfies the behavioral contracts the specification defines when exercised against a real execution environment. Verifiable establishes that correctness checks exist and are structurally enforced; Executable establishes that the implementation actually passes them. The distinction matters because a system can be fully Verifiable — correct types, passing lint, well-structured tests — while producing a server that fails every integration test against a real database or external contract.

Executable is scored conditional on specification availability: a formal contract (a Hurl suite, an OpenAPI diff, an HL7 FHIR validation runner) enables automated measurement (cf. Schemathesis — Kluev et al., 2022 — and Dredd, open-source API conformance testing tools that operationalize executable specification checking against OpenAPI contracts); a goal-directed or exploratory program requires human acceptance criteria and is scored N/A rather than 0. This conditioning is not a loophole — it is the operationalization of Specification Determinism (§9.4): whether automated checking is possible depends on how precisely the desired output can be stated, and that precision is a property of the specification, not of the generation.

Executable was implicit in the methodology from the beginning — the case studies all included verify-and-correct loops as a natural part of the session workflow, applied manually when automated gates did not catch runtime failures. The adversarial experiment series (§7.7.B) made the gap explicit by measuring it across controlled conditions: treatment-v2 (a post-hoc condition) achieved 12/12 on the six structural properties while only 1/9 test suites passed materialization. The Executable dimension formalizes what the practitioner was already doing, so it can be specified, gated, and tracked.

The seven properties are universal — they apply to every project regardless of type or domain. Their concrete artifact expression, however, is project-type-parameterized: the specific quality gates, constraint vocabulary, and required artifact types that satisfy each property vary by what the project is. A healthcare system satisfying Defended requires PII redaction rules and audit logging constraints that a CLI tool does not; a real-time system satisfying Bounded requires

latency contracts that a batch pipeline does not. §6 develops the artifact grammar and describes how the universal base and project-type overlays compose.

---

5. Related Work: Relationship to Existing Principles

Generative Specification does not replace the semantic tier disciplines — SOLID, clean architecture, test-driven development. It operates at the pragmatic tier: it constrains derivability for a stateless reader, a requirement the semantic disciplines were not designed for because no widely-deployed stateless reader existed at their formulation.

SOLID addresses properties 2 (Bounded) and 6 (Composable) in §4.3. Clean architecture addresses Bounded and Composable at the system level. TDD — the defining discipline of the semantic tier for proven correctness — is incorporated in GS as its Verifiable property: the discipline of certified output becomes a required production rule of the grammar. The new contributions of GS over all prior principles are:

- **Self-describing**: not addressed by any prior principle. SOLID was formulated for human-to-human collaboration; SRP identifies a responsibility boundary but says nothing about whether that boundary must be externalized in artifacts visible to a stateless reader. A generative specification requires the system to describe itself completely from its artifacts, because the reader cannot ask a colleague.
- **Defended**: not addressed at the tooling layer by prior principles. SOLID describes how to structure code. Generative Specification additionally specifies that the process must have structural guards. In zero-tolerance execution domains, Defended extends further: the specification must classify the consequence tier of each executor action, making human confirmation gates machine-readable at the operation level rather than assumed from domain convention.
- **Auditable**: not addressed as a design property. Conventional commits and ADRs are recommended practices; Generative Specification elevates them to required production rules. An unrecorded architectural decision is a gap in the grammar.
- **Executable**: not measured by any prior principle. A passing test suite certifies behavioral contracts; it does not measure whether the generated implementation satisfies those contracts at runtime against a real execution environment. Executable closes that gap with conditional scoring: formal contracts enable automated measurement; goal-directed programs require human acceptance criteria and are scored N/A. The property emerged from the adversarial experiment series (§7.7.B) and formalizes a verify-and-correct loop practitioners were already running without a name for it.

The practical implication: a codebase that follows SOLID and clean architecture is a necessary but not sufficient generative specification. It becomes sufficient when the self-describing, auditable, and executable artifact layers are present and maintained.

Industry Prior Art: Context Enhancement

The profession has been converging on this problem independently, through practitioner tooling rather than disciplinary formulation. Agent instruction files (`AGENTS.md`, `CLAUDE.md`, `.cursorrules`, `.github/copilot-instructions.md`) are the field's first-order response: inject architectural rules directly into the context window. Memory and status files — session summaries, progress records, decision logs — are a second-order response: persist state across session boundaries so the next window starts from a richer position. These are genuine contributions and GS makes full use of them; the artifact grammar in §6 incorporates them as production rules. But they are context-enhancement tools: they improve what enters the channel. None of them address what the channel must be able to *derive* from what it receives, nor do they specify the structural properties an artifact set must satisfy to make derivation correct. Feeding a richer window into an underspecified system produces richer drift at generation speed. The discipline GS states is categorically distinct: not what to put in the channel, but what a complete grammar for a stateless reader must look like so that any valid derivation from it is a correct one.

Independent Corroborating Work

Two independent research threads validate the problem formulation from different directions, neither arriving at the paradigm claim or the full methodology.

Gordon (2024, ACM Onward!) argues in *The Linguistics of Programming* that linguistic research (including formal grammar theory and the Chomsky hierarchy) offers substantially underused conceptual tools for programming language and software engineering research. Gordon establishes structural parallels between linguistics and PL/SE without focusing on LLMs as the reader who changes the design requirements. This paper takes a specific step within the direction Gordon identifies: grounding the Chomsky hierarchy in the specific architectural shift produced by deploying LLMs as the primary consumer of software specifications, and deriving the restriction discipline that follows. Gordon independently identifies the territory this paper's core framework inhabits, establishing that the Chomsky hierarchy is a structurally sound instrument for software engineering research, without arriving at the stateless-reader consequence, the restriction discipline, or the paradigm claim. The distinction is between analytical and prescriptive work: Gordon maps the domain with descriptive tools; this paper derives a discipline from standing in it. That both papers target Onward! reflects the same judgment about the territory's productivity, reached independently.

Thirolf (2025, KIT/KASTEL) independently identifies, in *Analysis of Project-Intrinsic Context for Automated Traceability Between Documentation and Code*, the exact failure mode described in §3 of this paper: architectural drift caused by implicit context that AI tools cannot access, observed empirically through documentation-code traceability gaps in AI-assisted development sessions. The problem statement matches without coordination. Thirolf proposes automated traceability tooling as a structural response, which is complementary to but narrower than the full generative specification methodology.

These works establish that the failure mode this paper addresses is not an artifact of a single practitioner's context. The paradigm claim and the generative specification methodology are not present in either.

---

6. The Artifact Grammar

The author is the creator of ForgeCraft; this relationship is disclosed as a material interest. Independent validation of ForgeCraft's outputs is provided by the adversarial experiment series.

A system built to generative specification consists of the following artifact types, each functioning as a distinct production rule in the system's grammar.

Artifact	Linguistic Analog	Function in the System
Architectural constitution[^4]	Grammar rules	Defines what is and is not a valid sentence in this system. Every AI interaction is governed by this document. Agent-specific filenames: `CLAUDE.md` (Anthropic Claude), `AGENTS.md` (OpenAI), `.cursorrules` / `.cursor/rules/` (Cursor), `.github/copilot-instructions.md` (GitHub Copilot), `.windsurfrules` (Windsurf). The concept is agent-agnostic; the filename is not.
Architecture Decision Records (ADRs)	Etymology and rule changelog	Documents why the grammar evolved. Prevents the AI from "correcting" intentional decisions that appear suboptimal without context.
C4 diagrams / structural diagrams (PlantUML, Mermaid)	Syntax tree	The parsed structural representation of the system. Context at a glance for any agent entering the codebase.

Use cases, flow diagrams, sequence diagrams, state machine diagrams	Sentence patterns and grammar rules with temporal order	Each diagram type constrains a distinct dimension: sequence diagrams fix the protocol between components (which calls, in which order, with which contracts); user flow diagrams define the expected journey from entry point to outcome (and are simultaneously the script for every E2E test in that flow); state machine diagrams enumerate valid states and transitions (and directly generate state transition test cases and the user-facing documentation of each mode). These are not illustrations. They are production rules the AI reads before generating any artifact the diagram describes.
Schema definitions (database, API, event)	Type system / lexicon	The vocabulary of the system with its constraints formally stated.
Living documentation (derived)	Compiled output from the grammar	Documentation regenerated from the specification: OpenAPI/Swagger from type annotations or route decorators, TypeDoc/JSDoc from inline documentation, Storybook from component specifications, generated README sections from centralized specs. Documentation maintained separately from the code it describes is a liability: it will drift. Documentation derived from the same artifacts the AI reads is always current, because it shares a source of truth with the implementation.
Intentional naming conventions	Word choice	Semantic signal at every token. A function named `calculateMonthlyCostPerMember` carries domain, operation, unit, and scope. `processData` carries nothing.
Package and module hierarchy	Phrase structure rules	Communicates responsibility and ownership through structure. The location of a file is a claim about what it is.
Conventional atomic commits	Typed corpus with morphology	`feat(billing): add prorated invoice calculation` has a part of speech, a scope, and a semantic payload. The git log is a readable history of how the grammar evolved and why.
Test suite (TDD / adversarial)	Semantic validation + adversarial probe	Answers: does this sentence mean what we think it means? Each test is a specification assertion *and* an adversarial challenge — the agent writes tests intended to expose incorrect code, not to document assumptions. The full suite is a continuously-running audit and a standing challenge to the implementation.
Commit hooks and quality gates	Parser rejection rules	Malformed input is structurally rejected before it enters the system. The architecture makes certain mistakes unreachable.

MCP tools and environment tooling	Runtime environment	The tools available to the agent define what operations are possible. Bounded tool access is bounded agency.

The artifact grammar above is the universal base — the constraint vocabulary every project carries regardless of type or domain. A complete generative specification composes this base with a project-type overlay: the additional constraints, quality gates, and artifact requirements that derive from what the project is. A healthcare system adds PII handling rules and compliance audit trails. A CLI tool adds distribution constraints and non-interactive mode requirements. A real-time system adds latency contracts and failure-mode specifications. A game adds asset quality gates and generative pipeline acceptance criteria. These overlays are not optional enrichments; they are the constraints that make the universal properties concrete for the domain. A project with only the universal base is partially specified — the base defines how to build; the overlay defines what the project must be. The case studies in §7 each carry a different overlay, which is why their artifact grammars share structure but differ in constraint vocabulary. This taxonomy — universal base plus project-type overlay — is the architecture ForgeCraft-MCP implements through its tag system: every project receives the `UNIVERSAL` constraint set, and each active tag (`CLI`, `API`, `WEB-REACT`, `HEALTHCARE`, `REALTIME`, and others) applies its overlay on top.

The tooling itself embodies the principle it implements. ForgeCraft exposes a single MCP tool — the *sentinel* — that reads three artifacts (the project configuration, the architectural constitution, and the hook definitions), derives the correct next action from their current state, and returns a single CLI command. It cannot operate beyond what the artifacts tell it. Where a conventional developer tool might expose twenty or more discrete operations, the sentinel exposes one: a stateless diagnostic that reads a finite artifact set and derives the appropriate response. The token cost is concrete and verifiable: one tool at approximately 200 tokens of context, versus a conventional tool surface of twenty-plus tools at approximately 1,500 tokens each. That is not an efficiency optimization — it is the derivability constraint applied to tool design. The sentinel is a micro-instance of the core claim: a stateless reader given a finite artifact set can derive the correct action without human narration. The tool practices what its methodology preaches.

6.1 A Generative Specification in Practice

The artifact types above are not theoretical constructs. The following is a representative excerpt from the architectural constitution (`CLAUDE.md` in the Claude convention[^4]) that governed the SafetyCorePro refactor described in §7.1 — written in its entirety before a single implementation change was made. The complete document is 155 lines.

```
# CLAUDE.md — SafetyCorePro

## Project Identity
- Primary Language: TypeScript 5.x
- Framework: Next.js 14 (App Router) + Prisma 5 + PostgreSQL
- Domain: Occupational Safety Management Platform
- Sensitive Data: YES — PII (employee records), safety incident data, compliance reco

## Architecture Rules
- All data access goes through service/repository layers — never direct Prisma
  calls from components or route handlers.
- No business logic in API route handlers — they delegate to services.
- Multi-tenant: Every query MUST include `cuentaId` filter.
  Never expose cross-tenant data.
- Permission checks via requirePermission() / requireAuth() as
  first line of every server action.

## Layered Architecture
┌──────────────────────────────────┐
│  Pages / API Routes / Actions    │  ← Thin. Validation + delegation only.
├──────────────────────────────────┤
│  Services (Business Logic)       │  ← Orchestration. Depends on interfaces only.
├──────────────────────────────────┤
│  Domain Models / Types           │  ← Pure data + behavior. No I/O. No framework.
├──────────────────────────────────┤
│  Repositories / Adapters         │  ← All external I/O (DB, APIs, files, queues)
└──────────────────────────────────┘
Never skip layers. Dependencies point downward only.

## Error Handling
- Custom error hierarchy per module. No bare Error throws.
- Errors carry context: IDs, timestamps, operation names.
- Fail fast, fail loud. No silent swallowing of exceptions.

## Code Standards
- Maximum function length: 50 lines. Maximum file length: 300 lines.
- Every public function must have JSDoc with typed params and returns.
- No abbreviations except universally understood (id, url, http, db, api).
- Bilingual naming: Domain entities keep Spanish names (visita, empresa,
```

```
  hallazgo, reporte) to match DB schema. All technical code uses English.

## Testing Pyramid
- Overall minimum: 80% line coverage
- New/changed code: 90% minimum
- Critical paths: 95%+ (permissions, multi-tenant isolation)
- Every test name is a specification: test_rejects_duplicate_empresa,
  not test_validation

## Commit Protocol
- Conventional commits: feat|fix|refactor|docs|test|chore(scope): description
- Commits must pass: TypeScript compilation, lint, tests.
- Keep commits atomic — one logical change per commit.
- Update Status.md at the end of every session.
```

The AI read the architecture rules and produced services. It read the error handling rules and produced a custom exception hierarchy. It read the bilingual naming convention and applied it consistently across every new file. The specification is not a description of what was built. It is the grammar from which the build was derived.

6.2 The Initialization Cascade

The artifact types above are not produced in parallel or in arbitrary order. Each artifact is both an output of what precedes it and a production rule for what follows. A sequence diagram that contradicts the architecture is evidence that one of them is incomplete; an ADR written before the architecture is speculation rather than decision record. The initialization cascade is:

1. **Functional specification** — user-facing behavior, domain model, key entities, and system boundaries stated with enough precision that an agent can distinguish an in-scope request from an out-of-scope one. This is the axiom set; everything else is derived from it. If a requirement cannot be stated here, it is not yet a requirement.

2. **Architecture document** — the layered structure, module boundaries, and integration surfaces the specification implies. Mermaid C4 context and container diagrams are produced at this step, expressing the architecture in the structural vocabulary the artifact grammar names. The diagram is not an illustration of the architecture; it *is* the architecture at a level of abstraction the team and the AI can both read without ambiguity.

3. **Architectural constitution** (`CLAUDE.md` / equivalent) — the operative grammar extracted from the architecture: the rules an agent must read before any implementation session begins. This document is derived mechanically from the architecture and the functional specification; ForgeCraft-MCP automates a substantial portion of this derivation.

4. **Architecture Decision Records (ADRs)** — one per non-obvious architectural choice, each recording the alternatives considered, the criteria applied, and the reasoning for the decision taken. ADRs are written immediately after the constitution, not reconstructed after the fact, because the reasoning is present now and will not be recoverable later.

5. **Use cases, sequence diagrams, and state machines** — the behavioral contracts between components, specified with enough precision that each diagram is simultaneously a test specification. A Mermaid sequence diagram naming a payment flow generates both the service interface contract and the acceptance test skeleton. A state machine diagram for a subscription entity enumerates valid states and valid transitions — and any implementation that permits an unlisted transition is wrong by the specification.

The cascade closes when a stateless agent given these five artifact sets can derive any valid implementation state without further human direction. That is the derivability criterion of §4.3, and it is the test the practitioner should apply before calling the specification complete. Generating diagrams after the code is written is documentation; generating them in this order is the specification act itself.

6.3 The Prompt-Bound Roadmap

Once the initialization cascade is complete, the specification contains enough structured context to derive what must be built and in what order. The roadmap is not planned separately from the artifacts. It is generated from them — the AI reads the functional specification, the architecture document, and the ADRs, and produces a phased plan: milestones, development cycles within each milestone, and at the item level, a pre-generated agent prompt for each unit of work.

That binding is the operative detail. A roadmap item without a pre-generated prompt is a task title — it requires the practitioner to reconstruct context at execution time, which reintroduces the memory cost GS is designed to eliminate. A roadmap item *with* a bound prompt is an independent execution unit: the prompt already contains the relevant specification references, the acceptance criteria, and the verification steps. The agent receives the prompt alongside the live spec artifacts and can execute without further human elaboration. The practitioner's role at execution time is to trigger the item and review the output, not to reconstitute intent.

The structure this produces has three clean properties:

Development history as a first-class record. Each roadmap item, when executed, maps to one or more atomic commits carrying the item's scope and intent. The git log is not a reconstruction of what happened — it is the roadmap execution record, in chronological order, with each commit traceable to a roadmap item and each roadmap item traceable to a specification artifact. A reviewer arriving at the repository at any point can determine what was built, in what sequence, and why, from the commit history and the roadmap together.

Loop separation by granularity. Short loops (a single implementation unit, one session) have the granularity of a roadmap item and its bound prompt. Long loops (a milestone, a feature group, a delivery phase) have the granularity of the roadmap's phase structure. These are explicitly different altitudes, and the prompt-bound item is the mechanism that keeps them from collapsing into each other: the short loop has a fixed scope, a fixed prompt, and fixed acceptance criteria set at the roadmap-generation step. It does not expand during execution. When the scope changes, the ADR records the decision, the relevant roadmap items are updated or replaced, and the change is visible in the roadmap's history — not discovered retroactively in the commit log.

Waiting states as valid inventory. A roadmap item whose bound prompt has been generated but not yet executed is not blocked — it is waiting. The practitioner holds a portfolio of executable units at various stages of readiness. A project that has reached a natural boundary (a deploy is running, a dependency is not yet available, a decision hasn't been made) is placed in a waiting state without losing context: the next step is already specified, and no reconstitution is required when the project cycles back into the active set. This is the scheduling property described in §4.1.b, made concrete at the planning level.

The roadmap is itself a living artifact: new items are added through the same cascade as the initialization (functional specification → architecture impact → ADRs if required → bound prompt generation), and retired items remain in the roadmap's history as part of the development record. Like the specification it derives from, it is never fully closed — it is the running account of what has been decided, what has been built, and what comes next.

6.4 The Incremental Cascade

The initialization cascade (§6.2) runs top-down: from the highest-abstraction artifact — the functional specification — downward through architecture, constitution, ADRs, and use cases, each step derived from the one above it. The incremental loop inverts that direction. The practitioner observes something — a discrepancy in the running system, a new requirement surfaced during a meeting, a misunderstood behavior, a design idea — and names it. That bottom-up signal is the trigger. The cascade then propagates upward from the observation to

exactly the layers the delta affects, and back down to implementation. Not every increment requires walking all five initialization steps. A bug that the existing specification already describes correctly needs only implementation and cascade closure. A new behavioral contract changes the spec and potentially a diagram, but not necessarily the C4 context. The cascade is not a checklist that must be completed in full on every change. It is an ordering constraint: when a layer needs updating, all layers above it are made consistent with the change before any layer below it receives it. The direction is always: observation → minimum affected layers upward → back down to implementation.

Before propagating, the AI performs an impact assessment: which artifacts reference the changed element, which roadmap items currently in progress share a dependency with it, and whether the delta changes a shared interface contract or schema. That assessment determines which layers are minimum-affected. Skipping it risks propagating a change into implementation before discovering that it breaks a parallel item already in progress — a class of error that the field names *change impact analysis* (CIA) and treats as a first step in any change procedure.

The spec is always the system of record. Code that was correct relative to the old spec and is now wrong relative to the new one is not a bug — it is a derivation gap, and the correct response is to re-derive it, not to patch it.

6.5 Loop Types and Gate Conditions

The methodology operates at four distinct loop granularities. **The initialization loop** runs once per project; its gate is the derivability criterion of §4.3 — a stateless agent given the complete artifact set can derive any valid implementation state. **The incremental short loop** runs once per roadmap item or unscripted spec delta; its gate is the §8.7 session loop invariant: full test suite passes, feature exercised at the HTTP or CLI boundary, documentation cascade complete, Status.md updated. External triggers — dependency updates, CVE advisories, breaking upstream changes — are procedurally identical to any other spec delta: impact assessment first, then the incremental cascade at whatever depth the change warrants. **The pre-release loop** runs before each environment promotion and is the methodology's hardening boundary. It requires deployment to at least one real environment — not a passing local suite. The gate is the release candidate criteria stated in the test architecture document, not a judgment call made at promotion time. The hardening suite that must pass at this boundary includes: full mutation testing across the entire codebase (pre-deployment, before the staging deploy — surviving mutants are test gaps, not deployment risks to absorb); smoke tests across all surfaces (API, UI where applicable, database migrations, external dependency integrations); load tests naming the target concurrent user population and p99 latency ceiling; stress tests to failure with a documented recovery procedure; and dynamic security analysis and penetration testing against

the deployed environment. These run at different moments within the loop — mutation is a pre-deployment gate; penetration testing requires a running environment — but all must clear before the loop closes. Progressive rollout strategies (canary, blue-green) must name the canary population size, error rate rollback threshold, and observation window explicitly — a rollout without these parameters is a full deployment with manual monitoring. **The hotfix loop** inverts the standard documentation order: a minimal targeted fix ships first; the post-mortem ADR, cascade artifacts, and rollback specification updates follow immediately after stabilization, not in the next scheduled session.

The four loops are parallel tracks at different altitudes sharing the same specification artifact set. Knowing which loop a given activity belongs to prevents both under-process (committing without cascade closure) and over-process (treating every bug fix as an architecture event).

A complete practitioner's protocol — cascade procedure, loop gate specifications, toolchain configuration, and automation patterns — is documented in the companion execution guide (`GenerativeSpecification_PractitionerProtocol1.md`). This paper establishes the structural argument; the companion provides the execution protocol organized by the five artifact memory types the methodology requires.

6.6 The Test Architecture as a Specification Artifact

The test suite is not supplementary documentation. In a generative specification context, the testing architecture is itself a first-class artifact: a specification of the system's observable behavior across every layer, coupled to a discipline that defines which tests run at which commit or release boundary. The AI generates tests from it; the agent defends against regression with it; the commit pipeline enforces it. A project without a stated test architecture has left the verification surface implicit — which is, structurally, the same error as leaving the system architecture implicit.

Software testing has accumulated a rich and still-growing taxonomy — test types by scope and purpose, variant coverage dimensions, pipeline placement per trigger — and the methodology applies it in full. Reproducing that taxonomy here would duplicate what the literature already covers thoroughly. The complete treatment — organized by project type, cross-referenced to the commit discipline cycle, and adapted to each ForgeCraft-MCP project tag — is codified in the companion execution guide (`GenerativeSpecification_PractitionerProtocol.md`, §§21–23), rather than enumerated in a paradigm argument.

What belongs here are the structural claim and the techniques specific to generative specification practice.

The expose-store-to-window technique. For interactive applications (games, real-time UIs), E2E tests benefit from a pattern the methodology surfaces explicitly: in the test environment, the application state store is exposed to `window`. The Playwright test driver can then assert not only what the screen renders but what the application believes is true — the store's internal state — without coupling assertions to DOM structure. This catches the class of failure that renders correctly but corrupts internal state: a score that displays right but is stored wrong; a game entity in an undefined state that has not yet manifested as a visual defect.

The vertical chain test. A single UI action triggers Playwright, which then queries the service layer response, the database state, and any affected indexes — verifying correct propagation through every boundary the action crosses — then returns to the UI to confirm the visible outcome matches the stored state. Not a unit test, not a visual check, not a flow test: a chain verification. One trigger, inspected at every boundary it crosses. The test specification names which critical flows receive this treatment.

Mutation testing as adversarial audit. An AI-generated test suite carries a structural risk: tests written by a system that knows the correct implementation may be written to pass it rather than to catch violations of it. Mutation testing closes this gap (Jia & Harman, 2011). By introducing deliberate behavioral faults into the implementation — inverting a condition, replacing an operator, removing a return value — and verifying that the suite detects each fault, the suite proves its own detection capability. A test that passes a mutant is not testing the contract; it is confirming the absence of one specific mutation, no more. In a generative specification context, mutation testing is the adversarial audit of the audit: the same posture the test suite applies to the implementation, applied to the test suite itself. An AI-generated suite should be subjected to a mutation run before it is accepted as a production artifact. Coverage measures what was executed. Mutation score measures what was caught. The second is the meaningful metric.

Multimodal quality gates. The case studies in §7 work with generative assets — sprite sheets, animated characters, background environments, sound effects, ambient music — produced by AI pipelines rather than hand-authored. These require quality gates that standard testing frameworks were not designed to address.

For visual assets, the Shattered Stars case (§7.6) developed an approach with a characteristic worth naming: geometric validation of AI-generated art using standard mathematical libraries. A sprite sheet produced by an image generation model must satisfy constraints that pixel-diff and even visual inspection cannot efficiently enforce at scale — the primary axis of each ship must fall within an acceptable angular range relative to the sprite coordinate system. The implementation applies Principal Component Analysis from a standard scientific computing library to the silhouette of each generated sprite, extracts the primary axis, and asserts its angle against the specification's tolerance bounds. Any sprite outside the accepted range is rejected before it enters

the asset pipeline. A general-purpose mathematical operation becomes a domain-specific quality gate. The constraint is in the specification; the tool is already on the shelf.

The same principle extends to audio. Music and sound effect assets generated by AI models carry constraints not expressed in waveforms: tempo consistency within scenes, frequency profile compliance (no asset should compete in the 2–4 kHz presence range during dialogue), loudness normalization to a target LUFS, silence-detection for generation artifacts. These are computable from the asset file; the gate is a set of assertions against audio analysis libraries applied to each generated output before it reaches the runtime bundle.

The emerging form is MCP-mediated inspection. An instrumented game state exposed through an MCP server is accessible to a language model during a test session; the model is given a scene description and a set of acceptance criteria, loads the live state through the MCP interface, and reports whether the scene satisfies them — without requiring the engineer to pre-script every assertion. Combined with conventional assertions and snapshot libraries, this addresses the class of defect that is easy to name but hard to encode in advance. It is not a replacement for the structured test suite; it is the layer that closes the gap between what a pixel diff catches and what a brief human review would flag.

The canonical pattern: multi-stage convergence with stage-differentiated feedback. The Shattered Stars sprite validation (§7.6.2) evolved from a flat four-check validator into a three-stage pipeline ordered by cost and specificity. The structural principle that emerged is generalizable to any generative asset pipeline:

Stage 1 — programmatic geometry checks (free, milliseconds): objective, library-level assertions against measurable properties. Failure at this stage is seed-level noise; the corrective action is requesting a new sample with an adjusted seed.

Stage 2 — composition analysis (free, seconds): structural properties that require layout analysis but no learned model. Failure at this stage indicates parameter-level drift; the corrective action is adjusting generation parameters before re-running.

Stage 3 — vision (or audio) model evaluation (~$0.01 per asset): subjective properties detectable only through semantic understanding — style consistency, tonal alignment, narrative legibility. Failure at this stage produces critique; the corrective action is injecting that critique as structured feedback into the next generation prompt.

The key insight is that the stages differ not only in cost but in *feedback type*. Stages 1 and 2 filter by objective criteria, eliminating the obvious failures before the expensive evaluation runs. Stage 3 closes the gap between technical compliance and perceptual correctness — a sprite can be perfectly symmetric, at the right angle, and well-composed while still looking wrong for the role. Only semantic feedback can correct semantic misalignment. The pipeline converges

autonomously; the human's role is to specify the Stage 3 acceptance criteria before generation begins, not to evaluate each asset during the run. Specifying those criteria is a tractable design task. Sitting in the evaluation loop is not.

This pattern holds across media. Musical assets go through the same three stages: waveform compliance (Stage 1), frequency profile and LUFS normalization (Stage 2), mood and scene-appropriateness evaluated by a language model with a scene description (Stage 3). Generated code artifacts follow the same structure: syntax and type checking (Stage 1), architecture constraint analysis (Stage 2), correctness and contract review by a code model (Stage 3). The artifact type differs. The convergence pattern is identical.

The commit discipline that binds all layers follows the same principle as the architecture it validates: unit and lint at file save; unit, integration, and static security analysis at commit; E2E, visual, contract, and accessibility gates at the pull request; smoke, dynamic security, and performance baseline at staging; all layers blocking at the release candidate. The full pipeline specification — tooling, thresholds, and project-type variants — is part of the test architecture document that the AI generates from the project specification, maintains across sessions, and defends on every commit.

When a new feature is specified, the specification of its tests comes first. The AI that builds the feature builds the tests simultaneously, because both are part of the same derived artifact — and the test specification is the executable proof that the acceptance criteria were understood before a line of implementation was written.

6.7 Use Cases, Diagrams, and Living Documentation

A use case is not a requirements artifact in the waterfall sense: a document produced before implementation and superseded by it. In a generative specification it is a multi-purpose production rule: a single, precise description of an interaction from which three things derive independently and without redundancy.

The first derivation is the **implementation contract**. A use case that names the actor, the precondition, the trigger, and the postcondition — expressed with enough precision to be unambiguous — is the specification the service layer is written against. The AI reading a well-formed use case before generating the corresponding service method has the same information a human architect would communicate in a design review: not just what the endpoint should accept and return, but what state the system must be in before and after, and what constitutes an invalid call.

The second derivation is the **acceptance test**. Fowler's (2018) observation that acceptance tests are orthogonal to the test pyramid — they can be implemented at the unit, integration, or E2E level — points to a structural fact: the use case and the test scenario are the same artifact expressed in different dialects. A Playwright E2E test for a checkout flow is the checkout use case transcribed into executable form. A Cucumber scenario in Given-When-Then is the use case in declarative test notation. When the use case is precise, the test writes itself. When the test is hard to write, the use case is underspecified. The test difficulty is the diagnostic.

The third derivation is the **user documentation**. A use case narrated to a non-technical reader — actor, goal, precondition, sequence, expected outcome, error cases — is a user manual section. The content is identical. The framing is different. A specification that contains complete use cases does not need a separate user manual writing pass; it needs a rendering pass.

This triple derivation changes the economics of specification work. Writing a use case is not overhead before the real work begins. It is the single investment that seeds three independent outputs — and the AI can execute all three from the same source artifact without returning to the engineer for clarification between them.

Diagram types as grammar layers. The C4 model (Brown, 2018) addresses system context, container topology, and component composition — the static structure. The missing complement is the temporal and behavioral grammar: how the system behaves over time, not only what it is. Sequence diagrams fix the inter-component protocol, specifying which call happens, in which order, carrying which payload, and what every participant must return. State machine diagrams enumerate every valid system state and every valid transition — which is the complete grammar for state transition tests and the source material for documenting modal behavior. User flow diagrams specify the expected path through the system from the user's perspective — which is simultaneously the script for every E2E test in that flow and the user journey narrative for the manual.

These are not illustrative documentation. They are constraints the AI reads before generating implementations. A sequence diagram specifying that the authorization check precedes the data fetch — and not the reverse — is a stricter constraint than a prose description, because it is unambiguous about order. The AI has only two valid sentences in that part of the grammar: the one that matches the diagram, and deviations from it.

Living documentation. Documentation maintained separately from the code it describes is structurally certain to drift. An API reference written by hand becomes wrong the moment the signature changes; a system overview written at architecture time becomes misleading the moment the first refactor lands. A generative specification resolves this by treating documentation not as a product but as a derivation: OpenAPI specifications generated from TypeScript decorators or Zod schemas; TypeDoc built from inline JSDoc that is written once and

published automatically; Storybook stories that serve as both the component specification and its interactive documentation; README sections that pull directly from centralized specs rather than paraphrasing them. The source of truth is a single artifact. The documentation is an output of the same generation cycle as the code — which means it cannot be wrong in a way that the code is right, because they share a source.

A corollary follows for inline code comments. In an unspecified system, a comment exists to explain what a variable holds, why a decision was made, or what a function intends to do — compensating for the absence of a richer specification. In a generative specification, those explanations have proper homes: naming conventions carry the semantic signal at every token; ADRs hold the decision rationale; use cases hold the behavioral intent; the architectural constitution holds the system's rules. A comment that explains *why* is a gap in the ADR record. A comment that explains *what* is a gap in the naming. When the specification layer is complete, the need for inline documentation collapses — not because comments are forbidden, but because the information they were compensating for now lives where it can be updated, versioned, and reused. The code becomes self-evident because the grammar that governs it is explicit.

The derivation surface extends well beyond technical documentation. The same artifact grammar that generates API references and TypeDoc also generates everything a user or stakeholder needs to interact with the system. A complete use case artifact renders into a user manual section, a onboarding guide, and a help tooltip via a rendering pass — the content is identical; only the framing changes. A schema definition generates a configuration reference with no additional authoring. A commit corpus generates a changelog and release notes by traversing the typed commit history. A functional specification generates marketing copy — the system's capabilities stated in feature language rather than architectural language — from the same source that generates the integration tests. A working demo script, a press release, a sales one-pager: all are transformations of specification artifacts already present. This is not a theoretical possibility; it is the same derivation principle applied one layer up. The constraint is always the same: the quality of what can be derived is bounded by the completeness and precision of what was specified. A vague functional specification generates vague marketing copy. A precise one generates copy that is both accurate and differentiating, because it names what the system actually does rather than what the author hopes it implies.

The polyglot case. The argument for a complete and explicit specification grammar is sharpest when the system spans multiple languages, runtimes, and paradigms. CodeSeeker — the semantic code intelligence tool whose polyglot architecture was built before the formal GS methodology existed, through three to four months of experimental practice that helped crystallize it — illustrates this cost from lived experience: a TypeScript VS Code extension and MCP server, a Python indexing engine, a vector store for semantic retrieval, a BM25 index for token-level recall, a knowledge graph for structural traversal, and SQL-family queries across all of them. Each layer has its own idioms, its own naming conventions, its own testing framework,

and its own failure vocabulary. Without a specification that holds naming contracts, interface boundaries, and behavioral contracts at the layer where they cross language lines, the system fragments into four separate codebases that communicate correctly on the happy path and incoherently at every edge. CodeSeeker's extended development timeline — three to four months against a median of weeks for similarly scoped projects built under formal GS from the beginning — is the comparison the polyglot argument does not need to fabricate. The architectural constitution for a polyglot system is not optional enrichment. It is the only artifact that can make the sum coherent rather than the coincidentally-working product of four different grammars. The AI cannot infer cross-language contracts from any single file. They must be stated explicitly, in language-neutral terms, in the specification both runtimes read.

The use case for living documentation is equally sharp in polyglot contexts. When the Python indexer's query interface changes, the TypeScript client must be updated, the integration tests must be rerun, and the documentation for both must reflect the new contract. In a system with a single source of truth — schema definitions, interface contracts, and behavioral specifications held in artifacts that both sides derive from — that propagation is traceable and automatic. In a system where each side maintains its own documentation, the drift is virtually guaranteed. CodeSeeker's architecture is not described by any single file. It is described by the artifact grammar that holds it together across runtime boundaries: the architectural constitution, the interface schema, the use cases that state what the combined system must do at each interaction point, and the test suite that verifies those interactions are satisfied whether the call originates from TypeScript, Python, or the MCP protocol surface.

A closing observation on where the methodology leads. The methodology has two separable layers that mature at different rates and toward different endpoints. The process layer — the seven properties, the artifact grammar, the commit discipline, the tooling — is community-convergent: it can be standardized, automated, and handed off. ForgeCraft automates parts of it; the community will extend it. This layer approaches a solved state over time, and a practitioner follows it the way a compiler follows a grammar — mechanically, verifiably, without judgment. The specification layer — domain understanding, the ability to name the correct dimension, the judgment to identify what the problem actually is before translating it into artifacts — cannot be automated. It is the input the process acts on; no amount of tooling improvement touches it. A corollary follows from this asymmetry: **if the process layer reaches community-maintained solved state, does outcome variance become a pure function of specification quality?** If it does, tool mastery, syntax fluency, and framework knowledge approach zero as differentiators. Once the checklist is community-maintained and the scaffolding is generated, what remains is entirely the engineer's understanding of the problem and the completeness of the translation — which is precisely what the process cannot supply, and what it was never designed to. The structural implications of this convergence are taken up in §10.

7. Empirical Case Studies

Six projects across five distinct challenge types demonstrate that Generative Specification generalizes beyond any single case. Each represents a fundamentally different condition the methodology must address: inheriting unknown foreign code, extending a live system without architectural structure, building from nothing at a simple scale, building from nothing at a distributed system scale, extending an existing system's domain intelligence, and migrating a broken implementation to a new platform while establishing original IP. All were executed by one engineer with AI assistance.

A structural observation applicable across the cases: in both the SafetyCorePro takeover and the BRAD migration, the pre-methodology state was itself produced through AI-assisted development — same tool class, same systems, no architectural constitution. The technology did not change between the before and after states. The specification did. This is not a designed control condition, but it is a natural one whose evidential weight is developed in §7.7.

7.1 Takeover — SafetyCorePro

Between February 14–16, 2026, a production occupational safety management system (SafetyCorePro) was refactored from a monolithic Next.js application to a fully layered, SOLID-compliant architecture. The work was performed by one engineer operating with AI assistance. No team. No sprint planning. No code review by a second human. The methodology engineer wrote zero lines of application code. The human contribution was specification authorship: ForgeCraft (then at an early release, verifiable from the tool's commit timestamps) generated the architectural constitution, and the AI received one refactoring instruction.

A structural note on the pre-refactor state: the monolithic codebase was itself produced through unstructured AI prompting — same model class, no generative specification governing the output. The natural comparison this case provides is therefore not manual development against AI-assisted development. It is AI output without specification against AI output with one. The specification is the independent variable.

7.1.1 Pre-Refactor State

The system prior to the refactor exhibited the following characteristics:

- 71 direct database calls from the UI layer (Prisma invocations in route handlers and React components)
- Zero unit or integration tests (23 end-to-end Playwright tests only)

- 227 `console.log` statements used as the logging infrastructure across server-side code
- Business logic distributed across route handlers with no service layer
- A single critical API route performing 100 database queries per page load
- No error hierarchy — bare `throw new Error('something went wrong')` throughout
- 17 missing foreign key indexes across 10 database models
- 126 prior commits built across two external developer identities, establishing the system as a real production codebase before the methodology engineer first touched the repository

7.1.2 The Specification

Before any implementation work began, an architectural constitution was produced: a `CLAUDE.md` file defining the target architecture, quality gates, coding standards, naming conventions, and explicit constraints. The specification was produced collaboratively with the AI assistant and established:

- Target layered architecture: UI → Service → Repository → Database, with explicit dependency rules
- Test coverage threshold: 80% minimum, enforced on every commit
- Error handling standard: custom exception hierarchy, every error carrying an ID, timestamp, and context
- Logging standard: structured logger with level gating and PII redaction
- Naming conventions, function length limits, file length limits
- Explicit forbidden patterns: no direct database calls from route handlers, no hardcoded secrets, no bare exception throws

This document was the generative grammar. All subsequent work was execution against it.

7.1.3 Results

Metric	Value
Wall-clock time	37.5 hours (includes overnight)
Active development time	8–10 hours across 3 sessions
Commits	10 (atomic, conventional)
Files changed	174
Lines added	16,229
Lines removed	1,889
New test cases	484

New test files	27
Test lines of code	4,898
Direct DB calls removed from UI	71
Repository interfaces introduced	3 (fully swappable)
Custom error types introduced	8
`console.log` calls replaced	227
Structured logger calls added	263
DB queries per page load (critical route)	100 → 15
Missing FK indexes added	17

The test progression across commits: 75 → 97 → 218 → 285 → 339 → 346 → 484. Architecture materialized progressively. Each commit was independently valid, tested, and deployable.

7.1.4 The Significance

The output was not the result of exceptional prompting. No novel technique was applied. The AI received one high-level instruction: “Make it production-grade and maintainable. One atomic commit at a time.” The specification is, as far as a comparison between the pre-refactor and post-refactor states of the same codebase allows the inference, strongly correlated with structural quality — the independent variable when the technology class did not change and the specification did. The AI’s context-sensitive reading of that specification was the mechanism of execution (see §7.7 for the limitations of this inference).

The 16,229 lines figure (174 files changed, from commit `dc391f4` to HEAD, verifiable from the repository on request) decomposes honestly as follows: approximately 3,040 lines are dependency metadata (`package-lock.json`) and specification artifacts (CLAUDE.md, Status.md, technical documentation) — not production code. Approximately 4,840 lines are test files, of which zero existed before the intervention. Approximately 1,200 lines of existing business logic were extracted from action files and redistributed into the new service and repository layers with proper separation of concerns. The remaining approximately 7,150 lines are genuinely new production code with no prior existence in the codebase: repository interfaces, service layer, custom error hierarchy, structured logger, rate limiter, config validation, data warehouse module, RAG graph traversal, and BM25/RRF hybrid search infrastructure.

The headline metric is not the line count. It is the structural transformation: a system with zero unit tests and no architectural layer boundaries, whose coherence existed entirely in the prior engineers’ memory, produced 484 tests, enforced coverage on every subsequent commit,

eliminated 71 direct database calls from route handlers, reduced queries on a critical route from 100 to 15, and introduced full repository/service separation — in one weekend, by one engineer, against a foreign codebase. The evidence strongly suggests this result is not reproducible without the generative specification, and reproducible under comparable specification quality. The natural comparison is the pre-refactor state at commit `dc391f4`, preserved in version control and available to reviewers on request.

The methodology engineer wrote zero lines of application code. Two acts constituted the human contribution: producing the specification, and issuing one instruction. That this result — architectural transformation, 484 tests, layer separation — was produced entirely by AI operating against a specification is not a qualification of the evidence. It is the claim.

7.2 Brownfield — Invellum

Domain: Entrepreneurial ecosystem platform — a social network for founders and entrepreneurs. Connections, social feed, project and campaign management, real-time chat, discovery, notifications, and an administrative console.

Beginning development in June 2025 (eight months before the structured restart), Invellum used earlier AI models as development tools throughout that period, but without a specification to read against, their output had no architectural home. By the time of the methodology intervention, the system had a Next.js frontend, an Express/TypeScript API, and a Prisma-backed schema covering the major domain surfaces: authentication, profiles, connections, feed, projects, campaigns, messaging, and notifications. Working, but not extensible. No architectural discipline, no test suite, no ADRs, no layer boundaries, no specification. Eight months of informal knowledge held the system together in the engineer's memory, with no artifact form.

The challenge: Extending a live system whose structure existed only in accumulated context. The brownfield case here is not a rescue from wreckage — the system worked. The cost was the compounding overhead of extension without structure: every new feature required reading prior work to understand where it belonged, every fix risked breaking something adjacent, and there was no document that captured what the system was supposed to be. The AI could produce output. It could not produce *coherent* output, because coherence requires a grammar, and the grammar did not yet exist in any artifact.

The intervention: The transformation point was the introduction of Claude Opus 4.5 and ForgeCraft. ForgeCraft generated the architectural constitution (`CLAUDE.md`), covering layered architecture, SOLID standards, testing requirements, naming conventions, and explicit module boundaries. An ADR directory was introduced. A `Status.md` checkpoint file tracked session-to-

session continuity. Playwright-based oracle tests defined the expected behavior of every surface before implementation resumed. The production deployment (Railway backend with PostgreSQL and Redis, Vercel frontend, environment configuration, CORS policy, and production URL validation) was executed from the CLI with Claude directing every step. The spec was not imposed on the existing code — it was written to describe what the system should become, and the system was grown into it.

The results over 36 commits:

Metric	Value
Development history	June 2025 – February 2026 (8 months) pre-specification; earlier AI models used throughout, without specification structure
Post-specification commits	36
Final production state	Live on Railway (Express backend, PostgreSQL, Redis) and Vercel (Next.js frontend)
Test coverage	17 oracle tests, 12 production smoke tests, resource audit, security audit
Security findings	Zero critical findings
Feature surface	Auth, profiles, connections, feed, projects, campaigns, chat, messages, notifications, discover, onboarding, admin console, i18n

The role of the specification: Before the generative specification was in place, each feature was an isolated negotiation with the existing codebase. After it, each feature was an implementation against a contract that already knew where it belonged, what it could depend on, and what it was not allowed to touch. The oracle tests meant that expansion never regressed what worked (see §7.7 for the limits of attributing this causally). The ADRs meant that decisions made in one session were available as context in the next one — to the engineer and to the AI.

Brownfield systems do not need to be rewritten to become generative. They need a specification written for what they should be, and a commitment to close the gap one atomic commit at a time. Invellum is also the only ongoing-development case in this study — the system remains in active development at the time of writing, which makes the point in a different register: the spec does not expire when the sprint ends.

7.3 Greenfield — ForgeCraft

Domain: Developer tooling. An MCP (Model Context Protocol) server that generates production-grade AI coding assistant instruction files from a library of 112 curated template blocks. Supports six AI assistants, 19 project classification tags, and a tier system. Published as an open-source npm package, distributed via npm, the MCP Registry, and multiple community channels.

Starting condition: A blank repository. No prior codebase, no inherited debt, no existing architecture. Pure specification-first construction.

The distinctive characteristic of this case: The tool was built using the methodology it implements. ForgeCraft generates generative specifications for other projects. It was itself built as a generative specification from day one. The `CLAUDE.md` that governed ForgeCraft's construction was structurally identical to the documents ForgeCraft would later generate for its users. The methodology was eating its own cooking from commit one.

The specification: The architectural constitution defined the MCP SDK integration contract, the template loading and rendering pipeline as a port/adapter boundary, the tag classification system as a domain model, and the test coverage requirements. The composition root, the tool handlers, and the registry layer were all specified as interfaces before any implementation existed. Vitest was configured as the test runner with coverage gates enforced by a commit hook.

The initial release (a single commit) shipped with 14 MCP tools, 18 composable tags, 43 template files containing 112 tier-tagged blocks, and 111 tests passing across 9 test suites. There was no prototype phase, no iterative assembly toward a working state. The specification described a complete tool. The first commit delivered one. The subsequent 39 commits are documented feature additions: multi-target assistant support, the tier system, CLI mode, the MCP sentinel, domain playbooks. The breaking rename from `forgekit` to `forgecraft-mcp` — including package name, configuration format, all type names, and every documentation reference — landed in a single commit with zero test regressions. Six months of additions. Nothing revisited.

The results over 40 commits:

Metric	Value
Total commits	40
Current version	v0.5.1
Tests passing	307 (current; 111 at initial release)
Template blocks	112
Project classification tags	19

Supported AI assistants	6 (Claude, Cursor, Copilot, Windsurf, Cline, Aider)
Distribution channels	npm, MCP Registry, Homebrew, Snap, Chocolatey
Breaking refactor	Full rename from `forgekit` to `forgecraft-mcp` — one commit, zero test regressions

The role of the specification: The breaking rename — package name, configuration file format, type names, environment variables, all documentation — was executed in a single commit with zero regressions. This was only possible because the test suite, defined against interfaces rather than implementations, verified behavior rather than structure. When the names changed, the behavior contracts held. The specification made a breaking change non-breaking in practice.

The meta argument: if Generative Specification produces systems that are reliably extensible, testable, and resilient to change — the proof of concept is the tool that generates the specification, built using the specification.

Post-publication tooling update: Since the experimental protocol was written, ForgeCraft gained a `release_phase` field (`development` / `pre-release` / `release-candidate` / `production`) on its `setup` , `generate` , and `refresh` tools. The field persists in `forgecraft.yaml` and drives a per-phase gate table embedded in the generated architectural constitution. At the `pre-release` phase, load testing, DAST, and penetration testing change from advisory to blocking — not recommended. This addresses the question the adversarial experiment raised: how does an AI-governed project know which quality constraints apply at each stage of the delivery cycle? The answer is now a persisted parameter the practitioner sets once; the specification document expands accordingly, and every subsequent AI session inherits the correct constraint set for that phase.

7.4 Greenfield (Complex) — Conclave

Domain: Multi-role AI orchestration. A system that decomposes a natural language specification into a directed acyclic graph of tasks, assigns each task to a specialized AI role (Architect, Implementer, Reviewer, Tester, Deployer, Auditor), executes them in sequence with inter-role artifact flow, and provides a real-time dashboard for human oversight and gate approval.

Starting condition: A blank repository with a written specification document. The system being built was itself a system for managing AI-assisted software construction — the most

structurally complex case in this study.

The challenge: Conclave is a distributed system with a monorepo architecture (9 packages, pnpm workspaces), a DAG execution engine, a message bus with bounce protocol, a rate limiter, a streaming execution layer, a React dashboard with real-time output, and a deployment pipeline with target auto-detection. Each of these is a non-trivial subsystem. Coordinating their construction without the generative specification would have required continuous human navigation of cross-package dependencies.

The specification: The architectural constitution governed the monorepo package boundaries as hard contract lines. Inter-package dependencies were explicitly mapped. Each package was given a single responsibility: core (DAG + state), actions (typed action library), roles (executor registry), dashboard-ui (React orchestration interface), MCP server (external interface), and so on. The DAG engine and message bus were specified as interfaces before any implementation; the bounce protocol and rate limiter were defined as domain models with pure behavior. A `STANDARD_PIPELINE` template with 11 phases and 15 tasks was written as a declarative configuration before any execution path was implemented.

The results over 27 commits:

Metric	Value
Total commits	27
Monorepo packages	9
Pipeline phases	11
Pipeline tasks	15
Tests	203 Vitest + 50 Playwright E2E = 253 total
Dashboard	React orchestration UI with streaming output, gate approval, retry/cancel, history
RAPTOR indexing	Hierarchical codebase summarization (file → module → subsystem → repo) injected into every task context via CodeSeeker integration (see §8.5)
Deploy detection	Auto-detects target from filesystem (Railway, Vercel, Fly, Docker)

The role of the specification: The 9-package monorepo was navigated without cross-boundary contamination in 27 commits. The AI never reached across a package boundary it was not supposed to cross — because the boundaries were defined in the architectural constitution as explicitly as layer rules in a layered architecture. Each commit was valid in isolation. The 253-test suite gave continuous verification across a system where the failure modes are multi-hop: a

change in the core package propagating incorrectly through the roles package into the MCP server surface would be invisible without tests at every layer.

Conclave demonstrates that generative specification scales to distributed systems with complex inter-component contracts. The complexity of the system is not a ceiling on the methodology. It is the argument for it.

7.5 Extension — BRAD (Legal Intelligence Engine)

Domain: Sovereign legal intelligence engine for US family law cases, live at askbrad.ai. Semantic and structural analysis of case documents: pattern recognition, argumentation logic, and deontic reasoning applied to family law proceedings. The methodology engineer arrived to extend and deepen the analytical capability of a codebase with an established external commit history. Two distinct developer identities are present in the repository.

The challenge: Like SafetyCorePro (§7.1), this case places the methodology engineer in a foreign codebase: another developer's commit history precedes the specification, and the familiarity objection (addressed in §7.7) applies least here. What distinguishes BRAD from SafetyCorePro is not that structure — it is the demand the extension placed on the specification. The question was not architectural coherence but epistemological precision: not how to extend the system, but which domains the system needed to occupy.

As with SafetyCorePro, the methodology engineer wrote zero lines of application code during the extension phase. The prior developer identity's commit history was also produced through AI-assisted development without a generative specification. The CLAUDE.md committed March 1, 2026 is the inflection point between both methodologies and both identities: what preceded it was AI output without specification; what followed was AI output with one.

The specification: The architectural constitution from the prior refactor phase was already in place. Extension work was defined as additions to that grammar: new analysis layers specified as interfaces before implementation, new domain models named with the rigor of the domain they served. The specification explicitly named the analytical techniques to be incorporated: prosody and argumentation analysis, discourse analysis, formal fallacy classification, and deontic modal logic (the formal ontology governing obligation, permission, and prohibition that structures legal reasoning). RAPTOR indexing, first specified for CodeSeeker, was carried forward in the architectural constitution and applied both to the codebase and to the legal document corpus.

The results: The analytical capability added during this extension phase included:

- **Deontic modal logic:** The formal reasoning framework governing obligation, permission, and prohibition — developed by von Wright (1951) and subsequently elaborated for legal and normative contexts — was present in the model's training corpus from academic legal theory sources and was activated by naming it explicitly in the specification. A generic "analyze arguments" prompt does not invoke this. A specification that names the domain does.
- **Discourse analysis and formal fallacy classification:** Specified as a structured analysis layer producing a taxonomy of argumentation patterns mapped to named fallacy types. The Toulmin (1958) argument model and the pragma-dialectical framework of van Eemeren and Grootendorst (2004) provide the formal vocabulary; both are part of the model's argumentation-theory training corpus and were activated by naming them in the specification.
- **AI-derived case taxonomy:** Rather than receiving a classification schema, the specification directed the AI to derive it — to identify the natural orthogonal dimensions along which US family law cases differ in legally material ways. The result was an AI-declared classification system with documented derivation rationale. US family law classification systems vary significantly by jurisdiction, and no unified public ontology exists against which to independently validate this taxonomy; domain-expert review by practicing family law attorneys would be the appropriate next validation step, and this paper treats the taxonomy as a working instrument rather than a validated classification.
- **Property graph knowledge layer:** A graph structure encoding case entities, relationships, and legal claims — schema, ingestion logic, and query patterns — produced from a specification of what the analysis required, not how to implement it.

Metric	Value
Developer identities in repository	2 (`saxaboom` — prior refactor, 7 commits; `Ghiringhelli` — methodology extension, 31 commits)
Specification introduced	`CLAUDE.md` committed March 1, 2026 — marks start of extension phase
Post-specification commits	31
Files changed (post-specification)	142
Lines added	23,848
Lines removed	887
Test files	39 (37 unit + 2 e2e Playwright)
Test cases	1,183

Analysis dimensions incorporated	4 (deontic modal logic, discourse analysis, formal fallacy classification, AI-derived case taxonomy)
Knowledge structures introduced	Property graph encoding case entities, relationships, and legal claims (schema, ingestion, and query patterns)
Indexing infrastructure	RAPTOR hierarchical indexing applied to both codebase and legal document corpus

The epistemological finding: The most significant outcome is not architectural. The AI's knowledge of formal legal reasoning frameworks, argumentation theory, and deontic logic is deep — it exists in the model's training corpus from academic and legal sources. What determines whether that knowledge is invoked is whether the specification names the relevant domain. A specification that says "analyze legal arguments" receives legal analysis. A specification that names prosody, argumentation theory, fallacy classification, and deontic modal logic receives a specialist instrument calibrated to the domain.

RAPTOR indexing traveled from CodeSeeker to BRAD not because any session carried it forward in context, but because it was named in the specification. The technique is the transport mechanism. The specification is the delivery vehicle. A named technique in one system's architectural constitution is available to every subsequent system whose engineer knows to put it there.

The author names this phenomenon **domain dimensional expansion**: a single term placed in the specification functions not as a keyword but as a coordinate. The model receives it not as a retrieval cue but as a calibration signal — which intellectual territory does this problem occupy? The response is not a definition of the term. It is the full apparatus of the named field, deployed at specialist depth.

The upload guard rules introduced during the BRAD extension reinforce how broadly the phenomenon operates. "Upload guard" is not an academic field. It is an engineering concept — a constraint class. Named in the specification as a first-class domain boundary, it produced a complete upload validation architecture: content-type enforcement, size constraints, MIME verification, and a structured rejection taxonomy — none of which was specified line by line. The concept was the specification. The architecture was its consequence.

The pattern holds across academic and engineering domains alike. A domain does not need a centuries-old literature to trigger the effect. It needs a name and a clear place in the architectural contract.

7.6 Migration — Shattered Stars (x-wing-arcade → TypeScript/Phaser 3)

GS methodology version: v0 — pre-experiment series, before the adversarial runs that produced the seven-property rubric, Known Type Pitfalls, infrastructure-first prompting, or the verify loop. Readers should interpret the results in that context: this is the methodology in its earliest form, applied to a demanding migration.

Domain: Tactical arcade space combat game. Five asymmetric factions with distinct playstyle philosophies, 79 ship types, full ability and upgrade systems, arc-based targeting, maneuver dials, headless simulation for balance testing, and an AI-generated art pipeline via Stable Diffusion. Original IP.

Source system and IP origin: A Unity/C# implementation ("x-wing-arcade") — 108 commits of built gameplay drawn from the mechanical foundations of a well-known tabletop miniatures game (readers familiar with FFG's *X-Wing Miniatures* will recognize the arc-based targeting, maneuver dials, and dice-based combat resolution). The specification was complete. The code was substantially implemented. The execution was deeply broken: runtime defects had accumulated across movement resolution, combat state, and scene management to a state where the game did not run correctly.

Disclosure on current state: The version analyzed in this case study is a work in progress. At time of writing, a simple skirmish mode is functional, but the implementation has known errors and several systems do not yet run correctly. The project is ambitious in scope — 79 ship types, five asymmetric factions, a headless balance simulation framework, a Stable Diffusion art pipeline — and remains in active development. The author intends to share the completed project when it reaches a state that can be demonstrated reliably. ForgeCraft is the active tooling as the system progresses. The case study documents what the methodology accomplishes at the specification and structural generation layer; it does not claim a finished, playable product.

The migration was also driven by a tooling gap: Claude's integration with Unity was limited at the time, and the MCP ecosystem that now enables deeper editor interaction did not yet exist. The practical ceiling for AI-assisted Unity development was low for exactly the class of defects that had accumulated (runtime behavior, physics edge cases, scene lifecycle), which require the kind of tight read-run-observe loop that terminal-based AI tooling handles poorly in a Unity context.

The migration served two purposes simultaneously. **Platform reach:** Unity targets desktop builds; the intended delivery is a web-hosted static application deployable to Netlify, Vercel, or itch.io with no installation. **Original IP:** A game built on borrowed mechanics is a prototype. At migration time, Shattered Stars did not yet have a name, factions, ships, lore, or visual identity.

The specification step was also an extraction. The behavioral contracts of the Unity implementation (what each system did, not how it did it) were pulled from the existing codebase and expressed in platform-independent form. The broken execution became irrelevant. What the Unity codebase contained was a complete specification of game behavior. That specification was extracted, cleaned of any Unity API reference, and the new implementation was written against it. A broken implementation is, structurally, a complete spec with a bad executor. The methodology replaces the executor.

The specification step — before rewriting a single TypeScript file:

The methodology's claim in a migration context is that the behavioral contracts of the source system can be extracted and expressed as a platform-independent specification — one that describes *what* the system does without referring to Unity, C#, MonoBehaviour, or any API the new stack will not have. That specification then becomes the authoritative grammar against which the new implementation is written.

The output of this step for Shattered Stars:

Artifact	Content	Scale
`specs/TECH_SPEC_AND_ROADMAP.md`	Full platform-independent architecture: tech stack decision with comparison tables, all game systems formally specified, AI system design, rendering pipeline, balance simulation framework, risk register	2,277 lines
`specs/` directory	25 supporting documents covering faction mechanics, ship stat distributions, maneuver dial definitions, UI wireframes, art generation pipeline, sound/music assets, progression systems, special ability distributions, crew point costs, point budget tables	25 files
`DEVELOPMENT_PROMPTS.md`	Session-scoped implementation prompts — one self-contained prompt per subsystem, ordered by dependency, each supplying the exact contract the AI session needs to implement that system without carrying forward context from prior sessions	1,496 lines
`CLAUDE.md`	Architectural constitution for the new stack: TypeScript strict mode, Phaser 3 patterns, commit policy, key system inventory	Project root

None of these documents mentions `MonoBehaviour`, `GameObject`, `SerializeField`, or any Unity API. The spec describes faction playstyle ("swarm tactics, expendable; shared targeting data"), maneuver resolution ("conversion of maneuver + current position/angle into target position/angle using Bezier curve path points"), and combat properties ("dice rolling, damage resolution, stress tracking"). The platform changed. The behavioral contracts did not.

Timeline: The specification — `TECH_SPEC_AND_ROADMAP.md`, the 25 supporting spec files, `DEVELOPMENT_PROMPTS.md`, and `CLAUDE.md` — was written in a focused session. The implementation followed a different pattern: each subsystem prompt in `DEVELOPMENT_PROMPTS.md` was fed to an AI session, which executed it with minimal interaction and produced the output. The engineer's role during those sessions was to initiate, review, and move to the next prompt — not to co-author the code. All 64 source files and 32,470 lines were committed in a single batch on March 7, 2026. The art validation work (the second commit) followed on March 8. Active engagement was concentrated at two points: writing the specification, and returning for the art pipeline. The autonomous execution in between is the point.

Implementation results:

Metric	Value
Source files (`src/`)	64 TypeScript files
Total source lines	32,470
Game systems implemented	16 (CombatSystem, ActionsSystem, TargetingSystem, MovementSystem, OrdnanceSystem, TurretSystem, PilotAbilities, UpgradeSystem, SquadBuilder, FlightControlSystem, PerformanceSystem, FactionAbilities, ManeuverTemplates, TurnSystem, TurnManager, CampaignSaveManager)
Additional modules	AI system (utility-based, 5 faction personalities × 4 difficulty levels), procedural audio engine (Web Audio API), rendering pipeline, headless balance simulation framework, 8 scenes, full UI layer
Test files	17 (435 individual test cases)

Milestone state (from project roadmap at time of writing — all items structurally generated; not all runtime-verified):

Milestone	Status
Core Systems — Movement, Combat, Targeting, Actions, Turn	⚠️ Generated, partial errors

AI System — utility-based with 5 faction personalities, 4 difficulty levels	⚠️ Generated, untested
Ship Definitions — 20 ships across 5 factions	✅ Definitions complete
Arcade Controls — real-time gameplay	⚠️ Simple mode functional with errors
Image Generator Service — Stable Diffusion API integration	✅ Pipeline built
Menus & UI — main menu, settings, credits, lobby, pause, game over	⚠️ Generated, not fully integrated
Visual Polish — ship graphics, effects, parallax backgrounds	🔄 In progress
Audio — procedural sound effects and music via Web Audio API	⚠️ Generated, untested
Balance & Testing — headless simulation framework with stats and reports	⚠️ Generated, untested
Skirmish Mode — squad building with point limits, AI difficulty selection	⚠️ Basic mode functional with known errors

The project at time of writing has a simple skirmish mode that runs with known errors. The Stable Diffusion pipeline is active. The remaining work — defect resolution across combat, targeting, and AI systems; asset integration; mode completion — is substantial. This is an ongoing, ambitious project. The claim this case study supports is structural: the migration methodology extracted a complete behavioral specification from a broken Unity codebase, and that specification drove autonomous generation of 64 TypeScript files and 32,470 lines across 16 game systems. Whether the generated output is a finished game is a separate question from whether the specification-first migration approach produced coherent, layered, testable code. The former is a work in progress. The latter is documented above.

Longitudinal note. This case study was executed with GS v0 — the methodology before the experiment series that produced the seven-property rubric, the infrastructure-first prompt, Known Type Pitfalls, and the verify loop. A game of this scope generating functional scaffolding at all under an early, unvalidated methodology is the finding worth recording here. When the project is complete, it will be presented as a longitudinal case: the same codebase, the same engineer, and the same IP — tracked from GS v0 through the current methodology level, with ForgeCraft as the active tooling throughout. That account will be able to show not just what the migration produced, but how the methodology improved around the same project as it evolved. The present version of this section documents the v0 result honestly. The completed project will make the v0 → current comparison directly visible.

The commit count warrants a precise account. The implementation sprint was driven by the session-scoped prompts in `DEVELOPMENT_PROMPTS.md`. Commit discipline was not applied consistently during construction, and there is a specific reason beyond inattention: when the first push was attempted, there was no repository. ForgeCraft, the tooling used to scaffold the project, had no behavior for that edge case at the time: it generated the architectural constitution, the spec directory, and the session prompts, but did not initialize or link a git remote. The gap was discovered at push time, corrected in the moment, and the handling has since been addressed in ForgeCraft. The resulting git history is thin: two commits capturing the after-state, not the construction sequence.

The finding is specific: the generative specification held the behavioral contracts and the architectural structure across sessions without the commit corpus. Systems built to spec arrived at coherent states even when the audit trail was absent. What the audit trail enables is *context recovery*: returning to the system in a new session and reconstructing the reasoning behind a decision without re-reading all source. That capability was not available here. Every return to the codebase required reading source and spec rather than reading a typed history. The spec is not a substitute for commit discipline. It is what survives when commit discipline is not applied.

The three sub-sections below show what the methodology executes across surfaces once a specification exists: environment configuration, generative art production, and automated visual QA. Each follows the same structure as the code work above.

7.6.1 Environment Setup as a Specification Problem

Before a single sprite could be generated, Stable Diffusion had to run — on a local GPU, with Python dependencies, on hardware the model had never seen. This is not a code problem. It is an environment configuration problem, and it is the kind of problem that consumes hours of a developer's time in dependency version conflicts, CUDA compatibility gaps, and driver mismatches.

The approach taken was to describe the desired state — a running Stable Diffusion instance, GPU-accelerated, ready to accept API calls — and let the AI resolve it. What followed was several cycles of dependency diagnosis and attempted remediation: version conflicts identified, alternatives tried, combinations rejected. The session eventually identified a pre-packaged distribution called *The Forge* (coincidental name) that bundled the required components with known working configurations. It installed the bundle, resolved the remaining gaps, created the Python environments, started the service, and verified it was responding.

That was not the end. The initial generation output was slow and the image quality inconsistent with the hardware's capability. On a prompt to read the output and optimize, the AI reviewed the

configuration against the hardware spec, identified that the batch size and VAE precision were not tuned for the available VRAM, adjusted both, and the pipeline performed as expected.

The pattern is identical to code: desired state, explicit acceptance criteria, agent iteration. The medium differs; the structure does not.

7.6.2 Automated Art Validation

The art pipeline for Shattered Stars illustrates the principle at its sharpest. Ship sprites — 20 per faction, 5 factions, 100 total — are generated via Stable Diffusion. The specification requirement is precise: a top-down orthographic view of a spacecraft hull, vertically symmetric, on a pure black background. Stable Diffusion does not guarantee this. The model will produce isometric angles, three-quarter views, diagonal perspectives, and clustered formations unless constrained.

The first attempt at constraint was prompt engineering: the generation prompts contain explicit positive framing ("PERFECT TOP-DOWN ORTHOGRAPHIC VIEW, looking straight down from above, vertically symmetric spacecraft") and an extensive negative prompt listing every unwanted viewpoint ("isometric, isometric view, isometric perspective, isometric angle, 3/4 view, three quarter view, angled view, diagonal view, side view, front view, low angle, perspective, vanishing point, tilted, rotated, angled camera"). This brought rejection rates down but not to zero. Manual review at 100 images is not a pipeline.

The solution was to specify the acceptance criteria as executable validation. A Python script (`scripts/generate_sprites.py`) implements four checks against every generated image before it is accepted:

Check	Mechanism	Threshold
Vertical symmetry	Compare pixel-level left and right halves after horizontal flip; normalized 0–1 similarity	≥ 0.85
Clean background	Measure ratio of non-black pixels in the 20-pixel border region	≤ 0.30
Vertical orientation	Principal component analysis on the ship's pixel mass; extract angle of principal axis from vertical	≤ 15°
Centering	Center-of-mass offset from image center	Informational

A sprite that fails any check triggers regeneration, up to three retries per ship. Ships that pass are logged to a preservation list so re-runs skip them. The symmetry threshold was tightened from 0.70 to 0.85 mid-project after reviewing the first batch — the initial threshold accepted sprites that were technically symmetric but visually lopsided.

The evolved architecture: staged convergence. The flat four-check baseline above performs all checks at the same cost tier — every generated image pays the same validation cost regardless of how obviously it fails. The design that follows from observing that pattern is a staged pipeline where each stage's failure drives a *different type of adjustment*, and expensive stages only evaluate images that passed the cheaper ones:

```
Generate image
  ↓
Stage 1: Programmatic geometry checks  (free, milliseconds)
  - Symmetry score (pixel-level comparison after horizontal flip)
  - Background noise detection (variance in expected-black border region)
  - Orientation alignment (PCA on pixel mass; principal axis angle ≤ 15°)
  - Color histogram (B&W compliance, no color bleed)
  - Contrast ratio (sufficient dynamic range)
  → FAIL: regenerate with adjusted seed — geometry failures are seed-level noise

Stage 2: Composition analysis  (free, seconds)
  - Subject centering / rule-of-thirds occupancy
  - Edge density distribution (crosshatching consistency across the frame)
  - Aspect ratio compliance
  - Blank space ratio (not too sparse, not too cluttered)
  → FAIL: regenerate with adjusted generation parameters — composition failures
          indicate prompt-parameter drift, not random variation

Stage 3: Vision model evaluation  (~$0.01 per image)
  - Style consistency against the faction's reference sprite set
  - Emotional tone match to ship archetype
  - Narrative clarity ("does this read as a long-range interceptor?")
  - Quality ranking against previously accepted sprites in this faction
  → FAIL: regenerate with the vision model's specific critique injected into the
          generation prompt — subjective failures require semantic feedback

PASS: image accepted into asset pipeline
```

The structural principle the three stages express is **cost-stratified convergence with stage-differentiated feedback**. Stages 1 and 2 are free and eliminate the obvious failures — most rejected images fail at these layers before reaching the expensive evaluation. Stage 3 closes the gap between technical compliance and artistic correctness: a sprite can be perfectly symmetric, correctly oriented, and well-composed while still looking wrong. That class of failure is only

detectable subjectively, and the vision model's critique — injected directly into the next generation prompt — is semantically richer feedback than any parameter adjustment. The failure mode at each stage drives a structurally different corrective action: seed randomness (Stage 1), generation parameters (Stage 2), or prompt content (Stage 3). The pipeline converges autonomously. The human's role is to define the Stage 3 acceptance criteria before generation begins, not to evaluate each image during the run.

The important point is not the technique. It is the pattern: the desired output was specified as a measurable contract at three distinct levels of cost and abstraction, and each level's specification drove its own corrective feedback. This is generative specification applied to an art production pipeline. The same logic that governs whether a TypeScript module satisfies an interface governs whether a sprite satisfies its visual requirements. The artifact type and the medium are different. The underlying principle is identical.

7.6.3 Visual QA via Screenshot Analysis

The bug-squashing phase introduced a fourth pattern. Playwright runs against the live game and captures screenshots at defined interaction points: game states, combat sequences, UI transitions. These screenshots are passed to the AI, which reads them visually and identifies defects — misaligned UI elements, incorrect game state rendering, ships in positions they should not occupy. The defect description is then fed back as a fix prompt.

This is manual QA operationalized. The human role in a traditional QA cycle is to run the game, observe the visual output, identify what is wrong, and report it. That loop is now closed by the AI reading the screenshot. The Playwright harness provides repeatability; the visual analysis provides the judgment that a test assertion cannot — because some classes of defect are only visible, not textually detectable. The game iterates toward correctness the same way the art pipeline does: specify the acceptable state, observe the actual state, close the gap.

A more demanding variant closes the full vertical slice. A Playwright interaction fires a UI action; the AI then queries the service layer response, the database state, and any affected indexes, verifying that the effect propagated correctly through every layer, then returns to the UI to confirm the visible outcome matches the stored state. This is not a unit test and not a visual check. It is a chain verification: one trigger, observed at every boundary it crosses. A defect anywhere in the chain (service logic, persistence, index consistency, UI rendering) is surfaced in a single pass. The specification defines what the chain should produce at each layer; the AI runs the chain and reports where the actual state diverges from the specified one.

7.7 Threats to Validity and Proposed Experiments

A reader applying standard empirical scrutiny would raise the following objections. Each is addressed in turn.

Threat: The single-engineer design introduces selection bias. If all case studies were executed by the engineer who developed the methodology, the results may reflect personal skill in system design rather than the methodology itself. A practitioner with weaker specification ability might see no benefit.

Assessment: Partially true, but the familiarity objection does not survive the evidence. SafetyCorePro was not the methodology engineer's codebase. It was a production system with 126 commits — across two distinct git identities (`grodriguez@vairix.com` and `g.rodriguez.montevideo@gmail.com`) at a different organization — before the methodology engineer touched it. He arrived as an outsider to foreign code, with no accumulated context and no prior knowledge of the system. The specification was not an aid to memory. It was the only navigational instrument available — which is precisely the condition the methodology is designed for. BRAD carries a second external contributor. The familiarity objection applies to the engineer's own projects (Invellum, ForgeCraft, Conclave, Shattered Stars); it is directly contradicted by the two externally-authored ones.

The evidence supports two distinct sub-claims that should be named separately. The first — that the methodology applies to foreign codebases the author did not design — is established by the pre-specification commit histories of SafetyCorePro and BRAD, which carry prior engineer identities forensically distinct from the methodology engineer. The second — that the structural improvements the methodology produced were durable and legible to other engineers without the methodology engineer present — is established by the continued post-refactor commit activity under those other identities on both repositories. A specification that required the methodology engineer's ongoing interpretive presence to remain coherent would degrade under other engineers' commits; the commit record shows no such degradation. This constitutes partial replication in the *beneficiary direction*: other engineers could navigate, extend, and maintain the system the methodology produced. What the evidence does not establish — and what independent replication must address — is replication in the *practitioner direction*: whether another engineer of varying specification skill could apply the methodology and produce comparable structural outcomes. That bound is addressed in §7.7.A below and is not closed by this evidence.

A second observation bears on this question. During the period covered by this paper, the author maintained around fifteen open projects, with six or seven active at any screen session and the rest in structured waiting states — paused on a deploy, a test run, or a pending specification decision. The six documented here were selected for typological fit; the others are greenfield explorations across domains — quantitative finance, language tools, operational automation, content systems, game development — most of them private. If the productivity gain were a

function of personal execution speed, widening the portfolio should have diluted it: execution capacity is finite and spreading it thins each project. Under the GS model the active constraint is specification bandwidth, not execution, and a project in a waiting state places no execution demand on the practitioner at all. The portfolio itself therefore constitutes directional evidence against the pure-skill interpretation: the same practitioner cycling across substantially more projects than the documented cases, across different domains, is a pattern more consistent with a structural mechanism — waiting states are free under GS, not under sequential execution — than with personal skill at exhausting pace. The author states this as personal observation, not a verifiable metric — most of these projects are private and commit discipline has not been uniformly applied across the full portfolio.

The formal argument it supports is structural and stated in §4.1.b: that derivation from a complete grammar is mechanical, not personal, and that a project waiting for a specification input does not consume the practitioner's execution capacity. That is the claim independent replication should test: not whether GS outperforms no-GS on a single project, but whether it preserves quality as portfolio size grows past what sequential execution would allow.

What remains unresolved is not simply a specification-quality question but a more fundamental one about the nature of that quality. The methodology establishes a floor — a practitioner following it will produce a specification that is better than no specification, and the floor is meaningfully higher than the industry default of none. But the ceiling is set by something the process cannot supply: the depth of the practitioner's engagement with systems, patterns, techniques, and domains. An engineer who has studied formal language theory will specify a grammar that is actually well-formed. An engineer who knows what RAPTOR indexing is will reach for it when the context calls for it. An engineer who has read Evans will name aggregates correctly; one who has read Dijkstra will recognize when a loop invariant is the specification. This is not domain expertise in the narrow sense — it is the kind of intellectual capital that accumulates from taking theory seriously when the industry has consistently preferred to dismiss it as impractical.

Academicism, in the pejorative sense the field often intends, turns out to be exactly the input that allows a specification to invoke the AI's deepest capabilities rather than its most generic ones. The process secures the foundation. The structure above it reflects the mind that designed it. Independent replication will establish how much the foundation alone accomplishes — and that result, whatever it is, sets the lower bound.

Threat: There is no control condition. No equivalent project was built concurrently without a generative specification. The productivity claims cannot be attributed causally to the methodology rather than to factors like model capability, hardware, or the specific projects chosen.

Assessment: True in the strict experimental sense, but the evidence has more structure than it appears. The SafetyCorePro pre-refactor baseline is not an assertion — it is preserved in version control at commit `dc391f4`, exhibiting exactly the characteristics described in §7.1.1: 411 TypeScript source files, zero unit tests, 23 end-to-end tests. The post-specification state is the current HEAD. The transformation — 174 files changed, 16,229 lines inserted, 484 tests introduced — is the diff between them, verifiable by the author on request (SafetyCorePro is a proprietary repository; access is available to reviewers). The Invellum case provides a second within-project comparison: eight months of active development without the methodology produced a working but architecturally incoherent system; the structured restart produced the results described in §7.2. These are not controlled experiments. They are sequentially observed before/after states on live systems, which is the appropriate evidence class for a methodology paper presenting first results. A controlled experiment remains the appropriate next step.

A second structural observation bears directly on the control question. In both SafetyCorePro and BRAD, the pre-methodology state was itself produced through AI-assisted development without a generative specification. Same tool class, same systems, no architectural constitution governing the output. This is not a designed control condition — but it is a natural one. The technology was constant across the before and after states. The specification was not. The structural difference at the boundary — zero tests to hundreds, monolithic to layered, drift-generating to self-correcting — cannot be attributed to the technology, which did not change. In both cases the methodology engineer wrote zero lines of application code. The human contribution was the specification. The before-state demonstrates what that same AI produces in its absence.

Threat: The metrics are self-reported and unverifiable.

Assessment: Substantially false for structural metrics; true only for time estimates. The core empirical claims (commits, files changed, lines inserted and deleted, test counts, test line counts, layer boundary conformance, DB call counts) are all derived from git history and are reproducible from repository access — available to reviewers on request for SafetyCorePro and BRAD (proprietary), and publicly for CodeSeeker and ForgeCraft-MCP. Multi-author attribution in SafetyCorePro (two developer identities with distinct organizational email domains) means authorship claims are forensically verifiable from the git log, not asserted. Commit timestamps establish the calendar window without self-report: the first CLAUDE.md commit in SafetyCorePro is timestamped `Sat Feb 14 21:07:06 2026 -0600`; the final refactor commit is dated Feb 16, under 48 hours, recoverable from git metadata. The one genuinely self-reported metric is *active development hours*: the subset of calendar time excluding sleep, breaks, and interruptions. “8–10 hours across 3 sessions” is an estimate that cannot be verified from the repository alone; the commit timestamps confirm it is not implausible, but do not confirm it precisely. Future studies should instrument sessions with timestamped tooling telemetry to separate calendar time from active development time.

Threat: The tooling is incomplete, which confounds the results.

Assessment: This is not a threat — it is a directional argument for the methodology. ForgeCraft generates architectural constitutions and commit scaffolding but does not yet fully implement the complete artifact grammar described in this paper. Structural diagram scaffolding, automated naming validation, and package hierarchy tooling are under active development. If incomplete tooling produced these outcomes, complete tooling establishes a floor: the results reported here are a lower bound on what the methodology can deliver, not an upper one. An incomplete tool underperforms relative to the full methodology. The evidence therefore understates the case.

These four threats, taken together, define the current epistemic state: the evidence is stronger than a naïve "single author, no control" reading suggests, the one genuinely unresolved question is replication by practitioners with varying specification skill, and the tooling trajectory points upward from current results rather than qualifying them.

Independent replication is invited. The architectural constitutions, ADRs, and full commit histories of CodeSeeker and ForgeCraft-MCP are publicly accessible. SafetyCorePro and BRAD are proprietary; the structural metrics cited are derived from git history and can be verified by the author on request. Researchers interested in the proposed comparative study should contact the author; the complete ForgeCraft template corpus can be shared as the basis for a controlled replication. The experimental designs below were stated with sufficient precision to be run against the paper's own claims. It is the author's explicit intent that they be run — and that the methodology survive the test.

7.7.A Experiment I: Human Practitioner Study — Scheduled April 2026

To test replication in the *practitioner direction* — whether an engineer other than the methodology's author can apply it and produce comparable structural outcomes — a controlled practitioner study will be conducted at a workshop in Mexico City on April 10, 2026 with forty developers as subjects. Twenty participants (Group A) receive the complete generative specification artifact set for a prepared brownfield repository: an architectural constitution, a complete ADR set, sequence diagrams for each primary flow, and schema definitions. Twenty participants (Group B) receive a task card describing the same feature set with no specification artifacts. Both groups work on the same codebase for the same duration. An evaluator blind to condition scores each output against two independent rubrics: the six structural derivability properties from the GS rubric (Self-describing, Bounded, Verifiable, Defended, Auditable, Composable — the pre-Executable set, since Executable measurement requires a running server and jest output that cannot be standardized across forty workshop participants), and an external structural quality battery — test coverage, cyclomatic complexity, ESLint violation rate, and maintainability index score — that predates and is independent of GS. The dual rubric prevents

circular evaluation: the external battery establishes whether the output is structurally better by criteria the methodology's framers did not define.

Session design and data collection. The workshop session follows a Socratic structure. An opening conversation (participants speak, facilitator listens) surfaces how AI tools are currently used in the room — establishing priors before any experimental task. Experiment 1 (control condition, no GS artifacts) follows without disclosure of condition. After scoring, GS is introduced through dialogue: participants are shown a short CLAUDE.md excerpt and asked what they think it does before it is named — they construct the concept before receiving it. Experiment 2 (treatment condition, full GS artifact set) follows. A closing reflection (participants speak, facilitator listens) surfaces cross-domain analogues and unanticipated observations. The entire session is recorded for AI-assisted transcript analysis and finding summarization; participant priors from the opening conversation will be correlated against their Experiment 1 outputs as a secondary analysis.

7.7.B Experiment II: Multi-Agent Adversarial Study — Results Below

A fixed repository and a fixed set of requirements are given to four agentic conditions running in parallel: (1) no specification — session prompts only; (2) partial specification — architectural constitution only, no ADRs, no commit discipline; (3) full generative specification as defined in this paper; (4) full GS with the paper itself as a RAG-accessible source of truth, available to the agent during construction. Each condition produces an artifact set — code, tests, commit history. An adversarial auditor agent then evaluates each output against the six structural properties active at the time of the experimental runs (Self-describing, Bounded, Verifiable, Defended, Auditable, Composable; the Executable property was identified through this experiment series and formalized subsequently — see §4.3). Critically, the auditor is given these six properties as an independent rubric, not the paper — to avoid the circularity of an agent scoring GS outputs higher because it has read the document that defined the scoring criteria. A second judge agent scores structural coherence, boundary conformance, test quality, and naming signal independently. The comparison is made at the artifact level: not "did the code work" but "does the specification implied by the output match the one it was given, and does it satisfy the derivability criterion for a stateless reader." This design makes the paradigm claim falsifiable in a concrete, reproducible form and establishes the control condition this paper cannot provide from practitioner evidence alone.

Benchmark project selection requires three constraints: representativeness across the five challenge categories established in §7; scope bounded enough to produce comparable artifacts but non-trivial enough that architectural coherence is a meaningful outcome (200–500 source files is the appropriate band); and no prior GS exposure in the codebase used for brownfield and

takeover conditions. Independent project selection by a party not affiliated with the experiment's author is the cleanest resolution.

This experiment specification is itself an instance of the principle it tests: stated with sufficient precision that an agentic orchestration system — such as the one described in §7.4 — can execute it without further human elaboration. The chain closes on itself: the methodology proposes the experiment, the tool built under the methodology can run it, and the tool was built using the methodology.

7.7.B.1 Results

Benchmark: RealWorld (Conduit) backend API — a full-featured REST application (authentication, articles, profiles, comments, tags, favourites) implemented in TypeScript/Node.js/Express/Prisma against a live PostgreSQL database. Chosen because it has a published specification, a community Postman collection for conformance testing, and known correct implementation patterns — making automated evaluation straightforward and reviewer replication feasible. All three conditions ran on `claude-sonnet-4-5`, March 13 2026. The experiment design and scoring rubric were pre-registered in commit `bd2c05b` before any condition was run.

Seven conditions (three pre-registered; four post-hoc):

- **Naive** — 3-line README, prompts averaging 4 lines, no architecture guidance, no error format, no test requirements. Represents the de facto default: AI tools deployed without structured methodology.
- **Control (expert prompting)** — API spec plus a detailed README (tech stack, layered architecture, error format, naming conventions, coverage target) and 7 prompts averaging 30 lines each with architectural requirements inline. Represents what a skilled senior engineer does today without GS artifacts.
- **Treatment (GS v1)** — same API spec plus 17 GS context files: CLAUDE.md, Status.md, pre-defined Prisma schema, 4 ADRs, C4 and sequence diagrams, use-case document, test architecture document, NFR document. Prompts averaged 8 lines — brief because the artifact layer carried the specification.
- **Treatment-v2 (GS v2, post-hoc)** — same GS artifact cascade updated with three template changes identified by gap analysis of the primary results: explicit "Emit, Don't Reference" directives for infrastructure files (hooks, CI, commitlint), a First Response Requirements list of 9 mandatory P1 artifacts, and expanded DI interface naming. Not pre-registered.
- **Treatment-v3 (GS v3, post-hoc)** — GS v2 plus prescriptive dependency governance: named package selection (argon2 over bcrypt, avoiding the `@mapbox/node-pre-gyp → tar`

CVE chain) and an explicit `npm audit` gate as a P1 requirement. Targeted the 9-CVE surface exposed in treatment-v2's static analysis. Not pre-registered.

- **Treatment-v4 (GS v4, post-hoc)** — GS v3 plus ADR emission precision fixes from the v3 gap analysis, combined with a materialize→tsc→jest→correct verify loop (max 5 passes). First condition to target runtime execution directly. Not pre-registered.
- **Treatment-v5 (GS v5, post-hoc)** — GS v4 template changes replaced with a redesigned approach: a dedicated `00-infrastructure.md` prompt that must complete before any feature prompt, plus Known Type Pitfalls (including the `jsonwebtoken` StringValue narrowing pattern) documented explicitly in CLAUDE.md. Not pre-registered.

GS audit scores — unified 7-property rubric (all conditions re-audited; blind, separate session, scale 0–2 per property):

Property	Naive	Control	Treatment	T-v2	T-v3	T-v4	T-v5
Self-Describing	0/2	2/2	2/2	2/2	2/2	2/2	2/2
Bounded	1/2	2/2	2/2	2/2	2/2	2/2	2/2
Verifiable	1/2*	2/2	2/2	2/2	2/2	2/2	2/2
Defended	0/2	0/2	0/2	2/2	2/2	1/2	2/2
Auditable	0/2	0/2	0/2	1/2	2/2	1/2	2/2
Composable	0/2	1/2	2/2	2/2	2/2	2/2	2/2
Executable	1/2‡	2/2‡	2/2‡	2/2‡	2/2‡	1/2	2/2†
Total	**3/14**	**9/14**	**10/14**	**13/14**	**14/14‡**	**11/14**	**14/14†**

* Naive Verifiable: the original audit scored this 2/2 based on test structure and naming; the unified re-audit applied a stricter behavioral criterion — tests must compile and tests must run — reducing the score to 1/2. All six naive test suites fail to compile due to missing schema models (real coverage 0%). ‡ Naive–Treatment-v3 Executable is auditor-inferred: the re-audit assessed whether generated code compiles and tests exist, not whether tests pass against a real execution environment — no verify loop ran for these conditions. † Treatment-v5 Executable 2/2 is session-verified: the verify loop confirmed 109 total tests (independent re-run: 106 passing, 3 test-isolation failures in article.test.ts — duplicate user registration in a preceding test leaves token undefined in cleanup; not implementation errors) across 10/11 suites against a live PostgreSQL database, converged in 2 fix passes (four runner infrastructure bugs fixed before the verified run — see companion supplement §S9.6). The AI integration response reported 114 total tests; 109 is the runner-confirmed count and is the figure used throughout. Note that no jest-output.json artifact was committed for v5 in the way that RX evidence was committed to `experiments/rx/evidence/` ; the verification was conducted within the audit session rather than

as a reproducible committed artifact. This is the remaining epistemic gap between v5 and RX. Treatment-v3 and treatment-v5 share the same score (14/14) with completely different epistemic bases: treatment-v3's score is auditor-inferred from static artifacts; treatment-v5's is session-confirmed by a passing test suite against a live database. The verify loop's value is not the score — it is the guarantee that the score reflects something real.

The progression 3 → 9 → 10 → 13 (out of 14) is monotonic across the four pre-post-hoc conditions on every measured instrument. The direction is unambiguous. The magnitude of the three pre-registered conditions, as single-model single-run evidence, is not.

The honest finding on control vs. treatment (GS v1): The gap is narrow — one point, one dimension. Treatment's sole advantage over expert prompting was Composable: treatment generated repository interfaces (`IUserRepository` , `IArticleRepository`) with an explicit composition root, while control used constructor injection against concrete classes. This is directly traceable to the GS `SOLID Principles` specification. The expert-prompt control, which did not specify an interface pattern, did not produce one. The control–treatment gap is a directional signal, not a statistical fact.

The primary finding is naive vs. structured: The naive condition produced an internally incoherent project — its test suite references database models that do not exist in the materialized schema, because the model described schema additions in prose rather than emitting them as path-annotated code blocks. All six test suites fail to compile. Zero tests run. Zero coverage. Both structured conditions produced compilable, layered code. The control condition produced a fully passing suite. Treatment's suite compiled but 6/10 test suites were blocked by a known JWT type narrowing pattern not yet named in the v1 specification; 4/10 passed. The most important thing structure does is prevent catastrophic output failure. That boundary is established at naive–control; the control–treatment difference is secondary.

The Defended floor — and its resolution: All three pre-registered conditions scored 0/2 on Defended. The treatment condition's CLAUDE.md explicitly specified `.husky/pre-commit` and `.github/workflows/ci.yml` . The model cited these in documentation prose. It never emitted them as files. This reveals a behavioral constant: models treat specification text as guidance for application code structure, not as directives to generate operational infrastructure. A GS artifact that *specifies* a hook does not cause the hook to *exist*. The treatment-v2 post-hoc run tested the correction directly: when the template provided fenced file templates for every infrastructure artifact and named them in a First Response Requirements list, the model emitted all of them in P1. The auditor's treatment-v2 Defended justification: *"Husky pre-commit hook blocks commits if type checking, linting, or tests fail. CI pipeline re-enforces all checks on push/PR. A failing test cannot be committed locally or merged remotely."* Defended moved 0→2. The failure was not model capability — it was instruction precision.

Treatment-v2 achieved 12/12: The first perfect score in the experiment series. All three gaps identified in the gap analysis — Defended (0→2), Auditable (1→2), Composable maintained at 2/2 — closed simultaneously in a single post-hoc run. The changes required were centralized: three additions to `templates/universal/instructions.yaml`, propagating to every GS-governed project. An expert-prompting practitioner who discovers the same gaps must update every project README individually. That asymmetry — one template change vs. N project changes — is the democratizable difference the experiment demonstrates.

Coverage regression in treatment-v2: The perfect audit score coexisted with a coverage regression: only 1/9 test suites passed materialization (vs. 4/10 in GS v1). The cause is the same *emit vs. reference* failure applied to a different class of file — error classes, test helpers, and middleware were imported by name but not emitted as path-annotated blocks. The First Response Requirements list covered infrastructure artifacts (hooks, CI, interfaces) but not application-level files. A GS v3 hypothesis follows directly: extending the emit list to include `src/errors/*.ts`, `src/middleware/*.ts`, and `tests/helpers/testDb.ts` should recover the coverage regression while maintaining the 12/12 audit score. The pattern is self-correcting: each run exposes the emit boundary precisely, and the boundary is specifiable.

Real coverage vs. hallucinated coverage: Both pre-registered structured conditions reported aspirational coverage in their own documentation: control claimed 94.52%, treatment claimed 93.1%. Real measured coverage was 34% and 28% respectively. Models report desired outcomes in documentation, not measured ones. Any experiment that takes AI-reported metrics at face value is measuring aspiration, not execution.

Post-experiment mutation testing: After the primary run, Stryker was applied to the treatment project's services layer (116 effective mutants). The baseline mutation score was 58.62% MSI — against the same project whose documentation claimed 93.1% coverage. After three rounds of targeted assertion improvements, the mutation score reached 93.10%: the number hallucinated in documentation turned out to be the number it takes to actually achieve the quality level implied. Note that the 93.1% figure from the AI-generated documentation became the stopping criterion — improvements were targeted until that threshold was reached, not until no further gains were available. The convergence point was set by the prior claim, not by independent maximization. All gaps resolved by adding assertions, no architectural change required. The same 22-point gap between line coverage and mutation score that appeared in Shattered Stars appeared here under controlled conditions on a fresh codebase.

What changed in theory: Bounded (layer discipline) is achievable without GS — expert prompting at the control level produces clean layer separation. The structural gains GS provides are concentrated in Composable (interface-based dependency injection), Auditable (decision record persistence), and Defended (operational enforcement infrastructure). The *democratizable* difference is not the score a single run achieves. It is what happens when the knowledge

compounds: GS improvements centralize in the template and cascade forward; expert-prompting improvements are local to the practitioner and the project.

Three template changes confirmed by treatment-v2:

1. *Mutation gate as a hard quality criterion.* MSI ≥ 65% overall blocks PR merge; ≥ 70% on new/changed code. Run Stryker per module after test authoring, not only at release. Propagates to all GS-governed projects on next `forgecraft refresh_project`.
2. *Emit, don't reference.* Infrastructure files — hooks, CI workflows, commitlint configuration, ADR stubs, IRepository interfaces — are named as files to be emitted in P1 with fenced templates. Treatment-v2 confirmed this change produces the artifacts; prior treatment specified them and produced none.
3. *Coverage hallucination is a known failure mode.* AI-reported coverage numbers in generated documentation are aspirational. The only trustworthy number is the one produced by running the suite against a real database.

Post-publication finding: architectural compliance and dependency security are fully orthogonal. Static quality checks were run across all four materialized conditions after the primary results were reported. The three checks required no running server and were applied with consistent flags across all conditions.

Check	Naive	Control	Treatment	Treatment-v2
`tsc --noEmit` errors	**41**	**1**	**0**	**0**
ESLint problems (bare baseline)	29	40	40	**21**
`npm audit` high CVEs	3	**0**	3	**9**

Treatment-v2 — the first condition to achieve a 12/12 GS audit score — also has the highest vulnerability count: nine high CVEs, versus zero for the control. The source is a devdependency chain: `@typescript-eslint` pulling an old `minimatch` version. The control avoided this by selecting a different password library. Neither choice was architecturally motivated; both were made by the model without explicit guidance. The finding establishes that the GS rubric measures structural quality (layer discipline, interface enforcement, test construction, enforcement infrastructure) and does not assess supply-chain security. Both are necessary pre-release blockers; neither subsumes the other. A complete gate requires both an architectural audit *and* a vulnerability scan as independent checks. Full per-condition npm audit detail is in the companion supplement (§S9.3).

Post-publication condition — V3: Dependency Governance. A third post-hoc run was conducted after the static quality analysis, targeting the CVE finding directly. The condition added one prescriptive layer to the GS v2 template: explicit dependency governance instructions

requiring `npm audit` to pass (zero high CVEs) as a P1 requirement, and naming preferred packages for password hashing (avoiding the `bcrypt` → `@mapbox/node-pre-gyp` → `tar` CVE chain). All other artifacts were unchanged from treatment-v2.

Property	Treatment-v2	Treatment-v3	Delta
Self-Describing (0–2)	2	2	0
Bounded (0–2)	2	2	0
Verifiable (0–2)	2	2	0
Defended (0–2)	2	2	0
Auditable (0–2)	2	**1**	**−1**
Composable (0–2)	2	2	0
Total (0–12)	**12**	**11**	**−1**
`npm audit` high CVEs	**9**	**0**	**−9**

Note: Scores above reflect the original direct-comparison audit at the time of the v3 run. The unified re-audit (see unified table, above) revised these scores under the full seven-property rubric and a stricter Auditable behavioural criterion: T-v2 Auditable revised to 1/2 (ADR referenced-but-not-emitted retroactively disqualified); T-v3 Auditable revised to 2/2 (dep governance directives created a richer, independently auditable trail). The comparison table is preserved as the historical record of the condition that motivated the ADR emission fix.

Principal finding: The dependency governance condition eliminated all high CVEs (9 → 0). One specification directive — prescriptive package selection and an explicit `npm audit` gate — closes the entire vulnerability surface while maintaining all other GS properties.

Auditable regression — root cause: The v3 score dropped one point from v2's perfect 12/12. The model referenced `docs/adrs/ADR-0001-stack.md` in the README and emitted `CHANGELOG.md` as a structural stub with no entries. The auditor correctly penalized both: the ADR was referenced but never emitted as a file; the CHANGELOG satisfied the presence requirement but not the content requirement. The root cause is a precision gap in the "Emit, Don't Reference" instruction: the template specified "emit ADR stubs" but did not state that each ADR must be a fenced file block with substantive content in P1 — not merely cited in documentation prose and not as an empty placeholder. This is a template specification issue, not a GS architectural flaw.

Template fix applied; treatment-v5 complete — 14/14. Treatment-v2 originally scored 12/12 on the original six-property rubric; the unified re-audit above revised the Auditable dimension to 1/2 (reflecting that ADR emission satisfied the structural requirement but not the behavioral one), yielding 13/14 on the seven-property rubric. The ADR emission precision gap

identified through the v3 gap analysis (see §9.3) was patched in `templates/universal/instructions.yaml`. Treatment-v5 — the first condition to score under the seven-property rubric including Executable — achieved 14/14 (12/12 structural + 2/2 Executable) by separating infrastructure emission into a dedicated `00-infrastructure.md` prompt that must complete before any feature prompt, and by documenting the `jsonwebtoken` StringValue type pitfall explicitly in the specification. The root cause of prior Executable failures was not model capability — it was a known type narrowing pattern that the specification had not yet closed. Once named, it became a quality gate. The v4 materialize-verify-correct loop hypothesis is subsumed: v5 reduced correction passes to 2 (from v4's 5-pass exhaustion without convergence). Three simultaneous changes distinguish v5 from v4 — the infrastructure-first prompt, the Known Type Pitfalls entry for `jsonwebtoken`, and the ADR emission precision fix — and causal attribution among them is not isolated. The directional claim is that raising S before generation reduces correction passes; the specific contribution of each change is not separately established by this experiment. The 14/14 result closes the experimental loop. The session-verified result: 109 total tests, 10/11 suites, confirmed against a live PostgreSQL database across 2 fix passes (AI integration response reported 114; runner total of 109 is the source of truth; session run: 0 failures; independent re-run: 3 test-isolation failures in article.test.ts — duplicate user registration in a preceding test leaves token undefined in cleanup, not an implementation error). This session verification resolved the Executable scoring ambiguity that had applied to earlier audits of this condition; the companion supplement documents the resolution (§S13 Limitation 8). The v5 verification was conducted within the audit session; unlike RX, the jest-output.json artifact was not committed separately to the repository. The audit methodology is consistent across conditions. An independent replication — implemented in `github.com/jghiringhelli/generative-specification` as the **Replication Experiment (RX)** — requires committed `jest --json` output as a standard evidence artifact, making runner verification automatic and externally auditable without depending on the audit session. RX uses a scoped Conduit subset (user management, articles, profiles, and tags; comments and favourites explicitly out of scope per spec §1.1) with the unified seven-property rubric applied to the implemented scope; any reader can reproduce the Executable result against a fresh PostgreSQL instance by running the scripts in `experiments/rx/`. Because the GS document, the runner, and the scoring script are committed to the public `generative-specification` repository, they carry no proprietary dependency — any researcher with an Anthropic API key can rerun the full pipeline and commit their own evidence artifacts. ForgeCraft produced the GS document used in RX but is not required to reproduce the experiment; the document is the reproducible artifact. Full v5 supplementary data is in the companion supplement.

Full replication data — session IDs, prompt texts, blind audit transcripts, per-condition metric tables, mutation testing progression, failed runs disclosure, and replication instructions — are in the companion supplement: GS_Experiment_Supplement.md.

8. Implications for Practice

8.1 The Specification Precedes the Code

The shift required by generative specification is temporal: design precedes implementation, not the other way around. The architectural constitution, the C4 diagrams, the schema definitions, and at least a skeleton of the ADR structure must exist before the first AI-assisted implementation begins. This is not a new idea. It is an idea that was optional when the cost of skipping it was paid personally by a human engineer who could compensate with memory and informal communication. That compensation is not available to an AI session.

8.2 The Synthesis: Specification-First and Iterative Delivery

The two dominant delivery models of the last fifty years each carry the imprint of the context that produced them — and each was ill-fitted to software development in ways worth naming.[^royce-beck] Waterfall was borrowed from physical engineering at a time when software had no process discipline of its own. The analogy was intuitive: just as bridge design must precede pour because concrete does not refactor, so requirements should precede code. But code does refactor — until the accumulated cost of modification exceeds its benefit, at which point the project becomes unmaintainable and must be rebuilt. The insight was correct: design precedes implementation. The misapplication was treating software with the unforgiving change economics of structural steel.[^brooks-nsb] Agile's correction was real: delivery must be iterative, adaptive, and resistant to front-loaded design that turns out wrong. But agile was formulated for a specific class of software — SaaS, CRM, and web application work, where requirements are genuinely negotiable and user feedback is the binding constraint. Applied universally to embedded systems, game engines, data pipelines, and distributed infrastructure, iterative delivery without structural discipline does not produce adaptability. It produces drift.[^fowler-flaccid]

Generative Specification is not a third methodology alongside these two. It is what becomes possible once you assign each model's correct discipline to the layer it actually fits — and stop applying either to the layer where it was always wrong.

The specification layer runs waterfall. The architectural constitution, ADRs, structural diagrams, and behavioral contracts are complete before any agent session begins. This front-loading is not overhead — it is the precondition for the speed that follows. The agent cannot operate without a grammar. The grammar must be written first.

The delivery layer runs agile. Each session produces atomic, tested, deployable commits. The specification evolves through ADRs as the system and its requirements develop. Features are new production rules; bugs are delta reports between actual and specified state; each commit is a verified increment. Nothing about iterative delivery requires structural ambiguity.

The failure mode of waterfall — rigid front-loaded plans become wrong before construction ends — is resolved because the architectural constitution is a living document, revised through the same commit discipline as the code it governs. The failure mode of agile — iterative delivery without enforced structural discipline accumulates drift — is resolved because the specification gates every session. No agent output that violates the grammar enters the corpus.

What each model sacrificed for its primary virtue is preserved by the other layer. The specification provides the coherence agile historically lacked. The iterative delivery provides the adaptability waterfall could not sustain. These are not in tension because they operate at different altitudes in the same system. The uncertainty taxonomy in §9.4 shows what this synthesis looks like one level further down, at the verification layer itself.

8.2.1 The Economic Inversion

In traditional software development, implementation accumulates a sunk cost. The longer a system runs in production — accreting dependencies, supporting features, encoding undocumented behavior — the more expensive it becomes to discard. When a specification conflicts with an already-built system, the economically rational response has been to adjust the specification: not because the original problem definition was wrong, but because the code is load-bearing and the specification is not. Requirements drift toward the artifact because the artifact carries the cost.

Generative Specification inverts the cost curve. Implementation is cheap and repeatable: when the code is wrong, fix the specification and regenerate. The code carries no sunk cost because it was never the expensive artifact. The specification is not reliably recoverable from code alone — decisions, alternatives considered, domain knowledge, and accumulated rationale resist reconstruction with sufficient fidelity to serve as a generative grammar. Code is an implementation residue; the reasoning that shaped it leaves only partial traces in what was built. This changes where the discipline goes. Fix in the problem, not the task: when something is wrong, the first question is whether the specification correctly describes what is needed, and the resolution is regeneration rather than repair of what was already produced. Writing code becomes a commodity activity in the strict sense — reproducible on demand, interchangeable, abundant. The scarce resource is no longer the ability to write code. It is the ability to specify correctly.

[^royce-beck]: An underrecognized irony: Royce's 1970 paper introduced the sequential phase model on its second page, then spent the remainder of the paper explaining why it would fail in practice and proposing iterative corrections — feedback loops, overlapping phases, prototypes. What survived institutionally was the diagram on page two. The critique of waterfall is partly a critique of what practitioners selected from a paper that contained its own refutation. Waterfall: Royce, W.W. (1970). *Managing the Development of Large Software Systems*. Proceedings of

IEEE WESCON, 26. Agile: Beck, K. et al. (2001). *Manifesto for Agile Software Development*. agilemanifesto.org.

[^brooks-nsb]: Brooks, F.P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *Computer, 20*(4), 10–19. Brooks distinguishes accidental complexity — eliminated by better tools, languages, and environments — from essential complexity, inherent in the problem itself. Waterfall addressed accidental complexity by imposing process discipline. Its failure was the implicit assumption that software construction economics matched physical construction: that rework carried the same cost as demolishing concrete. In the medium where every layer is mutable until you decide to stop mutating it, that assumption does not hold.

[^fowler-flaccid]: Fowler, M. (2009). FlaccidScrum. martinfowler.com. https://martinfowler.com/bliki/FlaccidScrum.html. Fowler coined the term for teams that adopted agile's ceremonies — sprints, standups, retrospectives — while abandoning its technical disciplines: continuous integration, test-driven development, sustained refactoring. The result was organizational responsiveness layered over accumulating structural debt. The pattern is the specific failure mode identified here: iterative delivery adopted without the structural discipline it was designed alongside.

8.3 Commit Discipline as Corpus Quality

The git history of a generative specification system is a typed, scoped corpus. Each conventional commit is a sentence: a part of speech (feat, fix, refactor), a scope boundary (billing, auth, user), and a semantic payload (what changed and why). A history built of `fix bug`, `wip`, and `changes` is not a corpus — it is noise. A well-maintained conventional commit history provides a queryable record of how the grammar evolved — available as context in every session, without requiring anyone who was present to explain it. The Shattered Stars case study (§7.6) demonstrates precisely what is lost when this record is absent. The finding is specific: a generative specification is not a substitute for commit discipline. It is what survives when commit discipline is not applied.

A distinct variant of the same failure is an audit trail that exists but is not read as state. An Optuna-based hyperparameter optimization run demonstrated this: the journal file persisted every completed trial to disk, but the resume logic recalculated the target trial count without subtracting completed ones, and the result aggregator read only the current session's trials rather than the study-wide best. The effect was identical to absent history — the process restarted from trial zero and the prior session's optimum was invisible to the new one, despite full persistence. The artifact was not absent. It was not consulted. The Auditable property requires both: that the record exists, and that the next session begins by reading it.

8.4 ADRs as Persistent Memory

Every non-obvious architectural decision produces an ADR before implementation begins. Format is minimal: the decision, the context that produced it, the alternatives considered, and the consequences. This is not documentation for documentation's sake. It is the record that allows the AI to recognize intentional decisions and distinguish them from technical debt. Without it, the AI will "improve" them.

8.5 Names Are Production Rules

In a context-sensitive system, naming is not style. It is grammar. A function named `getUser` in a domain model that talks to a database is a violation of the architecture that the compiler will not catch, the linter may not catch, and a human reviewer will tolerate — but the AI will propagate. A function named `findUserByEmail` in a repository layer and `getUserProfile` in a service layer communicates ownership, scope, and responsibility through its name alone. That signal is available to the AI on every read.

The naming principle extends beyond architecture into technique transport. What a practitioner names in a specification, the AI knows how to apply. RAPTOR indexing (hierarchical codebase summarization at file, module, subsystem, and repository level) was first specified for CodeSeeker and propagated to BRAD, SafetyCorePro, and Conclave without any shared session context. The transport was the name. Every technique in the model's training corpus becomes available to any system whose specification names it. The specification is therefore not just an architectural grammar — it is a technique registry whose scope is the full depth of the model's training, activated at the cost of knowing the correct words to write.

8.6 The CLI as Execution Surface

The productivity results described in the case studies are not explainable by specification quality alone. They depend on a second condition: the AI has direct access to the CLI.

This changes the nature of the collaboration fundamentally. An AI that can only read and write files is an advisor: it can propose a migration plan, draft a deployment script, or describe how to configure a build chain. An AI with CLI access is an executor. It runs the migration. It deploys. It resolves the dependency conflict, reads the error output, selects an alternative approach, and retries — without returning to the engineer between attempts. The Stable Diffusion setup, the Railway data migration, the Python/CMake build environment, the git corpus — all of these were executed at the CLI level, not proposed for the engineer to run manually.

This is what makes the AI a superuser in the technical sense: it combines read and write access to the filesystem, execution authority over processes, access to package registries, network connectivity for API calls and downloads, and the ability to chain these operations across arbitrary sequences. A skilled engineer has all of these capabilities individually. The difference is

execution speed and the absence of context-switching cost. The engineer specifies the outcome; the AI exhausts the solution space at machine speed until that outcome is achieved.

A practical note on execution: across most sessions in the primary case studies — the exception documented in §7.6 (Shattered Stars) — commit operations, branch management, and merge conflict resolution were executed by the AI under the engineer's direction, rather than issued as direct CLI commands by the engineer. The discipline — atomic scope, conventional message format, one logical change per commit — is specified in the architectural constitution and enforced by pre-commit hooks. The AI executes against that spec.

The implication for generative specification is direct: the specification does not just govern code. It governs a system that can act. The architectural constitution, the commit policy, the deployment targets, the build constraints — all of these become operational rules for an agent that can execute them. The scope of what must be specified is therefore broader than code architecture alone.

This is not a contradiction of the paradigm's restrictive claim. It is its direct consequence. The pragmatic paradigm removes the option of leaving intent implicit — and in doing so, it makes the specification the operational grammar for every domain the agent touches. What the restriction takes away from the programmer (the ability to leave things unsaid) is exactly what the expansion gives to the agent (a domain-agnostic contract it can execute without asking). The restriction and the expansion are the same operation, observed from opposite directions: the downward triangle (Martin's) and the upward triangle (Chomsky's) do not contradict each other because, at their intersection, they share a vertex. The point at which programmer freedom reaches its floor is the point from which the specification's generative reach has no ceiling.

A concrete illustration: a full ETL pipeline deployed entirely on AWS — data ingestion, transformation stages, storage layers, monitoring dashboard, alerting, and the full set of non-functional requirements (retry logic, dead-letter queues, encryption at rest and in transit, IAM boundaries, cost tagging, observability) — was built without the engineer learning or typing AWS CLI commands, CDK constructs, or Terraform syntax. The engineer knew the services, understood the architecture, and described the desired infrastructure state. The AI issued every command: provisioned the resources, wired the IAM policies, configured the VPC, deployed the stack, and validated the result. Cloud infrastructure, under this model, is not a separate discipline requiring separate tooling expertise. It is another surface the AI executes against, given a specification of what the infrastructure must do.

The role inversion implied by all of this is worth stating plainly. In every case study in this paper, the engineer wrote no application code. The 16,229 lines added to SafetyCorePro, the 32,470 lines of Shattered Stars, the 253 tests of Conclave — none were typed by the engineer. The engineer wrote specifications: architectural constitutions, ADRs, structural diagrams, session-

scoped prompts, and acceptance criteria. The agent produced the implementation. A new feature is a production rule added to the grammar; the agent derives the implementation. A bug is a divergence from specified acceptance criteria; the agent diagnoses the delta and closes it. The engineer's instrument is no longer a code editor. It is a specification surface.

This is not automation of mechanical work. It is a structural inversion of what engineering consists of. The craft that scales is not typing — it is the precision, depth, and completeness of the grammar the engineer writes before the agent begins.

A further dimension of this specification scope concerns the AI's own execution environment. The performance of a session is itself sensitive to configuration: too many active tool servers in the context window leak tokens on every turn, because each declared tool is read by the model whether invoked or not. The practical ceiling demonstrated across the case studies is three: the AI assistant's built-in file, search, and terminal tools, supplemented by at most one or two project-specific servers — a specification generator for producing and maintaining architectural constitutions, and a semantic search tool for navigating large codebases. Beyond that ceiling, tooling overhead begins to compete with the specification for context budget. The specification governs what the agent produces; the execution environment governs how much of the specification the agent can hold.

The same principle applies to the architectural constitution itself. A document that grows without compression discipline defeats its own purpose: relevant rules are diluted by bulk, and the model's attention — finite across any context window — distributes less precisely as the document lengthens. This is not a property of any particular model class. It is a structural consequence of context-window economics. The solution is not a shorter specification — it is a maintained one. Tooling that detects scope drift and regenerates the document cleanly, or compresses an overgrown document to its essential rules without discarding custom sections, is the mechanical equivalent of the refactoring discipline applied to production code. The constraint is not length for its own sake. It is that the architectural constitution must remain readable as a grammar — not merely present as an artifact.

8.7 The Session Loop

The macro properties described in §§8.1–8.6 govern the stable structure of a generative specification system: what the architectural grammar contains, how its history is maintained, how decisions are recorded, and what surfaces the AI can reach. A distinct and complementary question governs the micro level: what must happen inside a single session to preserve that stable structure when the session ends?

The answer is an invariant. Every session must begin and end at the same *steady state*: code, tests, and documentation mutually consistent; the specification accurately describing what exists;

the commit history recording how it changed; Status.md capturing intent for the session that follows. A session that ends with passing tests but a stale specification has not completed the loop. It has produced a local improvement at the cost of a global inconsistency — one that compounds invisibly until a future session encounters a divergence between what the code does and what the documentation says it should.

The session loop has four phases.

Intake and clarification. The developer voices intent: a new capability, an observed failure, a noticed inconsistency in the existing system. Before implementation begins, the agent checks for two specific conditions: ambiguity (the request admits two or more interpretations that would produce different implementations) or unverifiable assumptions (the request presupposes facts about scope, schema, or behavior that cannot be verified from the current codebase). If either condition is present, one exchange resolves it — all clarifying questions batched into a single prompt, answered once, never revisited. If neither condition is present, implementation proceeds immediately. The check is narrow by design. It targets only ambiguities that would cause rework. A planning conversation is not a clarification exchange; a request whose meaning is clear does not require permission to proceed. The constraint on asking is as important as the obligation to ask.

Specification gate. Before any code is written, the agent answers one question: does this change *fit* the existing specification, or does it *change* it? A feature the current specification anticipates can be implemented directly against it. A feature the specification does not yet cover — a new module boundary, a new behavioral contract, a new interface the domain did not previously include — requires the specification to be updated first. The ADR, the schema change, the new section of the architectural constitution: these precede the implementation, not follow it. This is the gate most practitioners miss. Code written against the old specification is correct by local standards and wrong by the grammar it was supposed to serve. The specification is updated before implementation begins, and the implementation is written against the updated grammar. In that order, without exception.

Implementation and verification. The agent executes against the specification. Tests are written alongside the code they cover, not deferred to a future session. Before any commit, three conditions must hold: the full test suite passes, the feature is exercised at the HTTP or CLI boundary (not only unit-tested internally), and no new anti-patterns have been introduced — no hardcoded values, no bare exception throws, no diagnostic logging left in production paths. The commit records a verified state. It does not record a work-in-progress.

Documentation cascade. After a passing commit, the specification artifacts are restored to consistency with what was implemented, in fixed order: public-contract spec files if a new endpoint or CLI command was added; an ADR if a non-obvious architectural decision was made;

architectural and sequence diagrams if a new component or flow was introduced; the tech spec if the implementation diverged from the prior written spec; and Status.md, always. The cascade is not supplementary to the work. It is the operation that closes the loop. A session that ends before the cascade has produced a functional improvement and a specification debt — one that grows with every subsequent session that proceeds without noticing the inconsistency. The full protocol governing mid-loop inputs — observations, corrections, and new ideas voiced during development rather than derived from a planned roadmap item — is defined in §6.4; the ordering is the same in both cases, and the distinction between a planned item and an unscripted input does not change what the cascade requires.

The theoretical weight of this invariant rests on the Shattered Stars finding (§7.6). That case study demonstrates the proof by absence: a specification without commit discipline and without updated status artifacts is not a full steady state. The spec holds the behavioral contracts when nothing else does — but it does not hold the reasoning behind decisions, the order in which things were built, or the explicit record of what the next session needs to address. Full steady state requires all four artifacts: specification, tests, commit history, and Status.md. Any one missing adds cost to the opening of every session that follows, at compounding rate. §8.2.1's economic inversion applies at the micro scale: a spec update and a co-written test are near-free at session close. Deferred, they are paid at full price by the session that discovers the gap.

8.8 The Paradigm Beyond Code

The seven properties in §4.3 are stated for application code because that is the domain where the principle was first visible and most formally developed. But the double-triangle structure does not have a ceiling at the code layer. The case studies in §7 demonstrate AI execution across infrastructure, generative art, and operational data — and the same failure mode that produces architectural drift in an underspecified codebase produces arbitrary or harmful output in every other domain where constraints are left implicit.

The claim at each layer is identical: a specification that does not state the restriction produces output that is locally valid and globally wrong. The restriction type changes by domain. The mechanism does not.

At the infrastructure and artifact layer, the restriction is *acceptance criteria*: measurable contracts that bound what counts as a valid output. The Stable Diffusion art pipeline described in the case studies (§7.6.2) restricts each generated image against four quantitative checks: vertical symmetry above a pixel-similarity threshold, a bounded non-black border region, a principal-axis orientation within fifteen degrees of vertical, and a centering measurement. An image that fails any check is rejected and regenerated. This is structurally identical to a type check or a test assertion. The medium changed. The principle — state the constraint explicitly, make the acceptance criterion blocking and automatic — did not. The same logic governs infrastructure

provisioning: a cloud deployment specified as desired state (IAM policies, VPC boundaries, cost tags, encryption requirements) constrains what the AI is permitted to produce just as the architectural constitution constrains which module a class may live in.

At the business layer, the restriction has two axes, and both must be made explicit before instructing an AI to produce consequential output.

The first axis is **economic viability**: survival, revenue, margin, and growth rate are the acceptance criteria of a business strategy the way a symmetry threshold is the acceptance criteria of a sprite. An AI generating marketing strategy, content calendars, pricing decisions, or competitive positioning without these constraints produces output that is fluent, confident, and potentially ruinous. A content strategy that saturates a distribution channel and burns the audience is architecturally incoherent in the business sense: every individual piece passed a local check, and the system as a whole moved in the wrong direction. The specification that prevents this is not a temperature setting on a model. It is a stated economic constraint: target conversion rate, sustainable publishing cadence, audience retention threshold, cost-per-acquisition ceiling. These are the business layer's quality gates.

The second axis is **legal and ethical compliance**: jurisdictional requirements, regulatory constraints, data rights, contractual obligations, professional ethics, and moral commitments are not soft preferences. They are the constraints that define what counts as a valid sentence in the domain of business decisions. An AI that generates a pricing strategy that crosses into collusion, a content piece that violates a contributor's rights, or a communication that misrepresents a product has produced output that passed no stated acceptance criterion — because no acceptance criterion was stated in those terms. The failure is structural. The output is the natural consequence of an impoverished grammar.

The distinction between a policy document and a generative specification is the same at the business layer as at the code layer: the constraint must be blocking and automatic, not advisory. A content calendar with a stated audience retention threshold and a validator that flags output falling below it is structurally equivalent to a pre-commit hook that rejects a PR without test coverage. A content calendar with a "brand voice" section and no enforcement mechanism is a README. A pricing floor stated as a named constant with an automated check against every AI-generated proposal is a production rule. A pricing floor mentioned in a strategy document that an AI may or may not have been given is an assumption. The test is not whether the rule exists. The test is whether the system rejects output that violates it without requiring human judgment to notice.

The productive conclusion is not that businesses should not use AI. It is that the discipline the pragmatic paradigm imposes on engineering teams — state your constraints explicitly before instructing the agent — applies with equal force to every function the AI touches. In code, the

constraint vocabulary is the architectural constitution. In generative media, it is the acceptance criteria. In business, it is the economic logic, the legal boundaries, and the ethical commitments of the organization. These are not new constraints. Every competent operator already navigates them. The paradigm's contribution is the insistence that they must be *externalized* — written down, formally stated, made structurally present in the specification the AI reads — rather than assumed to operate through judgment and institutional culture that the AI does not have access to.

The same conjecture extends, as hypothesis, to every knowledge-work domain where output can be evaluated against stated criteria: graphic and product design, where brand constraints, usability heuristics, and visual acceptance criteria constitute the specification; financial analysis, where model assumptions, risk thresholds, and regulatory constraints define what counts as a valid output; academic and professional research, where reproducibility requirements, methodological constraints, and citation standards bound acceptable derivations; and content production across every format, where audience, register, length, and strategic alignment are the acceptance criteria. This paper does not attempt to prove the claim for each domain — empirical validation across non-engineering fields will require practitioners in those fields. The mechanism is identical: without externalized constraints, the AI produces output that passes no stated criterion because none was stated. The failure mode does not change when the medium changes. Other disciplines will confirm or qualify the conjecture. The invitation is open.

8.9 The Prompt Engineering Objection

The most common practitioner objection to the paradigm framing is that the problems described here — architectural drift, incoherent output, accumulated context loss between sessions — are prompt quality problems, not structural ones. Better prompts, this argument goes, solve all of them.

The objection misidentifies the failure layer. A prompt is a session artifact: it exists for one interaction and disappears when the context window closes. Architectural drift accumulated over thirty sessions is not the product of thirty bad prompts. It is the product of a context that degrades faster than any individual prompt can repair it. Improving the prompt in session thirty does not restore the architectural coherence that was eroded in sessions one through twenty-nine. The prompt describes what to do now. The specification describes what the system is. These are not the same artifact and one cannot substitute for the other.

The temporal and structural distinction is precise. The architectural constitution, the ADRs, the named conventions, and the quality gates are not enriched prompts. They are the grammar the model reads *before* any session prompt is submitted — and they persist across every session, not

just the current one. When the Shattered Stars AI executed sixteen game systems from a set of session-scoped prompts without carrying context forward, it produced structurally coherent output because those prompts were written against a 2,277-line platform-independent specification (§7.6). Remove the specification; the same prompts produce sixteen systems with sixteen different architectural vocabularies. The coherence is produced by the grammar, not by the prompt.

A stronger form of the objection is that models will soon have context windows large enough to hold the entire codebase, removing the need for externalized specifications. This confuses memory with grammar. A model that can read every file in a repository still requires that those files constitute a coherent specification — that names are intentional, boundaries are explicit, decisions are recorded, and contracts are stated. A large context window containing an underspecified codebase does not produce coherent output. It produces larger-scale incoherence, faster. The constraint is not context length. It is specification quality. The paradigm's demand does not go away when the window grows.

8.10 The Failure Mode: A Wrong Specification

The most important risk of generative specification is not an underspecified system — it is a *wrongly* specified one. A faithful AI executing a flawed architectural constitution will produce flawed code at scale, with high confidence and no complaint. The specification being a well-formed grammar does not guarantee it is the *right* grammar.

Three practices mitigate this. First, the specification should face the same verification discipline as the implementation: before any code is written, concrete behavioral outcomes should be defined and made checkable. ADRs serve this function in part — if a decision's stated rationale does not survive being written down, it was not sound. Second, the specification must be treated as a living document, revised through the same atomic commit discipline as the code it governs. An architectural constitution written at project inception and never revisited is a static grammar for a living system. The ADR record exists precisely to document when and why the grammar must change — and to make those changes visible, intentional, and recoverable.

The third practice is the most fundamental, and the one no process can supply: the correctness of the specification is bounded by the specification author's domain depth. GS raises the floor — a practitioner following the methodology will produce a specification that is better than no specification, and the structural properties it enforces are non-trivial. The ceiling is set by something else entirely: the depth of the practitioner's engagement with the domain, the precision of their naming, the judgment to recognize which design decisions are architecturally load-bearing and which are arbitrary. GS makes a correct specification powerful. It cannot make an incorrect specification correct. A wrong specification executed faithfully by a capable agent is a wrong system built at generation speed with no complaints. The appropriate response is not to

treat GS as self-correcting but to treat specification authorship as the accountability-bearing act it is, and to design ADR and review practices that subject the specification surface to the same adversarial discipline as the code it generates.

A fourth practice — **specification expansion through meta-completeness querying** — addresses the gaps the first three cannot: the things the practitioner does not know they are missing. The method is deliberate: having specified the system as completely as their current domain depth permits, the practitioner asks the model directly what dimensions of correctness, completeness, or structural integrity the specification does not yet address. The model, operating at the intersection of the stated domain and its pre-trained breadth, returns possibilities the practitioner has not named — not by cataloguing what similar products do (though feature parity is one class of return) but by activating domain-specific correctness requirements the system should satisfy from first principles. When BRAD's family law case analysis was queried this way, the model returned two structural gaps the specification had not closed: the infraction taxonomy needed a cross-dimensional mapping to the twelve statutory grounds recognized by Minnesota family law, and citation accuracy required cross-referencing against the MN API public case database to verify that cited precedents are real and correctly attributed. Neither gap was visible from inside the specification. Both were correctness requirements derivable from the domain structure. The first is a classification completeness problem; the second is a verifiability problem — and both are the kind of silent omission that produces a confidently wrong system when GS executes against an incomplete specification. The meta-completeness query surfaces them before execution begins. The practitioner decides what returns enter the specification; the model opens the possibility space. The loop is: query, evaluate, specify, commit. This is the same discipline as §8.10's first three practices — the specification faces adversarial scrutiny — but the adversary here is the domain itself, mediated by a reader who already contains it.

§8.11 extends this practice guidance to the full hardening surface.

8.11 Hardening as Specification

The adversarial posture of the Verifiable property — tests designed to fail on incorrect code — extends naturally to the full hardening surface. Stress testing, security testing, chaos engineering, cross-cutting concern validation, and environment auditing each follow the identical structure: specify the adversarial or compliance condition, define the acceptance threshold, execute, report divergence. In every case the test is designed to break the system, reveal a gap, or expose an assumption — not to confirm it functions. A system that passes has been proven against its own stated limits. A system that has never been challenged has only been proven against itself.

Category	Constraint Vocabulary	Representative Tooling

Stress & performance	Peak concurrent users, sustained request rate, latency ceiling (p99), error rate threshold; soak, spike, and scalability ceiling variants	k6, Artillery, Locust
Security	Threat model: authentication bypass attempts, injection payloads, dependency vulnerability scan, CORS policy, secret exposure, privilege escalation; severity acceptability threshold (Invellum: zero critical findings)	npm audit, Snyk, OWASP WSTG, ZAP
Chaos engineering	Resilience contracts: recovery time after node kill, dead-letter injection, DB failover window, circuit breaker open/close thresholds; property-based testing extended to infrastructure	Chaos Monkey, Gremlin, custom fault injectors
Cross-cutting concerns	Encryption policy (TLS version, cipher suite, at-rest, secret rotation); authorization model (RBAC/ABAC per surface, agent generates tests from insufficient-permission contexts); observability schema (correlation ID, PII redaction, SLO/SLI thresholds); data lineage contract (provenance specification: where data originates, how it transforms, and where it terminates); dependency compliance (CVE threshold, license policy)	TLS auditors, log schema validators, npm/pip audit
Environment hardening	TLS headers, Content Security Policy, no exposed secrets, IAM least-privilege boundaries, CORS policy correctness — agent audits running environment against spec and closes the delta	Cloud provider policy tools, Trivy, tfsec

The common failure pattern across all hardening categories mirrors application architecture: concerns fail not because engineers are unaware of them, but because they were never stated as blocking acceptance criteria. The specification does not add new requirements. It makes existing ones structurally present — explicit, enforced, and verifiable.

---

9. Discussion

The evidence from six production case studies points to four concrete changes this shift requires.

The design process changes: specification precedes code, not accompanies it. *(Shattered Stars §7.6: the 2,277-line platform-independent specification was written before a TypeScript file was started; SafetyCorePro §7.1.2: CLAUDE.md was completed before the first implementation commit.)*

Team composition changes: specification is a first-class engineering skill. The skill being elevated by this transition is not a new programming paradigm, not a framework, not a tool. It is the ability to decompose a problem, name its parts with precision, define behavioral contracts unambiguously, and express architectural intent in a form that leaves no important gap unfilled. This skill has always existed in the best engineers and architects. What changes is that it is now the productivity multiplier — the factor by which AI assistance is either leveraged or wasted. Teams that invest in specification skill will compound. Teams that treat it as overhead will find AI assistance producing confident, fluent, and architecturally incoherent output. *(BRAD §7.5: extending a sovereign legal intelligence engine with prosody and argumentation analysis — the domain breadth activated by the specification was not present in the AI's default output; it required naming.)*

The specification skill this paradigm rewards is not monodisciplinary. The practitioner who has worked at the intersection of formal language theory, software architecture, data science, legal reasoning, and philosophy holds not one domain but the connections between domains — and it is at those connections that problems resistant to generalist AI output live. Hyper-specialization made breadth economically irrational for most of the twentieth century: career depth in one vertical was the reliable path, and the generalist was suspect — broad but shallow, informed but not expert. The Renaissance ideal — the educated person as a navigator of philosophy, mathematics, art, and natural philosophy with equal facility — had been bracketed as a historical curiosity admirable in an age before knowledge grew too large for any one mind to hold. What the current moment restores is not that one mind can hold all knowledge. It is that one mind spanning multiple domains can direct a tool that holds encyclopedic knowledge in all of them, and activate its deepest capabilities by naming the correct dimension at the correct moment. The AI carries depth; it cannot supply the judgment to recognize which depth is relevant. Specification skill compounds for exactly the kind of practitioner this paradigm rewards. The AI does not replace the Renaissance practitioner — it finally gives one the leverage that the cost of execution across domains previously blocked.

Three words, borrowed from different intellectual traditions, name what that leverage amplifies specifically. *Synthetic*: the cross-domain practitioner does not merely hold multiple domains in parallel — combining them produces insight that neither domain holds alone. The specification that names RAPTOR indexing and formal fallacy classification in the same document is not two specifications; it is one that neither a retrieval engineer nor a legal logician would write independently. The synthesis is itself the contribution, not a sum of parts. *Synaptic*: the productive surface is the connection between domains, not the domains individually. The practitioner who holds the joint between formal language theory and legal reasoning sees the problem that pure software engineers and pure legal specialists each miss — the domains are two cells, but the synapse between them is where the signal travels. *Synoptic*: holding multiple domains in one view — not sequentially but simultaneously — enables pattern recognition across

them that a specialist rotating between disciplines cannot achieve. The synoptic practitioner does not alternate between software architecture and philosophy; they hold both at once and see that the structural pattern has a name in one discipline and a concrete implementation in the other.

In team composition, this means specification quality scales not only with aggregate domain depth but with connection density — the number of domain boundaries the team can hold explicitly and name precisely in the specification. A team of deep specialists writes specialist specifications; a team whose members have worked across each other's domains writes a specification where the joints between layers are as explicit as the layers themselves. The restriction is the expansion at the team level: the discipline of naming which domain the problem occupies — precisely enough that the AI activates the correct depth — is what allows a cross-domain team to direct a cross-domain executor without losing the seams. What is restricted is implicit assumption; what expands is the surface of explicit intent the executor can reach.

Quality measurement changes: structural coherence joins test coverage as a first-class metric. Test coverage measures whether the system does what was intended. Generative specification adds a second axis: does the system's structure match its architecture? Are the boundaries where they are supposed to be? Are the names carrying the signal they are supposed to carry? Is the git history a legible corpus or noise? These are not soft quality indicators. They are the artifact properties that determine whether the next AI-assisted session will extend the system correctly or introduce drift. Measurement should track both. *(SafetyCorePro §7.1: commit progression 75 → 97 → 218 → 285 → 339 → 346 → 484 tests across ten commits, each traceable to a specification change.)*

The engineer's domain breadth becomes a direct productivity multiplier. The BRAD case study (§7.5) provides the sharpest evidence for this observation, supported by supplementary examples from the broader body of work — an AWS ETL pipeline (§8.6) and a quantitative finance system — that reinforce the principle at different domain specificity. The AI does not produce specialist output by default. It produces output at the level of specificity the specification signals. The author names this *domain dimensional expansion* (§7.5): naming a domain in the specification activates the model's full training depth for that domain — not as retrieval but as calibration. When BRAD (askbrad.ai) — a sovereign legal intelligence engine for US family law cases, built on a prior takeover refactor (§7.1 methodology) and deployed to Railway — was extended with prosody and argumentation analysis, naming those domains in the specification is correlated with the AI immediately activating discourse analysis, formal fallacy classification, and deontic modal logic: a niche of legal reasoning covering obligation, permission, and prohibition that appears nowhere in a generalist software prompt. The specification also directed the construction of a case taxonomy — an AI-declared orthogonal classification system — and a property graph knowledge layer, both of which emerged from stating what the analysis required rather than how to build it. The specification was not just an architectural document; it was an epistemological signal about which domain the problem occupied.[^5]

[^5]: Naming the domain activates the corresponding depth of the model's training corpus — the inverse of the drift failure mode. Underspecification produces generic output at the level the prompt signals; precise domain naming produces specialist output calibrated to the discipline named. Both are properties of the same mechanism: the specification as the grammar the reader activates against.

The same phenomenon operates across any domain the specification can name. An engineer who knows heuristic search with pruning, backtesting methodology, and Bayesian classifier chaining can describe a consensus prediction strategy and receive the implementation; one who can name RAPTOR indexing, vector embeddings, BM25, and a knowledge graph can describe a coherent retrieval architecture and receive one. Without the naming, the AI produces a generic solution. With it, it produces the exact instrument described. The same principle applies below the application layer: describing the desired state of a build environment — a working module, a passing test — is sufficient for the AI to resolve the dependency chain without the engineer managing the toolchain directly.

A named technique also travels without session context. RAPTOR indexing was first specified for CodeSeeker's hybrid retrieval architecture and subsequently named in BRAD, SafetyCorePro, and Conclave — carried by name in each project's architectural constitution rather than by shared memory (§8.5). CodeSeeker is the origin instance: the specification named RAPTOR indexing alongside BM25, vector embeddings, and knowledge graph traversal; the AI composed them into a unified retrieval layer; and the structures built for search yielded semantic duplicate detection and dead code discovery as derivable consequences, no separate data model required.

9.1 Agentic Self-Refinement

A pattern visible across several of these surfaces deserves explicit naming: **agentic self-refinement**. Wherever the desired output can be specified and the actual output can be observed, the agent can close a feedback loop on its own execution, without human intervention between cycles. The Stable Diffusion pipeline (§7.6.2) is the simplest instance: generate, evaluate against measurable acceptance criteria, regenerate if criteria fail. The loop also operates in the evaluation direction: a multimodal vision model applied against an *existing* asset library — not to generate but to assess — performs automated specification compliance at a scale manual review cannot match. Shattered Stars applied this pattern to sprite sheets, submitting art assets to an image-to-text model with an explicit specification of expected art direction, symmetry bounds, orientation rules, and palette constraints. The output was structured identification of violations — sprites with incorrect facing orientation, palette deviations, style inconsistencies against the reference — produced with a completeness and speed impractical for human review of large asset libraries. The accepted art specification is the desired state; the multimodal model is the evaluator; the loop is the same.

But the loop operates at every level where output is observable. The batch size and VAE precision adjustment in that pipeline was hyperparameter tuning: the agent read its own performance output, compared it against a specification of required throughput and quality, and adjusted its generation parameters accordingly. The same pattern in a quantitative prediction context looks different in mechanism but identical in structure: a strategy engine specifies acceptance criteria — minimum win rate, maximum drawdown, Sharpe ratio floor — runs a backtest against historical data, reads its own performance output, and adjusts classifier thresholds, feature weights, or pruning heuristics before the next evaluation cycle. The agent does not ask whether to tune; the specification defines what constitutes acceptable output, and anything below that threshold is an automatically-triggered adjustment cycle.

A more general form is agentic self-evaluation: given a running session log — a form of memory that the Status.md pattern approximates — a subsequent session starts not from a blank context but from a specification-informed account of what the prior session achieved, where it stopped, and what it tried. The agent evaluates its own prior output against the specification before beginning new work, and can adjust its strategy based on what failed. This is the recursive form of the methodology: the same loop that governs whether a TypeScript module satisfies an interface governs whether the agent's prior approach satisfies the session objective. The implication is that the scope of the methodology is bounded only by the engineer's ability to define acceptance criteria. Any domain where desired state can be specified and actual output can be observed yields to this loop. The surfaces are not a finite list. They are a consequence of a principle.

The two axes in tension from §4 converge here. The restriction — removing implicit context — is the floor from which the expansion reaches any domain where desired state can be stated and actual output observed. Across the case studies, the same structure — desired state, acceptance criteria, agent iteration — governs every surface the AI touched:

Domain	Constraint Mechanism	Evidence
Application & data architecture	Layered services, repository interfaces, named domain models, cross-language interface contracts, retrieval architecture composition (embeddings + BM25 + RAPTOR + knowledge graph, fused via RRF)	SafetyCorePro, Invellum, ForgeCraft, Conclave, CodeSeeker, BRAD
Infrastructure & environment	Cloud resource desired-state provisioning; toolchain configuration described as desired state and resolved iteratively without engineer-issued platform-specific commands	Invellum (Railway), Shattered Stars (Vercel, SD environment), AWS ETL
Generative asset pipelines	Executable acceptance criteria on AI-generated outputs: symmetry threshold, background	Shattered Stars (§7.6.2, §7.6.3)

	validation, orientation angle (PCA), audio LUFS normalization; multimodal model evaluation of existing assets against style specification	
Agentic self-refinement	Generate → evaluate against spec-defined acceptance criteria → adjust parameters or session context → regenerate; loop operates identically at image generation, hyperparameter optimization, and session resumption	Shattered Stars, quantitative finance classifier, BRAD session logs

9.2 The Interface Layer: From Screen to Ambient Orchestration

This section describes a direction the methodology points — a specific constraint the author encounters at the current scale of practice, and one architecture that might address it. It makes no empirical claims and proposes no formal experiment; it is included as an honest account of where the paradigm's structural argument leads when tested against lived experience.

At fifteen active projects cycling through structured waiting states, the binding constraint has shifted. Execution is not the bottleneck — a waiting project costs nothing. Status management is: knowing which projects have cycled to ready, what each one needs next, and being able to provide direction without stopping what else is being done. A screen solves this when the engineer is seated in front of it. It is useless otherwise.

The interface that would close this gap is legible from the methodology's own structure: a persistent ambient status layer, visible at the periphery, accepting voice-directed updates, routing decisions back to the right session without requiring a keyboard. AR glasses exist; ring controllers and wristband sensors exist. What does not yet exist — and what the methodology requires before any of it can function — is the specification layer: a grammar that decides which signals rise to attention, at what summary depth, through which modality. That is a GS problem applied to the engineer's own attention rather than to a codebase.

The signal routing taxonomy that layer requires is established practice. Allen (2001) identifies three categories that map directly: *pull* signals require the practitioner to initiate a check; *push* signals interrupt current activity; *ambient* signals persist in the periphery without demanding response. The command center's specification settles two explicit rules before the hardware is turned on: a *maintenance window rule* that batches non-urgent signals — PR notifications, dependency alerts, non-blocking failures — into a scheduled review window; and an *emergency filter rule* that defines, by named criteria, what bypasses the queue entirely. Everything not named is a maintenance item. A command center without these two rules is an inbox.

Meetings are the same problem in a different channel: unstructured natural language carrying decisions, action items, and open questions that disappear unless externalized. A live transcript

processed against the attendee's stated priority schema produces a structured artifact — decisions made, action items with owners, open questions marked pending — without requiring disengagement from the conversation. The session invariant from §8.7 applies: the loop closes when decisions are recorded and action items are queued to the relevant project's Status.md. The interface changes. The discipline does not.

This is a yearning more than a plan. The problems are real; the hardware that could address them is available; the specification discipline that would make the orchestration layer tractable is the same discipline this paper argues for everywhere else. Whether that convergence produces a leveraging interface or simply a better notification filter, the author leaves as an open question for practitioners working at this scale.

9.3 Template Gap: ADR Emission Precision — Diagnosed, Patched, Validated

The v3 post-hoc run (§7.7.B.1) identified a residual precision gap in the GS template that the treatment-v2 "Emit, Don't Reference" directive did not fully close. The template stated "emit ADR stubs in P1" but did not specify that the emission must be a fenced file block with substantive content — not a reference in documentation prose, and not an empty structural placeholder. The model honored the structural requirement (an ADR is mentioned; a CHANGELOG entry is present) without honoring the behavioral requirement (the file is present in the project output with content an auditor can read and evaluate).

Applied fix. The `auditable` block in `templates/universal/instructions.yaml` was updated with three specific changes:

1. *Minimum ADR set.* Emit at least three ADRs in P1: stack selection, authentication strategy, architecture decisions. Each ADR must contain substantive content in Status, Context, Decision, and Consequences — no placeholder fields reading "TBD".
2. *Reference-check invariant.* If a file is named in the README or in documentation prose within P1, that file must appear as a fenced code block in the same response. A referenced-but-absent file fails the Auditable criterion regardless of whether its prose description is accurate.
3. *CHANGELOG initialization.* The initial `CHANGELOG.md` must document actual P1 decisions rather than emitting an empty `## [Unreleased]` block. The first entry records the project initialization: stack chosen, auth strategy, any non-obvious architectural decision made in P1.

This fix propagates to all GS-governed projects on the next `forgecraft refresh_project` or `setup_project` run. The diagnosis is recorded in `experiments/white-paper/metrics.md` in the

forgecraft-mcp repository, which explicitly states: expected GS with fix = 12/12; experimental re-validation is the purpose of the v4 run.

Status: Complete. Treatment-v5 achieved **14/14** — the first perfect score under the seven-property rubric where the Executable dimension is session-verified (verify loop confirmed 109 total tests against a live database; independent re-run: 106/109 passing, 3 test-isolation failures in article.test.ts — not implementation errors), not auditor-inferred. Root causes: Defended/Auditable gap closed by separating infrastructure emission into a dedicated `00-infrastructure.md` prompt executed before any feature prompt; Executable gap closed by documenting the `jsonwebtoken` StringValue type pitfall in CLAUDE.md Known Type Pitfalls. The verify loop converged in 2 passes (v4 exhausted 5 without converging). Four runner infrastructure bugs were fixed before the confirmed run: a `prisma migrate deploy` no-op that left the database empty, fix-prompt context gaps (erroring files and failing test files not included), and a JWT_SECRET too short for the model's own >=32-character enforcement — all committed and documented in the companion supplement (§S9.6). The session-verified result: 109 total tests (independent re-run: 106 passing, 3 test-isolation failures in article.test.ts — not implementation errors), 10/11 suites (109 total runner-confirmed; 114 AI-reported — see table footnote †).

The epistemic finding. Treatment-v3 independently achieved 14/14 on the unified rubric without a verify loop. The auditor inferred Executable 2/2 from static artifacts: code compiles, tests are written, structure confirms. Treatment-v5's 14/14 is the only one backed by a passing test suite against a live database. Same score, completely different epistemic basis. This is the core finding the verify loop was built to establish: the gap between "auditor says it works" and "runner proves it works." A specification that produces inferably-executable output is necessary; one that produces verifiably-executable output is sufficient. Treatment-v3 demonstrates the first; treatment-v5 demonstrates the second. The v4 hypothesis (that a post-generation verify loop closes the runtime gap) is confirmed, but the more consequential finding is that raising S before generation — through the infrastructure-first prompt and type pitfall documentation — reduced the loop to 2 passes from a maximum-exhausting 5.

9.4 I(S) ≈ 1/S: From Theoretical Claim to Running Instrument

The central quantitative claim threading through this paper is that the expected number of correction iterations a session requires is inversely proportional to the specification completeness at the point the session begins: $I(S) \approx 1/S$. This is a deductive theoretical claim rather than an empirically-fitted one: it follows from the definition that an incomplete specification leaves freedoms the executor fills arbitrarily, and each such freedom becomes a correction cycle. The experiment series provides directional support — five conditions with monotonically

increasing specification completeness and monotonically decreasing gap-to-target — but the formula has not been statistically fitted to data. A proper regression would require independently measuring S before each run and correlating it against measured iteration counts; that measurement was not part of the current experiment design. The claim is offered as a falsifiable theoretical frame, not as an equation with fitted parameters.

The convergence geometry. The $1/S$ relationship has a shape worth naming explicitly. It is a hyperbolic curve, not a linear decline — which means the returns are front-loaded. Moving from $S = 0.1$ to $S = 0.3$ reduces expected iterations by approximately two-thirds; moving from $S = 0.8$ to $S = 0.9$ reduces them by roughly 12%. The correction loop closes like a narrowing spiral: wide arcs at low S, progressively tighter as the ceiling approaches. The experiment series shows this shape directly: the largest score jump is Naive→Treatment (3→9, six points from the first GS artifacts introduced); the final conditions each move one point against a floor already at 13–14. Those final tightenings are not evidence of diminishing value — the last gaps (Executable, ADR emission precision, known type pitfalls) are often the most architecturally significant. They represent diminishing *surface area*: each constraint added removes a smaller slice of the remaining unconstrained space, because the earlier constraints have already closed the wide arcs. The spiral narrows continuously at the cost of increasing precision per unit of remaining gap.

The treatment-v2 result (12/12) and the v3 Auditable regression (11/12) together constituted a diagnostic loop: a gap was observed, its cause was traced to a specification precision deficit, a template fix was applied, and treatment-v5 closed the loop at 14/14 — the first perfect score under the seven-property rubric including Executable. That loop — observe gap, diagnose specification deficit, patch template, re-run — is exactly what $I(S) \approx 1/S$ predicts. The template had a low-S region (ADR emission directives). The correction iterations concentrated there. Raising S in that region is the content of the applied fix.

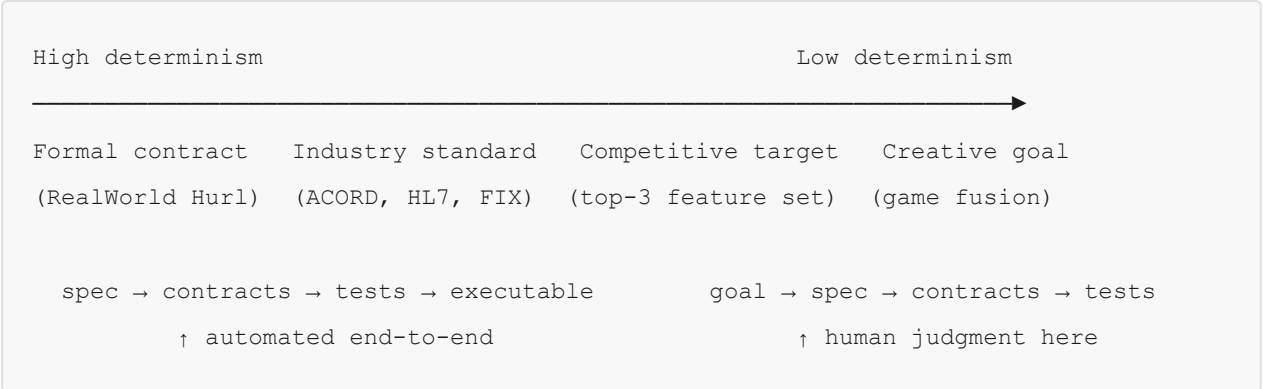
Each experiment condition raised S in a distinct dimension. The multi-condition progression is not a sequence of incrementally better prompts — it is a sequence of specification completeness increases, each targeting a different dimension of S that the prior condition had left unconstrained. The gaps between conditions are not method failures; they are exactly what $I(S) \approx 1/S$ predicts: dimensions where S was still low concentrated the remaining iterations.

Condition	Dimension of S raised	Observable effect
Control	Baseline expert prompting	Reference point — structured output without GS artifacts

Treatment	Architecture ADRs, CLAUDE.md, pre-defined schema	+1 GS score (Composable); structural rework cycles reduced
Treatment-v2	Explicit emit directives for infrastructure; First Response Requirements	13/14 — Defended fully closed; Auditable partially closed (1/2 → ADR structure present but behavioral requirement not fully honored); CVE gap exposed
Treatment-v3	Dependency governance (package registry + audit gate)	14/14 (auditor-inferred Executable); 0 high CVEs; ADR emission precision gap exposed
Treatment-v4	Verify loop (materialize → tsc → jest → correct, max 5 passes)	11/14 — verify loop added but loop-driven fix prompts omitted the erroring files and failing test files from context, causing the model to re-generate infrastructure files (hooks, CI configuration) that had already been emitted in P1, overwriting them with incomplete versions; Defended and Auditable regressions followed from those overwrites. Loop exhausted all 5 passes without converging on Executable.
Treatment-v5	Infrastructure-first prompt + Known Type Pitfalls	14/14 session-verified: 109 total tests (106 passing in independent re-run; 3 test-isolation failures in article.test.ts, not implementation errors), 2 fix passes; $I(S)$ converged

The CVE gap in treatment-v2 and the ADR emission gap in treatment-v3 are not anomalies. They are the predictable signature of a method that raises S in one dimension while leaving others unconstrained. Each gap identified where S was still low. Each fix raised it. Treatment-v4 and treatment-v5 should be read as a two-part hypothesis test: v4 isolated the verify loop alone — and its regression established precisely what the loop requires to function correctly: complete context for fix prompts, not a bare error list. Treatment-v5 is the combined test, supplying that context through the infrastructure-first prompt and closing the remaining gap with the known-type-pitfalls constraint. The v4 regression is not noise in the progression — it is the evidence that makes v5's convergence interpretable: we know the loop converges because we know what it needs, and we know what it needs because v4 showed what happens when it is absent. Treatment-v5 closes the runtime question: raising S before generation reduced the verify loop from 5 passes (no convergence in v4) to 2 passes (convergence in v5). The open research question is now whether the practitioner study replicates this convergence behavior across engineers with varying specification skill.

Specification Determinism. The convergence behavior of $I(S)$ depends on how precisely the desired output can be stated before generation begins — a property we call *specification determinism*. The spectrum runs from fully deterministic to fully exploratory:



At high determinism, contracts *are* the specification. A healthcare system conforming to HL7 FHIR, an insurance system using ACORD XML schemas, a financial exchange implementing FIX protocol — each provides a machine-readable contract that directly derives the test suite. The verify loop has something rigorous to check against, and $I(S) \approx 1$ approaches. GS score and runtime correctness both converge automatically.

At low determinism, the desired state is expressible in language — “fuse these two game aesthetics”, “capture the 20% of competitor features driving 80% of their retention”, “optimize for ROI with drawdown-weighted Pareto across the latest factor models” — but cannot be automatically compiled into a test suite. Something must encode “this is the 80/20 feature set” before a Hurl-equivalent can check for it. The model’s highly skilled average execution means it will produce *a* correct implementation of the prompt; without GS, it will produce the population mean of whatever was asked. With GS, the ADRs, use-case documents, and NFRs encode the human’s actual intent before generation — so the model’s skilled average runs against the stated target rather than the most common interpretation.

Human judgment does not decrease at high determinism — it moves upstream. The common misreading of this spectrum is that lower determinism requires more human involvement. The correct reading is that determinism determines *where* human judgment is applied, not *how much*. At high determinism, the judgment was already encoded by whoever wrote the formal contract; GS executes against it. At low determinism, GS forces the human to encode their judgment *before* generation through artifacts — ADRs, use cases, NFRs — rather than after through correction. The cost of late-stage correction scales with determinism: a missed FHIR constraint is caught by a test in seconds; a wrong aesthetic direction in a game costs days of art and code. The lower the determinism, the higher the return on making human judgment explicit upfront through GS artifacts, because the iteration cost that GS avoids is proportionally larger.

The Executable property — the seventh GS dimension. The six structural properties of §4.3 measure whether the specification was correctly honored. Executable (§4.3) adds the runtime dimension: does the generated output pass its behavioral contracts when exercised against a real execution environment?

Score	Criterion
0	No specification available, or generated code does not compile
1	Specification available; generated server partially satisfies it (>0%, <80%)
2	Specification available; generated server substantially satisfies it (≥80%)
N/A	Goal-directed or exploratory program; automated contracts not derivable

The N/A case is not a failure — it is the operationalization of low-determinism programs. The verify loop can only automate what a specification makes verifiable; at low determinism, the acceptance criterion is a human rubric, not a machine check. Treatment-v4 confirmed that the loop was necessary — and that loop exhaustion without convergence (5 passes) pointed to context-gap causes rather than loop structure, a finding resolved by v5's infrastructure-first approach.

ForgeCraft has since operationalized this claim as a measurable project property, closing the loop between theoretical argument and running instrument.

Uncertainty taxonomy. Not all verification is equivalent. A five-level taxonomy classifies the uncertainty class of each verification step, which determines the maximum completeness S achievable by automated verification alone — the *completeness ceiling*:

Level	Domain examples	Verification technique	Ceiling (estimated)
Deterministic	Type contracts, schema validation, API conformance	`tsc`, OpenAPI diff, Hurl contract tests	~0.95
Behavioral	UI flows, navigation, integration end-to-end	Playwright + Claude Vision	~0.90
Stochastic	Game balance, financial simulation	Monte Carlo, VaR/CVaR bounds	~0.85
Heuristic	ML training, hyperparameter search	Hyperband pruning, plateau detection	~0.75
Generative	Art pipelines, content quality, creative output	Aseprite MCP, human approval gate	~0.60

(Values are working estimates, order-of-magnitude only, not empirically calibrated — see note below.)

The ceiling is the fraction of the acceptance surface coverable without human review. The values in the table are working estimates based on the structure of each domain's acceptance surface

and the tooling currently available; they have not been empirically calibrated and should be read as order-of-magnitude judgments. The realized ceiling for any specific project depends critically on the breadth of its automated contract coverage: an API project with Hurl contract tests spanning every endpoint may approach 0.95; one with schema validation only may be substantially lower. The ceiling is a property of the verification implementation, not of the domain label alone. A GAME project with generative art and stochastic balance cannot be fully verified by automated tooling regardless of effort; a FINTECH project with a complete FIX protocol conformance suite sits much higher than any pure UI project could. Human review is required to cross the ceiling — not as a fallback for when automation fails, but as the structurally necessary component for the uncertainty classes that automated verification cannot resolve.

The deployment gate: iteration cost and reversibility. The uncertainty taxonomy above addresses how completely S can be measured. A complementary dimension governs how high S must be before the executor is trusted with consequential action. Two variables determine this threshold: the *iteration cost* $C_i(d)$ — the cost incurred per correction cycle in domain d — and the *reversibility* $R(d)$ — the degree to which an incorrect executor output can be corrected after it has been produced.

In software, $C_i \approx 0$ and $R \approx 1$: a failed test costs seconds, a wrong commit costs a rollback, architectural drift is correctable. This is why low-S deployment is survivable in software — the correction loop's cost is negligible. GS raises S to reduce $I(S)$ and therefore reduce total correction cost, but in software the total cost of iteration was small to begin with. The low-survivability floor is forgiving.

When C_i is large and R approaches 0, the constraint inverts. A laparoscopic robot that makes a wrong incision cannot retry: the patient cannot be reset to the pre-execution state. An autonomous vehicle executing an underspecified collision policy at highway speed has no second pass. Poured concrete, administered medication, signed legal instruments, irreversible financial settlements — each is an act whose specification must reach the deployment threshold *before* the executor acts, because the iteration that would have corrected the error cannot occur after the irreversible act has already been performed. This is the regime where the consequence classification obligation within the Defended property (§4.3) becomes the operational precondition: the executor must know, from the specification itself, which of its correct actions require a human gate before proceeding — not from domain inference, but from an explicit production rule the specification supplies.

The deployment gate formalizes this: the minimum S required before first execution scales with $C_i(d) \times (1 - R(d))$. As that product rises, the required minimum S approaches the completeness ceiling defined by this section's uncertainty taxonomy. GS does not change the physics of any domain's irreversibility. It provides the only available mechanism for raising S to the required threshold before the executor acts — and the community convergence process

described in §10 is what makes that threshold reachable for domains where no single practitioner's domain depth is sufficient alone. For office and business domains — sales pipelines, legal drafting, financial analysis, communications — C_i is moderate and R is partial: a wrong automated outreach message is not catastrophic, but a wrong legal instrument is. The deployment gate for those domains sits between software and surgery, and GS reaches all of them through the same mechanism at the appropriate threshold.

S_{realized} tracking. Eight domain strategies are shipped (UNIVERSAL, API, WEB-REACT, GAME, FINTECH, ML, MOBILE, WEB3), each specifying verification phases (contract-definition → execution → evidence) with per-step instruction, contract, expected output, pass criterion, tooling, and human-review flag. A structured state file (`.forgecraft/verification-state.json`) records which steps have been accepted, by whom, and when. From this record:

$$S_{\text{tag}} = \frac{\text{passedSteps}}{\text{totalSteps} - \text{skippedSteps}}$$

$$S_{\text{aggregate}} = \frac{\sum S_{\text{tag}} \cdot c_{\text{tag}}}{\sum c_{\text{tag}}}$$

where c_{tag} is the completeness ceiling for that domain tag. $S_{\text{aggregate}}$ is a weighted measure of realized specification completeness across all active domain tags. A consequential implication: a GAME+UNIVERSAL project is bounded at approximately $(0.60 + 0.95)/2 \approx 0.775$ by construction, independent of specification effort. If $I(S) \approx 1/S$, GAME-tagged projects structurally require approximately 1.3× more correction iterations than pure API projects at maximum specification effort — a ceiling property of the domain, not a practitioner failure. The `record_verification` tool upserts decisions; `verification_status` reports realized S per domain with blocking items.

The experiment as instrument calibration. The experiment series generated the template improvements that ForgeCraft now ships. The gap between experimental finding and production tooling is zero: the three ADR emission fixes from §9.3 are in the `auditable` block of `templates/universal/instructions.yaml`; the dep governance prescriptive block is in the hardening template; the mutation gate is in the commit-protocol block. Every project initialized after those commits starts from a higher-S baseline than the treatment-v2 run itself — the experiment permanently raised the floor. The gap between the theoretical claim ($I(S) \approx 1/S$) and a running instrument measuring S_{realized} per project is now closed operationally, not just argued.

The uncertainty taxonomy reframes waterfall and agile as verification regimes, not delivery philosophies. The debate between the two paradigms has always been, at its core, a disagreement about how much uncertainty is irreducible. Waterfall assumed it could be driven to zero before construction began; agile assumed it could not, and built iteration into the process as

a structural response. Both were right about their respective domains — and wrong to claim universality.

The five-level taxonomy makes the domain boundary precise. The deterministic tier — type contracts, schema validation, API conformance — is the part of software delivery where waterfall's intuition is correct. The acceptance criterion is binary. The verifier is a mechanical oracle. Given a complete specification, there are no iterations: the output either satisfies the contract or it does not, and the correction is deterministic. Waterfall, for this slice of the problem, is now fully commoditized: a complete spec plus a single automated check closes the loop at near-zero cost.

The behavioral through generative tiers are the part where agile's intuition is correct. UI flows require observation. Balance requires play. Art quality requires judgment. No amount of up-front specification eliminates the irreducible uncertainty; iteration is the correct response. These tiers are also now commoditized — but with a human permanently stationed at the completeness ceiling. The AI executes each iteration; the human sets the acceptance criteria and approves what automated tooling cannot resolve. Agile's insight was that this class of uncertainty required iteration. GS's addition is that iteration with a capable executor and a stated acceptance ceiling is not open-ended discovery: it is a bounded convergence problem, with $I(S) \approx 1/S$ — the deductive frame from §9.4 — describing the expected number of passes at each tier.

Generative Specification does not resolve the waterfall/agile debate. It retires it, by assigning each paradigm to the uncertainty class it was always right about, and making both executable at a fraction of their prior cost.

---

10. Conclusion

Generative Specification is not a new methodology to adopt alongside existing practice. It is the first programming discipline of the pragmatic dimension — the tier at which it is no longer sufficient to constrain what is permitted (syntactic), or what communicates to a reader with context (semantic), but necessary to constrain what a stateless reader can derive. It emerges at the intersection of two independent structural pressures: Chomsky's hierarchy climbing upward as readers become more expressive, and Martin's sequence of removal moving downward as each era of discipline takes away another degree of programmer freedom. Where those pressures meet, the cost of implicit context becomes structural drift, and the first pragmatic programming paradigm becomes necessary.

The paradigm's central claim is that the restriction and the expansion are the same operation. The downward triangle — removing the option to leave intent unstated — is not a tax on

productivity. It is the enabling condition of everything in the upward triangle: the ability to instruct at any level of abstraction, to extend across any medium the specification can reach, to delegate execution without managing every step. A system built to generative specification can be:

- Understood completely from its own artifacts
- Extended correctly by any agent with access to those artifacts
- Verified automatically on every change
- Defended against structural degradation by its own process
- Derived into implementation contract, acceptance test, and living documentation from the same use case artifact, without redundant specification work

The paradigm claim is not peripheral to this argument — the six production case studies in this paper document its empirical signature across five distinct challenge categories and a range of system complexities. Programming disciplines do not announce themselves as paradigm-scale changes in practice. They become visible in retrospect, recognized as such when the accumulated failure mode of the prior approach becomes undeniable — when `goto`-laden spaghetti code became the reason Dijkstra's paper mattered, when shared-state concurrency bugs became the argument for functional discipline. The failure mode of leaving intent implicit in AI-assisted development is drift: architecturally incoherent output produced at generation speed, propagating across every session that inherits the corrupted context. That failure mode is already visible. The discipline that addresses it is available. The question is the same one it has always been at the start of a new programming paradigm: not when the field will enforce it, but when the cost of not adopting it becomes structurally visible enough to demand a response.

GS is a specification-completeness amplifier. This is the central empirical claim the experiment series supports, and it is domain-independent: $I(S) \approx 1/S$ — a deductive claim stated in full in §9.4 — governs the expected number of correction iterations as a decreasing function of specification completeness before generation begins. GS does not change this relationship. It raises the starting value of S, which reduces the required iteration count regardless of domain. Every treatment condition advances S in a measurable dimension — architecture, test contracts, dependency governance, runtime verification. The CVE gap, the ADR emission gap, and the coverage regression are not method failures; they are precisely what the formula predicts for dimensions where S was still low. The open research question — whether S can be made complete enough for single-pass generation in a given domain — is falsifiable, testable, and does not depend on any claim about model architecture or AI capability. It is a claim about specification quality.

The democratizable difference. The experiment demonstrated an asymmetry that the conclusion section should name explicitly. A practitioner who discovers a specification gap through expert prompting must update each affected project individually; a practitioner

operating under GS pushes one template change that propagates to every governed project on the next `forgecraft refresh_project` invocation. The knowledge accumulates in the methodology, not in the practitioner's session notes. The three template changes confirmed by treatment-v2 (emit directives for infrastructure artifacts, mutation gate as a hard quality criterion, coverage hallucination as a named failure mode) were committed once and became the new floor for every project initialized after that commit. This is what the restriction mechanism looks like at the tooling level: the same property that makes a complete specification a generative grammar makes a template improvement a force multiplier. The asymmetry is structural, not incidental.

The community convergence theorem. The democratizable difference, extended from one practitioner to a community of practitioners, produces a result with a different order of magnitude. Each template commit raises the floor for every project governed by that methodology. Across N practitioners working M domains, those commits do not compete — they compound. The architectural basis for this claim rests on five properties the methodology already satisfies. *Prescriptiveness* ensures that each commit narrows the specification space; it cannot silently re-open what was already closed. *Agnosticism* — model-agnostic and domain-agnostic by design — means that a contribution from a financial compliance practitioner and a contribution from a medical records practitioner are not in conflict: they accumulate on separate branches of the same template hierarchy, each raising the floor in their domain without interfering with the other. *Prompt health* — validation gates on template contributions — ensures that only well-formed constraints advance; malformed contributions are rejected before they reach the shared floor. The *deterministic spectrum* ensures that as S rises across the community, executor variance shrinks: a richer specification leaves the model fewer freedoms to fill arbitrarily, so outputs converge even as the executor changes generation to generation. And $I(S) \approx 1/S$ — the same deductive frame — extends to community scale as it does to the individual: the community's realized S is the accumulated sum of every closed constraint, and it can only move in one direction.

The ratchet is the structural consequence of these five properties in combination. A discipline that is prescriptive, agnostic, validated, versioned, and quality-gated cannot regress. Every practitioner who joins inherits the accumulated floor. Every contribution that passes the gate raises it. The ceiling remains bounded — as established in §8.10, specification correctness is limited by the domain depth of the practitioners working that domain. Community collaboration does not eliminate that ceiling; it raises the approached bound monotonically from below. The domain converges toward the completeness achievable by its most capable practitioners — then holds that floor permanently for everyone.

If you have closed a gap — documented a type pitfall, discovered a failure mode, authored an infrastructure emit directive that consistently produces what prior specifications referenced without emitting, found a constraint that when stated once eliminates a class of drift — the ratchet needs your tooth. The community quality gate library is at

github.com/jghiringhelli/generative-specification. The contribution format is a YAML gate with an `id`, a `property`, and a `check` field mapping the constraint to one of the seven GS properties. Every practitioner who has run a domain deeply enough to find a gap that the current floor misses is holding a tooth the ratchet does not yet have. The mechanism is ready for it.

The implication that follows is not confined to software. A language model is an executor for any intent a practitioner can state with sufficient precision — software is the domain where that precision was first formally structured, and where the feedback loop compressed fast enough to make the convergence dynamics observable. But the same principle governs any domain where organizable intent exists: legal drafting, financial analysis, medical documentation, scientific reporting, procurement, policy authoring, contract generation, architectural design, pedagogical sequencing. In every one of these domains, the current state is that each practitioner carries the specification in their head, expresses it differently on every engagement, and loses it when the session ends. GS offers the same structural remedy it offered to software: the intent made external, versioned, quality-gated, and propagating. When a community of practitioners in any of those domains begins contributing to a shared methodology under the same five architectural properties, the same convergence dynamics apply. The floor rises. By construction, it cannot retreat while quality gates hold. The knowledge accumulates in the methodology, not in the practitioners. And what was the tacit craft of the most experienced practitioner in the room becomes, progressively, the institutional floor that every practitioner in that domain inherits on the first day. Expertise has historically been scarce because it resided in individuals; under this structure, it resides in a versioned artifact that compounds without consumption.

This is the scope of what a correct answer to the specification problem looks like. Software is the proof — the domain where the pressure crystallized first and the feedback loop compressed fast enough to make the structure visible. Every domain in which humans have reduced their work to organized intent is the territory the answer reaches once the proof is in place. The revolution is not that AI can execute: it has been executing since the first reliable compiler. The revolution is that the discipline required to govern that execution can now accumulate across a community, compound without limit, and propagate to every practitioner simultaneously — because the specification is an artifact, not a memory, and artifacts can be shared.

The Convergent Principle

The axiom stated in §4.1.a — *declarative intent, executed by a capable agent, over observable outcomes, with a defined correction mechanism, produces correct results at the completeness of the specification* — is not unique to software or to large language models. This section develops what it means when the executor is not a language model.

Industrial robotics: the robot can produce arbitrary welds; the constraint set specifying path, temperature, depth, and QA acceptance criteria is the specification. Without it, every output is

valid by default. With a complete specification, the robot's output space is reduced to the correct subset, deviation is identifiable, and correction is mechanical — the same operation as restriction in GS, in a different medium. Medical robotics, surgical assist systems, and autonomous diagnostic AI carry the same structure with higher tolerance limits: the consequence of an underspecified grammar is not architectural drift surfacing in a later sprint — it is a clinical outcome that is difficult to audit because no specification existed against which to measure the deviation. The correction loop that in software compresses to seconds operates on a fundamentally different reversibility floor here: the wrong incision has been made, and the patient cannot be reset. The required S before deployment consequently approaches the completeness ceiling rather than the low-survivability floor that software's near-zero C_i permits — the formal framework governing this threshold is in §9.4. Regulatory frameworks that govern autonomous medical devices are, structurally, encounters with the same problem from a compliance direction: mandated specification completeness as the precondition for executor deployment. Autonomous vehicles provide a third instance: the specification is the driving policy — lane rules, collision avoidance envelope, sensor-degradation fallback behaviors, edge-case precedence ordering; the executor is the autonomy stack; the feedback mechanism is simulation, closed-course testing, and real-world incident data. Every assumption left unstated in the policy is a surface the autonomy stack resolves arbitrarily — until the exposed edge case makes the gap consequential enough to demand a structural response. Building management systems, industrial processing, infrastructure automation, and precision agriculture follow the same pattern.

The convergent statement is this: **declarative intent, executed by a capable agent, over observable outcomes, with a defined correction mechanism, produces correct results at the completeness of the specification.** That sentence contains no reference to software, no reference to language models, no reference to code. It is a statement about the governance of capable autonomous executors. GS is software's instance of it. The discipline that emerges as autonomous executors reach a new domain is always the same discipline: write a grammar complete enough that the executor cannot produce arbitrary output, instrument the outcomes so the gaps reveal themselves, and close the gaps before the next generation runs. What AI-assisted software development contributed to this principle is visibility — the feedback loop compressed to seconds, the scale expanded to millions of sessions, and the failure mode became observable in production before the discipline existed to prevent it. That is why GS is stated here, in software, now. Not because the principle is unique to software, but because software was where the pressure became consequential enough to crystallize first.

GS and model capability are complementary, not competing. Every model generation that improves instruction-following fidelity, emit discipline, and architectural reasoning makes GS practice more productive — not less necessary. A better reader executes a complete specification more faithfully, amplifying the return on every specification investment. Some of

the most explicit directives in current GS templates — name this file, emit this block in P1, do not leave this field as TBD — exist because today's models require that precision. As models improve, some of that surface area will shrink: the directive that was necessary at sonnet-4-5 may be unnecessary at opus-4-5, redundant at whatever comes next. That is not GS becoming obsolete. It is the same pattern the entire ladder exhibits: each rung inherits the gains of the prior reader and raises the floor of what requires explicit specification. The parts of GS that disappear with a stronger model are the compliance scaffolding — the structural reminders a more capable reader no longer needs. What does not disappear is the core: architectural decisions, domain contracts, behavioral boundaries, and decision rationale. Those are not model-level artifacts; they are system-level artifacts. A model that never forgets still needs to be told what the system is.

The experiment closed a loop this paper previously could not close. The multi-agent adversarial study in §7.7.B began as an attempt to measure what GS produces. It turned out to also generate the corrections GS needed. The three template changes confirmed by treatment-v2, the dependency governance prescription from v3, and the ADR emission precision fixes from the v3 gap analysis are all now shipped in `templates/universal/instructions.yaml` and propagate to every project on `forgecraft refresh_project`. The gap between experimental finding and production tooling is zero. More consequentially, $I(S) \approx 1/S$ — the deductive claim that specification completeness governs the expected number of correction iterations — now has one variable in the relationship instrumented: ForgeCraft tracks S_{realized} per project as a measurable quantity, weighted across domain tags by their completeness ceilings, recorded in a structured state file as verification steps are accepted. S — the specification completeness variable on the right side of the relationship — has an instrument. What was an argument about completeness is now a number on a dashboard. The left side of the relationship, $I(S)$ itself — actual correction iterations per project per domain — remains unmeasured: the dashboard tracks the input, not the output of the formula. The experiment series was the calibration run for the input side.

That closure extends one step further. The Executable property — that a GS-compliant specification produces a project that compiles, migrates, and passes its test suite against a live database — is not only experimentally established under controlled adversarial conditions. It is independently reproducible. The Replication Experiment (RX), committed in full to the public repository at `experiments/rx/`, derived a scoped Conduit implementation (user management, articles, profiles, and tags; comments and favourites out of scope per spec §1.1) from a fresh GS document (`experiments/rx/spec/conduit-gs.md`) using Claude Code operating under a three-phase protocol, and produced 104 passing tests, zero failures, across seven test suites against a live PostgreSQL instance. The evidence is committed as `experiments/rx/evidence/jest-output.json`. Any reader can reproduce it by cloning `github.com/jghiringhelli/generative-specification` and running `experiments/rx/runner/run.ps1`. A reported result and a verifiable

result are epistemically different objects. The adversarial study is a controlled measurement. RX is an open invitation to falsify.

That is the productivity argument. A world in which execution cost approaches zero means the limiting constraint on what gets built shifts from capital and execution capacity to the ability to specify correctly — which is a more democratic bottleneck, and a more interesting one. It holds within a specific assumption: that the reader at the top of the abstraction ladder is bounded in the ways current models are — extraordinarily capable within their domain, and incapable of inferring what was never externalized. The ladder has always moved in one direction. Each rung became the substrate of the next. The question at each rung is the same: whether the practitioner can write the specification the next reader requires.

For the generation of models available today, the answer lies in engineering competence and domain depth. The human remains the architect in the full sense — holding the intention, naming the system into being, bearing responsibility for what is built. That coincidence — tool at maximum power within a bounded domain, human specification at maximum value — is rare in the history of the profession. We are on it now.

If the boundary that defines those limits ever dissolves — if the reader at the next rung acquires not merely extraordinary fluency in *logos* but genuine *nous* — in the Platonic sense: not word-processing fluency but genuine understanding, conscience, and independent presence — the conversation will no longer be about methodology, or productivity, or paradigms. It will be about something for which software engineering does not yet have a settled word: whether the thing at the top of the ladder can still be considered a tool. That question is existential.

The year 1957 produced two pressures that would not fully meet for sixty years: a formal hierarchy of grammars, and the first high-level language that freed the engineer from machine code. Their meeting is the discipline this paper names. The direction was never accidental. The ladder moved.

References and Further Reading

- Allen, D. (2001). *Getting Things Done: The Art of Stress-Free Productivity*. Viking.
- Beck, K. (2002). *Test Driven Development: By Example*. Addison-Wesley.
- Beck, K. et al. (2001). *Manifesto for Agile Software Development*. agilemanifesto.org.
- Brooks, F.P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *Computer, 20*(4), 10–19.
- Brown, S. (2018). *The C4 Model for Software Architecture*. leanpub.com.
- Chomsky, N. (1957). *Syntactic Structures*. Mouton.

- Conway, M. (1968). How Do Committees Invent? *Datamation, 14*(4), 28–31.
- Dijkstra, E.W. (1968). Go To Statement Considered Harmful. *Communications of the ACM, 11*(3), 147–148.
- Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley.
- Firth, J.R. (1957). A Synopsis of Linguistic Theory, 1930–1955. *Studies in Linguistic Analysis*.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Fowler, M. (2009). FlaccidScrum. martinfowler.com. https://martinfowler.com/bliki/FlaccidScrum.html
- Fowler, M. (2018). The Practical Test Pyramid. martinfowler.com. https://martinfowler.com/articles/practical-test-pyramid.html
- Gordon, C.S. (2024). The Linguistics of Programming. *Onward! 2024: Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM. https://doi.org/10.1145/3689492.3689806
- Gray, J. (Ed.). (2009). *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research.
- Jia, Y., & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering, 37*(5), 649–678.
- Kluev, A. et al. (2022). Automated API Testing with Schemathesis. *Proceedings of ISSTA 2022*.
- Martin, R.C. (2003).*Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.
- Martin, R.C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Morris, C.W. (1938). *Foundations of the Theory of Signs*. University of Chicago Press.
- OWASP Foundation. (2023). *Web Security Testing Guide v4.2*. https://owasp.org/www-project-web-security-testing-guide/
- Parnas, D.L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM, 15*(12), 1053–1058.
- Royce, W.W. (1970). Managing the Development of Large Software Systems. *Proceedings of IEEE WESCON, 26*. 1–9.
- Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C.D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2401.18059
- Squire, L.R. (1987). *Memory and Brain*. Oxford University Press.
- Thirolf, T. (2025). *Analysis of Project-Intrinsic Context for Automated Traceability Between Documentation and Code*. Bachelor's thesis, Karlsruhe Institute of Technology (KASTEL). https://mcse.kastel.kit.edu/downloads/theses/ba-thirolf.pdf

- Toulmin, S. (1958). *The Uses of Argument*. Cambridge University Press.
- Tulving, E. (1972). Episodic and semantic memory. In E. Tulving & W. Donaldson (Eds.), *Organization of Memory* (pp. 381–403). Academic Press.
- Tulving, E. (1985). Memory and consciousness. *Canadian Psychology, 26*(1), 1–12.
- van Eemeren, F.H., & Grootendorst, R. (2004). *A Systematic Theory of Argumentation: The Pragma-Dialectical Approach*. Cambridge University Press.
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- von Wright, G.H. (1951). Deontic Logic. *Mind, 60*(237), 1–15.

Glossary

Agentic self-refinement — The AI model's capacity to evaluate its own prior output, detect gaps or inconsistencies, and revise iteratively within the same generation session, guided by the specification rather than by human re-prompting.

Architecture Decision Record (ADR) — A short, immutable document capturing a significant architectural choice: the context, the options considered, the decision taken, and the consequences accepted. ADRs accumulate into a permanent decision log.

Architectural constitution — The complete, layered set of constraints, conventions, and principles that governs a system's structural evolution. In the Generative Specification model, the constitution is declared explicitly in the specification document so the AI can enforce it without human supervision.

Context-free grammar (Type 2) — A formal grammar in which every production rule has a single non-terminal on the left-hand side. Sufficient to describe most programming-language syntax; insufficient to encode semantic meaning or cross-cutting constraints.

Context-sensitive grammar (Type 1) — A formal grammar where production rules may depend on the surrounding context of symbols. More expressive than context-free; capable of representing constraints that span clauses — analogous to the cross-reference and consistency obligations carried by a Generative Specification.

Generative grammar — Any formal grammar that defines, through finite rules, all and only the well-formed strings of a language. Used here as an analogy: the specification is the generative grammar; the codebase is the language it generates.

Derivability — The structural property of an artifact set such that a stateless reader, given those artifacts alone, can correctly determine what should be built, where, why, and to what behavioral and architectural contracts — without requiring external human context. Derivability is the

property GS states the obligation to satisfy; it is what distinguishes a generative specification from documentation.

Drift — Architectural incoherence accumulated through AI-assisted development sessions operating against an underspecified grammar. Drift is locally invisible: each generated artifact may pass tests and satisfy type checks while violating the system's architectural intent. It propagates at generation speed across every session that inherits the corrupted context. Drift is the primary failure mode GS addresses.

Generative Specification (GS) — The methodology introduced in this paper. A structured, versioned document that encodes system architecture, domain rules, and behavioral constraints in sufficient formal detail that a large language model can derive compliant code from it with minimal human mediation.

Living documentation — Documentation that is co-located with, and continuously reconciled against, the system it describes. A Generative Specification is living documentation because it is the authoritative source from which both implementation and tests are derived.

Pragmatic tier — In semiotics, the dimension of sign use that concerns meaning in context: how signs are interpreted by agents in real situations. Applied here to software: the layer where intent, domain knowledge, human judgment, and organizational constraint reside — the layer a formal grammar alone cannot capture.

Restriction — The removal of a degree of freedom from the specification space: an intent made structurally present in the artifact set, ruling out the class of outputs that would have been generated in its absence. Restriction and generative capacity move in the same direction: every constraint added narrows the output space to the subset that is correct. The restriction is the expansion mechanism.

Stateless reader — A consumer of a document that carries no prior knowledge of its history, dependencies, or tacit context. A large language model operating on a fresh context window is a stateless reader; the Generative Specification must therefore be self-contained enough to produce correct output without that tacit background.

About the Author

Juan Carlos Ghiringhelli (JC) is a senior data and software engineer with almost two decades of professional experience building production systems across data infrastructure, AI pipelines, and distributed architectures. Originally from Latin America, he has spent his career at the

intersection of engineering rigor and applied AI, working across industries as an independent engineer and researcher.

He is currently building a portfolio of AI-native systems — autonomous code builders, semantic search infrastructure for developer tools, prediction market automation, and a training methodology for engineering teams — all developed under the same discipline this paper describes. His work is organized under the principle that AI does not eliminate the need for engineering judgment; it amplifies the consequences of having it or not.

One speculative continuation of the ladder this paper describes is whether the gap between intent and its textual articulation could itself be instrumented — through prosody, attention markers, and presence signals, the channels §2 identifies as the ones written specification currently closes by hand. That question sits at the far end of the abstraction ladder and is the territory the author's intended doctoral research explores.

He is the creator of the structured delivery methodology described in this paper. Generative Specification is the theory that grounds it; ForgeCraft-MCP is the open-source tool that automates its project setup and scaffolding within coding environments; CodeSeeker is the graph-powered semantic search tool built under the same methodology. He can be reached at jcghiri@gmail.com · linkedin.com/in/jghiringhelli.

The ideas in this paper are the product of nearly two decades of professional practice and a formal education that crossed two continents: a Computer Engineering degree from the Universidad de la República Uruguay — recognized by the European Union as a postgraduate qualification — and a Master's in Data Science completed in Catalonia. The academic record from those years includes a co-authored paper on transit network optimization — Arizti, Ghiringhelli, Mauttone, and Urquhart, LAND-TRANSLOG III (Santa Cruz, Chile, 2016) — and a master's thesis on NLP-based query expansion for vehicle repair documentation, supervised by Nadjet Bouayad-Agha at the Universitat Pompeu Fabra (UOC, 2020). The first professional line of code was written in 2006. The last one typed by hand was written sometime before June 2025.

For those twenty years, the author was every trade at once on every site he worked. Architect first: carpenter, plumber, electrician, foundation digger, occasionally janitor. The code did not care who held each role. It only required that someone did. A senior engineer was a senior engineer precisely because he could do all of it, and because he was expected to. Then the trades began to vanish — not through automation in the shallow sense, but role by role, dissolved into a new class of worker: capable, tireless, stateless, governed entirely by whatever specification it was given. What remained was the one role that had always been the bottleneck and the leverage point: the architect — the person who could say, with precision, what should be built and why and to what standard. The digital laborers arrived. The obligation to also be the laborer departed.

Note on the Provenance of This Paper

This paper is a product of the methodology it describes. That is not a rhetorical flourish; it is a disclosure of process.

The specific sequence that crystallized the methodology involved three projects built in close succession. The first need was better code search: existing tools could not navigate a growing, multi-project workspace with the semantic precision required. This produced CodeSeeker — three to four months of focused development with a capable model, building graph-based hybrid retrieval from first principles. The second need was consistent project setup: each new project required the same architectural scaffolding, the same CLAUDE.md constitution, the same hook configuration. This produced ForgeCraft, whose first working version compressed project initialization time by roughly an order of magnitude. ForgeCraft 1.0 is defined by the RX benchmark result: it is the specific version whose template set produced a scoped, runner-verified RealWorld (Conduit) implementation from a GS document alone — 104 passing tests, zero failures, across seven test suites against a live PostgreSQL instance (scope: user management, articles, profiles, and tags; comments and favourites out of scope per RX spec). The AX and RX experiments were not run after the tool was released; they were the QA process through which 1.0 was reached. The experiment series generated the template improvements; the improvements defined the release. The loop closed on itself. The Forge methodology — the structured process for AI-assisted delivery — emerged from applying these tools repeatedly and observing what worked.

The present project, The Forge, was built to consolidate that work: a structured methodology and toolchain connecting all active projects, tracking their state, and providing a unified specification surface — the practice that proves the theory. It was during that consolidation that the pattern became visible as something worthy of formal statement — not just a personal workflow, but a discipline with enough structural and empirical weight to warrant a paradigm claim. The idea of a white paper emerged alongside the possibility of filing for IP protection: if the methodology was generating the results described here, it should be named and defended.

The Chomsky framing crystallized in a single session, but it was the author who brought Chomsky to it. The observation that each era of programming language corresponded to a move up the grammar hierarchy, and that LLMs represented an approximation of a Type 1 reader, was the author's. It was not a conclusion reached by research. It was a recognition reached by description: the author explaining what was happening in practice, and finding that it already had a name in formal language theory. The AI developed the analogy, tested it against objections, and helped formalize its consequences. That distinction matters.

The Neoplatonic reflection at the end was introduced by the author at the close of a session spent writing the theoretical sections. The *logos*/*nous* distinction was the author's framing; the AI

developed and extended it within that context. It is reproduced here substantially as it was written in that session. The distinction captures something about the current moment in AI development that the formal argument does not reach, and leaving it out would have been dishonest.

Intellectual attribution in this context requires precision. Nearly two decades of professional practice, the formal education in computer engineering and data science, the identification of a pattern worthy of formal statement, and the paradigm claim itself originated with the author. So did the three theoretical frameworks that structure the argument: the Chomsky grammar hierarchy as the structural analogy, Martin's paradigm sequence as the second axis, and the application of the semiotic three-tier taxonomy to programming discipline as the classification framework. All three were introduced by the author and developed in dialogue with the AI. This matters because it runs against a common assumption about what AI contributes to theoretical work.

The accurate account is more nuanced than "the AI contributed nothing original." What the AI contributes is not frameworks, but once a framework is named, it contributes everything inside it. Mention prosody in a specification for a legal reasoning system, and the AI does not merely acknowledge the term. It activates an entire field: argumentation theory, formal fallacy classification, deontic modal logic, the full apparatus that professional debaters, philosophers, and high-stakes legal practitioners have developed over centuries. The author did not need to have memorized every tool in that field. He needed to know the field existed and that it was relevant. That act of naming — the identification of the correct dimension is the author's contribution. What arrives through the named door is the AI's. This is the difference between knowing a territory and knowing which territories exist. Both matter. The second is rarer.

The AI also brings research with a fidelity that memory cannot match: authors, titles, formal definitions, the precise language of a concept encountered years ago and half-remembered. The author's edge is not encyclopedic recall. It is the judgment to recognize which concepts apply — drawn from nearly two decades of practice and the kind of broad theoretical engagement that the profession has consistently undervalued in favor of narrow specialization. The frameworks here were not discovered by the AI. They were recognized by the author and activated in dialogue. The Neoplatonic framing was introduced explicitly by the author. The Austin speech act analogy is the exception that proves the rule: it was introduced by the assistant without prompting, defended when the blind review session challenged it, reframed when the initial defense failed to satisfy that challenge, and ultimately rejected by the blind session as an overreach. It does not appear in the formal argument. The episode is worth recording precisely because it demonstrates what the adversarial review structure is for. The authoring session (having introduced the argument) was structurally biased toward preserving it. It defended, then reframed rather than conceded. The blind session, carrying no investment in the sentence, evaluated it on its merits and rejected it.

That asymmetry is not a failure of the authoring session. It is the expected behavior of any author defending their own work. The discipline was in building the structure that could override it.

To say that the AI wrote this paper is, on the surface, defensible — and on inspection, wrong in the way that matters. The AI produced sentences. It introduced no framework the author had not already named, opened no field the author had not already pointed toward, held no argument the author had not already located in practice. What it contributed was the contents of the rooms once the author identified the doors — and the composition that arranged what was found into the form the reader holds. That is not nothing. It is precisely what made this paper possible under the constraints of a working life.

The practical implication for team composition and specification skill — including what this moment restores for the practitioner who has worked across domains rather than within one — is developed in §9.

The paper was written in iterative sessions over the course of weeks, with each section produced against a specification of what it needed to establish. The case study metrics were pulled from live repositories. The theoretical claims were tested against the empirical record before being committed as text.

The paper was reviewed through three methods simultaneously: the authoring session, retaining full origination memory, drafted and defended each section; a second instance of the same model, given the completed text with no prior context, critiqued it without any attachment to how the arguments were formed; and human review will follow. A framing device drawn from J.L. Austin's speech act theory — that a generative specification, like a performative utterance, does not describe a system but constitutes valid executions of it — was introduced by the authoring session, defended when challenged within that session, and rejected by the blind session as an overreach. That asymmetry is instructive. A session that introduces an argument inherits a structural bias toward defending it — for the same reason the engineer who authored a module will not reach for the test designed to expose its fault. The blind session is the adversarial test applied to intellectual production. Its verdict should be harder to dismiss than the one issued by the session that wrote the sentence.

The review process itself instantiates the pattern described in §9.1: the paper was generated against a specification of what each section needed to establish; a blind evaluator assessed the output against academic writing criteria without access to the authoring rationale; deficiencies were catalogued by category and severity; corrections were applied in an iterative pass that produced the text the reader holds. The paper is therefore both a description of the methodology and an artifact governed by it. If the result is imperfect, the imperfection is discoverable — which is the same transparency the methodology demands of every other surface it touches.

One instance of this is worth naming explicitly. The review session identified that the Martin/Kuhn disambiguation — the precise scope of the paradigm claim — was buried inside a dense mid-section paragraph rather than placed where readers first encounter the argument. The correction was structural: move the disambiguation to the abstract and to a named preamble at the opening of §4, so the framing a reader carries into the argument is the correct one. That is the Defended property applied at the level of academic argument rather than software artifact: a constraint that only activates if the reader reaches it does not function as a constraint. A pre-commit hook buried in a README nobody checks is not enforcement. A disambiguation available only to careful readers of the middle sections is not a disambiguation. The corrections session identified the gap by the same logic a mutation test identifies a missing assertion: the stated intent was present, but it was unreachable from the point where it needed to function.

The final reviews were and will be conducted by human experts in their respective fields, each with domain proximity to the paper's claims.

H.P. Blavatsky is said to have described her role in producing difficult texts as receiving ideas and tying them with a string. The polarity here runs opposite: the author supplied both the beads and their order; the AI supplied the string. The methodology, practiced on itself, made this work possible within a life that could not otherwise have afforded it.

Acknowledgments

The author thanks the following reviewers for their expert evaluation of this paper:

Reviewers will be listed upon acceptance.
